INFO 4555: Business Intelligence Systems
Assignment 4: WhereScape RED Tutorial

For the WhereScape assignment, you will be going through a WhereScape RED hands-on tutorial. WhereScape Software gave our class permission to use their professional training materials and update them as necessary to fit the needs of our course. This assignment is a portion of the full WhereScape RED developer training module (approximately ¼ of the total material). The material for this assignment has been reduced to give students the skills necessary to complete their team projects, without going into details only needed for advanced production environments.

The grading of this assignment will be based on the completion of the WhereScape generated data model diagram (at the end of tutorial) and the existence of all of the tables and data generated by the tutorial in your netid database schema. Complete the tutorial and take advantage of the opportunity to learn the base WhereScape skills you will need to complete your team project work.

Ideally you can use a second device (or two monitors) for viewing this document as you proceed through the tutorial using your AWS AppStream.

Before you can proceed with this tutorial, you will need to setup your WhereScape environment by going through the steps outlined in the document "**WhereScape Repository Setup.pdf**".

RED Design Interface

WhereScape RED has a number of different windows/tabs that are utilized in the building and maintenance of a data warehouse.

Each window/tab may in some cases be broken into panes.

There are four main windows/tabs that are used extensively in the building of a data warehouse.

These are:

- Builder window
- Scheduler window
- Diagram window
- Procedure Editor window

But before any of these windows/tabs can be accessed, the user must Log in using the Login Window

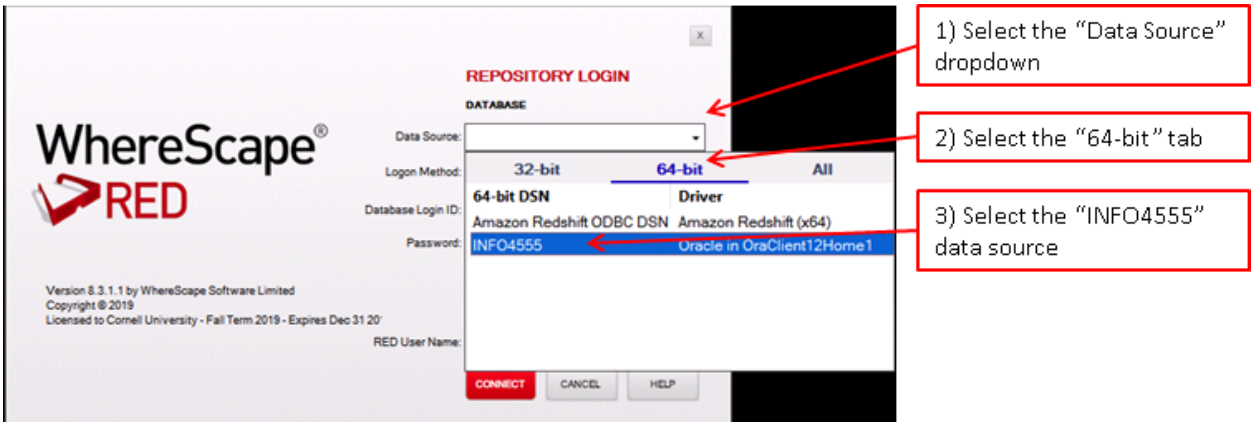
Logging In

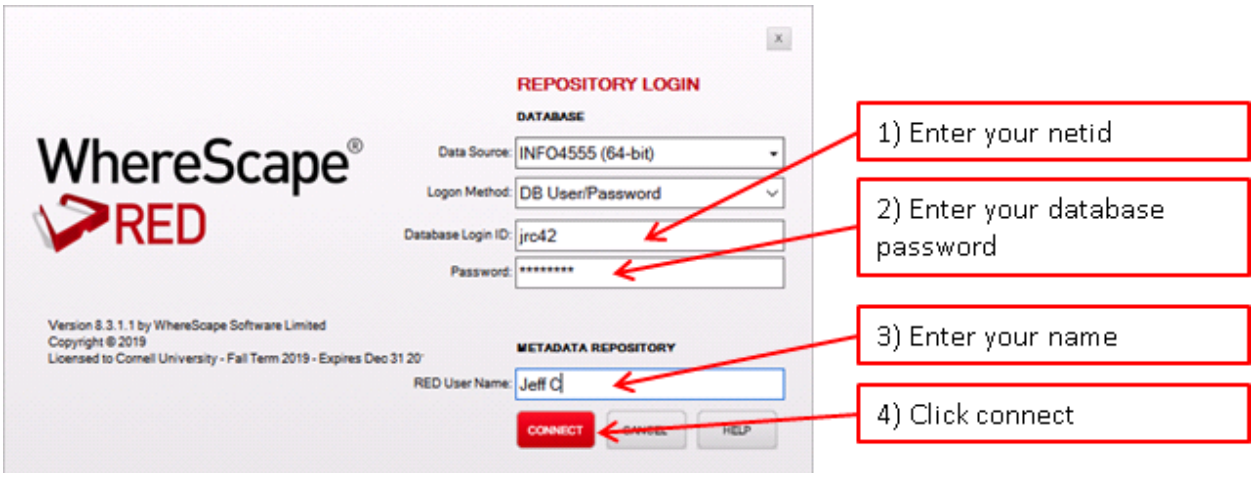
Having installed and configured WhereScape RED via the "WhereScape Repository Setup.pdf" you are ready to begin the tutorial.

INFO 4555: Business Intelligence Systems
Assignment 4: WhereScape RED Tutorial

To log in:

- First log on to your Amazon AppStream.
- Next, Click on the WhereScape RED icon
- The **Repository Login** screen displays.





Properties Definition: Login Screen

The Repository Login Screen is split into 2 functional areas; the DATABASE section to specify the Oracle logon to our RED Environment and the METADATA REPOSITORY section for Audit Purposes to track your development activities.

DATABASE

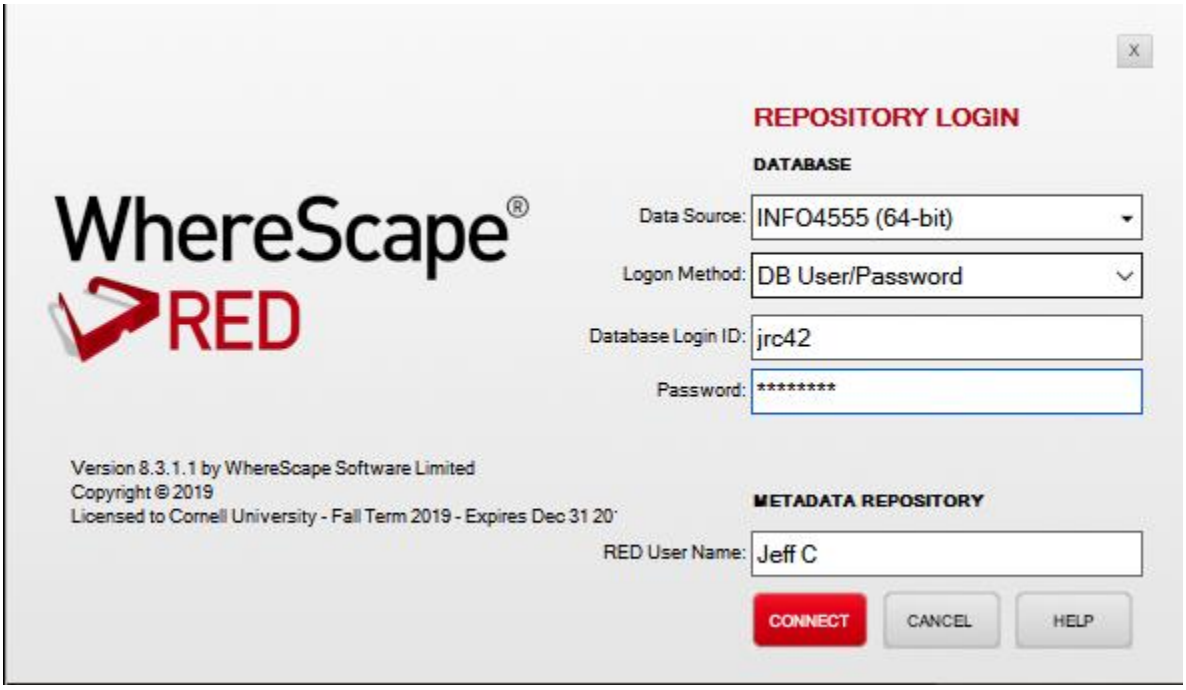
- The **Data Source** is the ODBC Administrator's Data Source Name that links to the WhereScape RED Repository (the Oracle database that we specified on the install where all of our internal definitions are held, as well as some or all of our data warehouse objects). The drop-down will show you the list of appropriate data sources. For this class the Data Source is "INFO4555"
- **Logon Method** determines how we log on to the RED environment.

- **DB User/Password** will have us enter a suitable **Database Login ID** & **Password** combination that allows us access to the data source above.
- **OS/DSN Authentication** uses the Trusted/Single-Signon Method such that the (already authenticated) security profile that you logged on to the PC with will be used to log on to the RED environment. Here, the Database Login ID & Password will be greyed out.

METADATA REPOSITORY

- The **RED User Name** is the name that will be associated with any procedures, tables, etc, and scheduled jobs that are created from within WhereScape RED. Normally this would be your full name. It is not a security profile but an auditing mechanism with great benefits in a multi-developer environment. This will tag your work done within WhereScape RED with your name. Helpful when multiple developers are in play.

- For our class, we are using "DB User/Password" logon method. The Database Login ID will be your netid database account we set up for Assignment 3: SQL.



- Enter your **First Name** and **Surname/Family Name** in the **RED User Name:** field and click **Connect**.

The **Builder** screen displays.

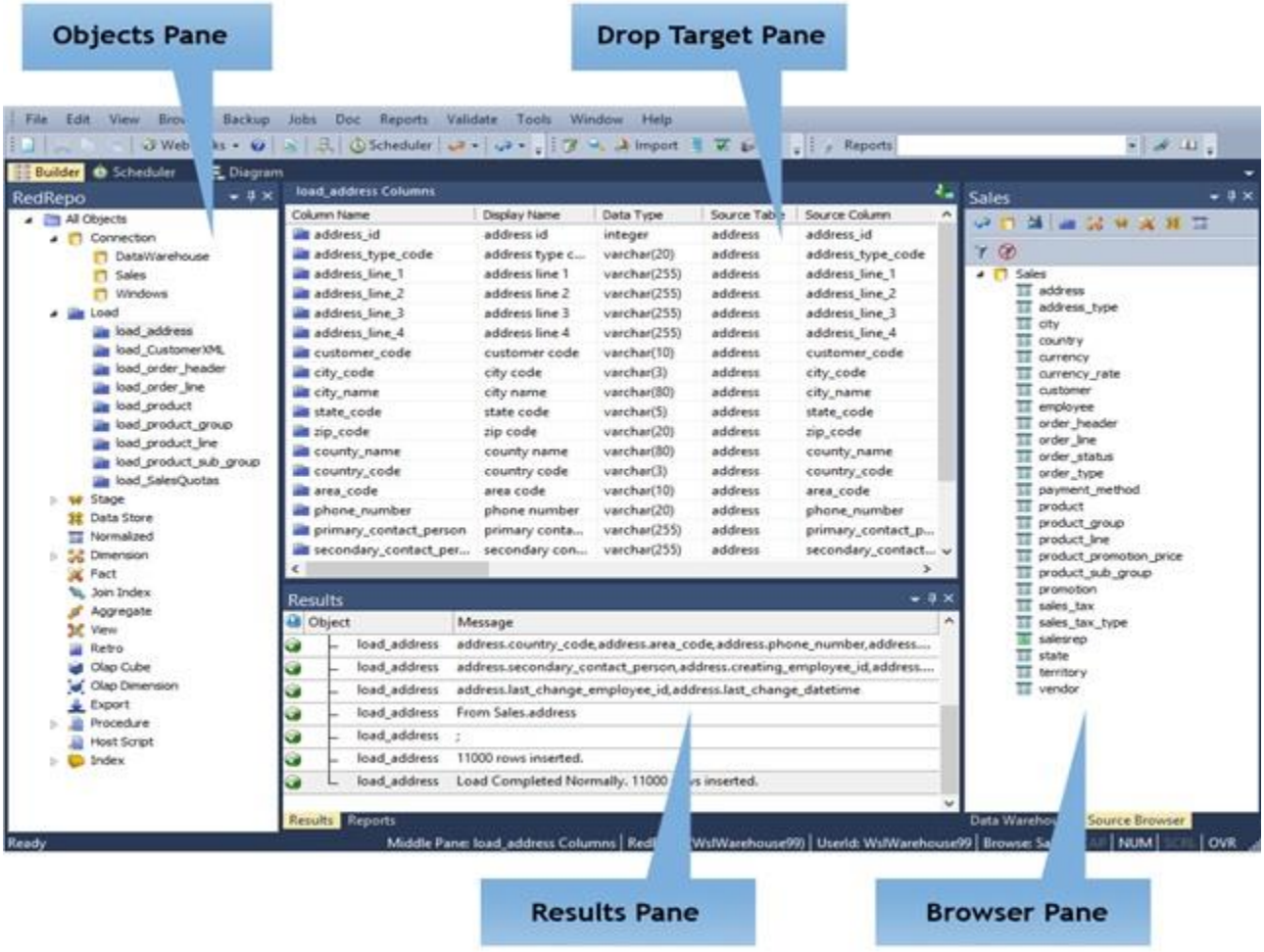
In all cases, the next time you log on to RED, these options will be retained so that your selection is made only once.

If a password is required, RED will NOT remember this password and you will need to enter it.

Security is very important in our data warehouse.

RED Builder Window

The Builder window/tab has 4 standard panels:



The left pane, the **Objects Pane** contains all the objects within the metadata repository.

These objects are stored in object groups (e.g. Stage Tables).

The object groups are in turn optionally stored in Projects, and the Projects are optionally stored within Project Groups.

The middle pane, the **Drop Target Pane** or **Work Pane,** is used as the drop target in drag and drop operations.

It is also used to show the results of various queries on both the metadata and the underlying source and database tables.

The status line at the bottom of the screen displays the current contents of the middle pane.

The right pane, the **Browser Pane**, allows us to browse our **Connections**.

It is further split by tabs along the bottom to show a source connection as well as the contents of the data warehouse connection.

Typically this pane is used as the source of information in the drag and drop operations. The status line at the bottom of the screen displays the connection used in the right pane.

The bottom pane is the **Results Pane** and it shows the results of any command executed on an object.

Multiple messages can be displayed but only the most recent is displayed in full. Expand the '+' sign next to an object name to see the complete list of messages relating to the object.

The results pane is also split into two with tabs along the bottom.

When a report is run from the main menu, the results are displayed in the Reports tab of the Results Pane.

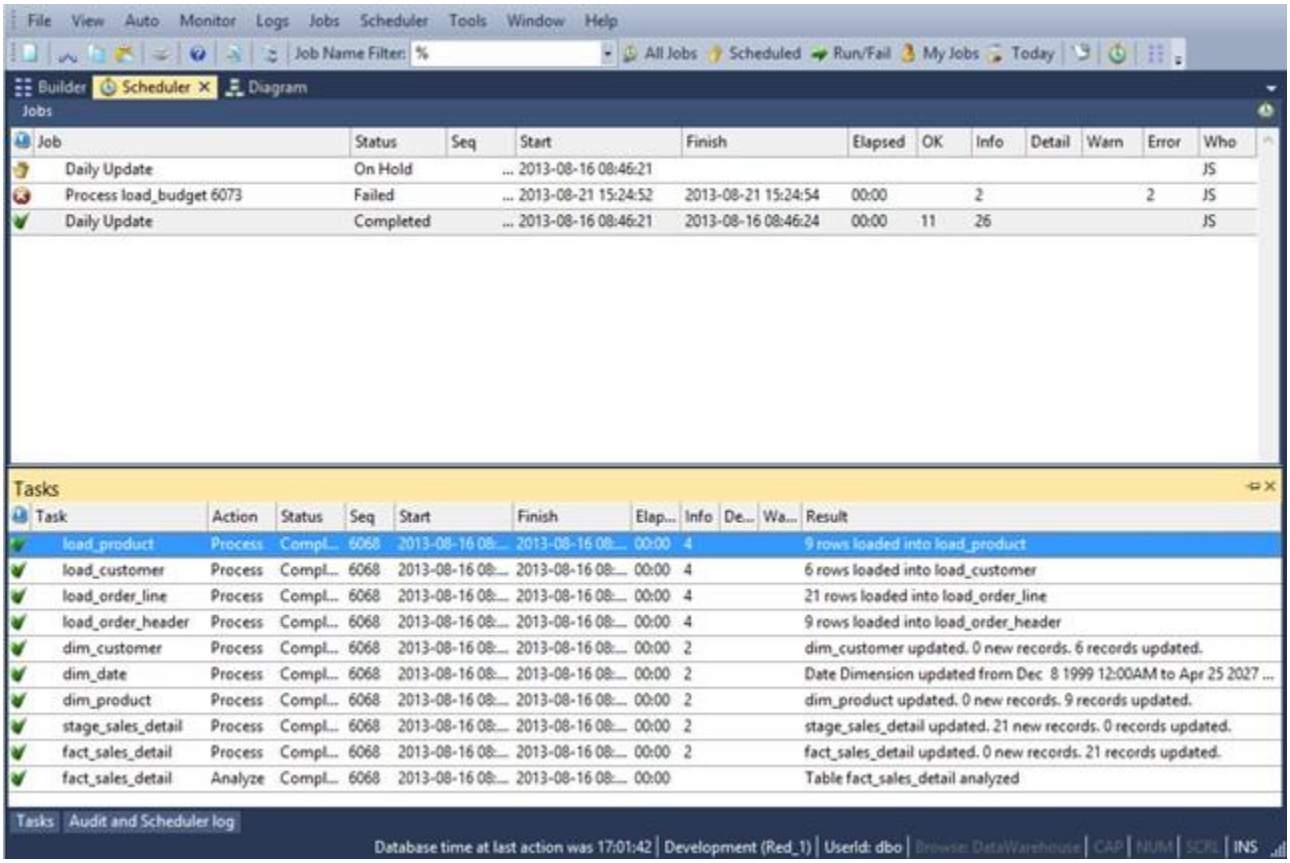
In all Panes & Windows, context specific Pop-up menus assist by showing the most likely menus for the work you are doing.

F5 or **Ctrl+R** can be used to refresh either of the browser panes when the cursor is positioned within these panes.

The left, bottom and right panes can be dragged out of their docking places and docked elsewhere. Docking handles appear when a pane is dragged.

RED Scheduler Window

The Scheduler Window is used as the main interface to the scheduler.



Jobs can be scheduled, monitored, edited and deleted through this window.

The window consists of two panes.

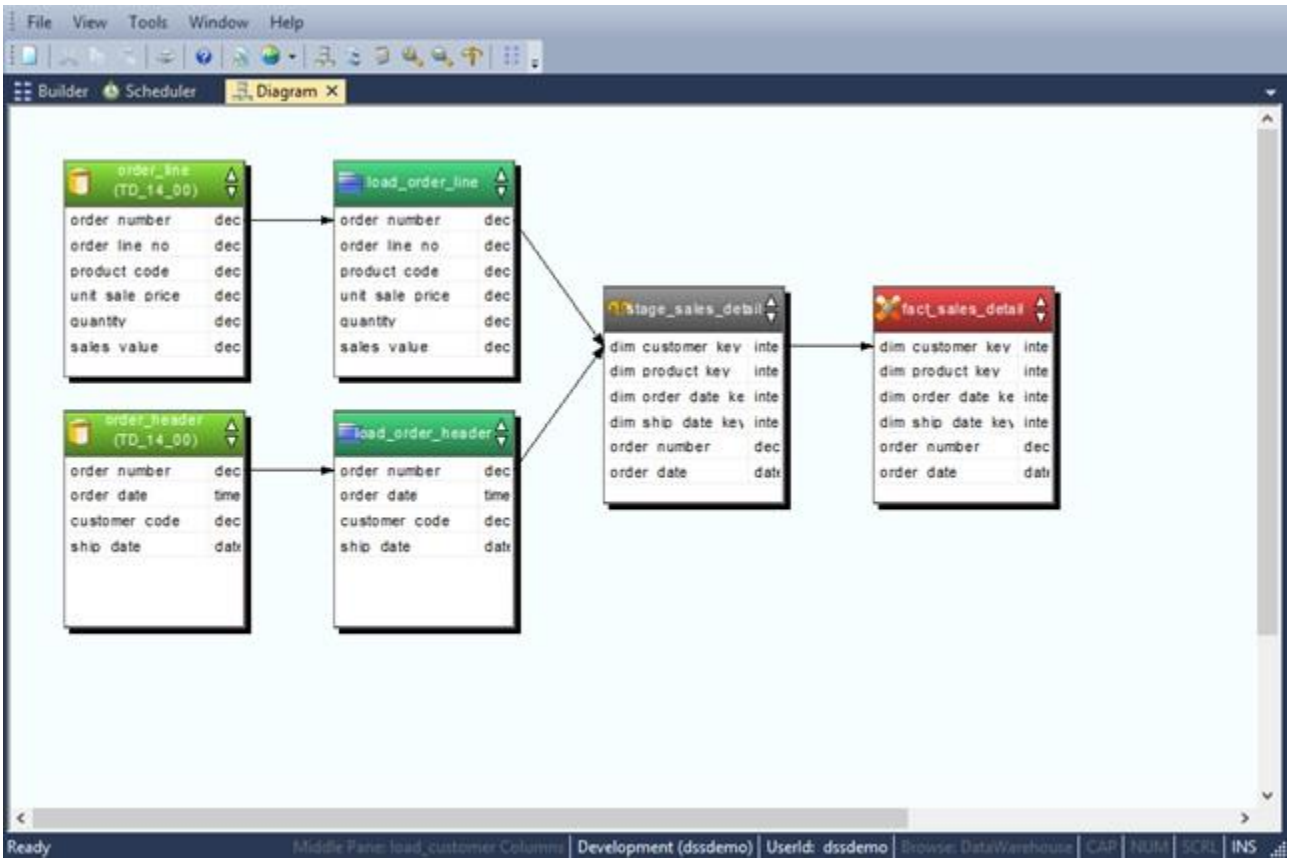
The toolbar provides a quick way to display various job selections in the top pane.

Double-click on the job in the top pane to see the tasks of the selected job in the bottom pane.

Double-click on the task in the bottom pane to see the audit trail displayed in a separate tab in the bottom pane.

RED Diagram Window

The Diagram Window is used to display the tables of the data warehouse in diagrammatic form, showing the various sources or targets of the selected object.

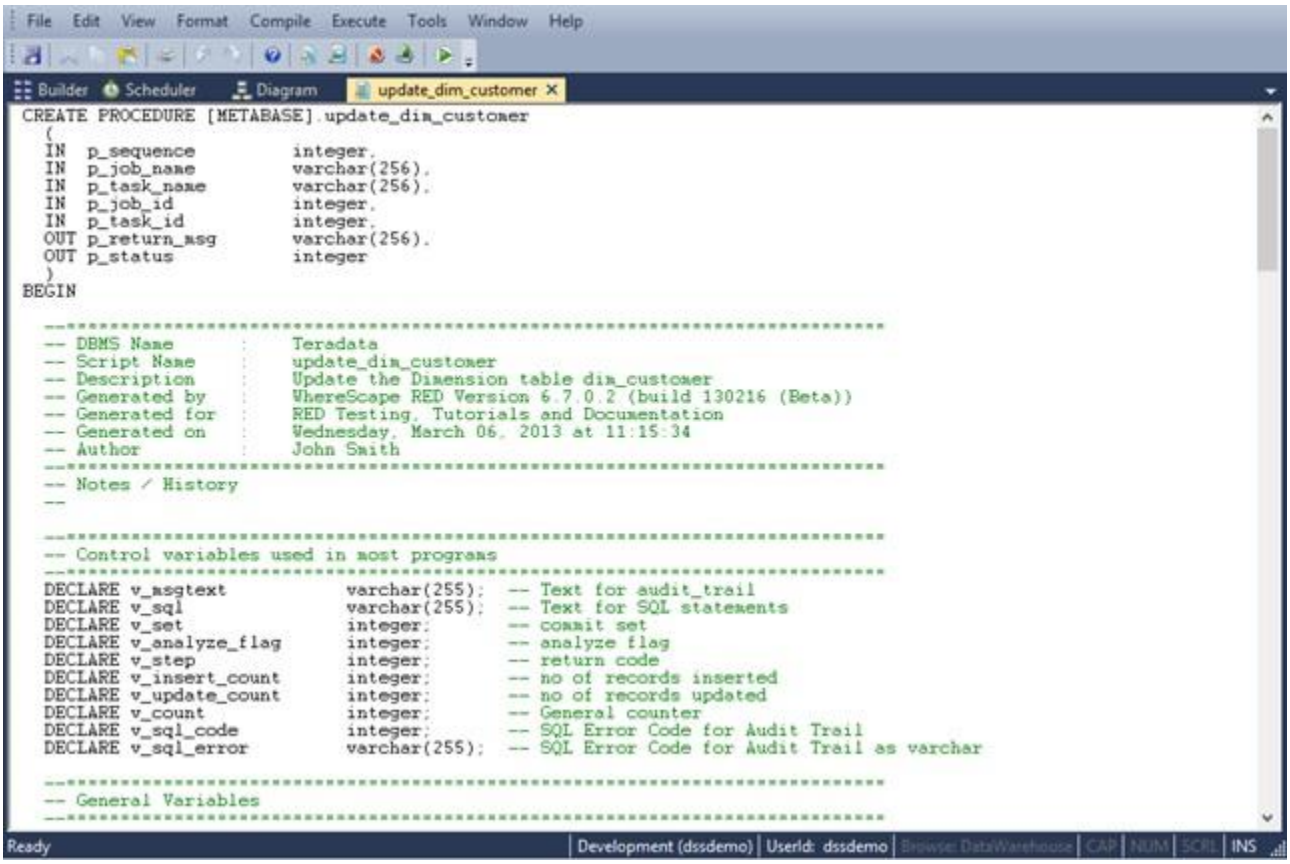


The Diagram Selection is as follows:

- Schema Diagram (for fact tables, aggregate tables and OLAP cubes)
- Source Diagram (for any table)
- Joins Diagram (for any table)
- Links Diagram (for any table)
- Impact Diagram (for any table)
- Dependency Diagram (for any job)

RED Procedure Editor Window

The Procedure Editor Window provides a means of viewing, editing, compiling, comparing and running procedures.



Multiple such windows can be open at any one time, each processing a different procedure.

Comments (identified by a leading double dash --) are displayed in green in this window and the procedural code in black.

The font is a fixed pitch font (by default) to make the indentation and alignment of code easier to view.

NOTE: The font, colors and indent size can all be changed if desired from within the **Tools/User Preferences** menu.

Key Objective / Overview – Data Connections

Before any data can be brought into a data warehouse, you need to connect to the underlying data sources.
In this lesson you will learn the requirements to define a **Connection** to a source of data in order to extract the data using these connections.
You will also learn to:

- identify the initial data that your business users need.
- determine what is the best source to get the data from (it may not always be the obvious source).
- define your connection preferences in RED.
- explore the content of new Connections.

RED Definitions – Connections

Connections are the access mechanisms to the different information stores throughout the organization. They define the information necessary to get to each set of data whether as a source or even as your main target. RED is context sensitive in that once the connection is defined, what you can do with it is governed by how it is defined. i.e the options for a Database connection are quite different to that of a Text file connection. In this way you are guided to ensure that you handle these connections appropriately.

Create a Connection to a Database

The single-most important connection of all is the connection to the data warehouse itself. There is only ever one data warehouse connection defined in RED and it is identified by a check-box on the Properties page (and typically it would have an appropriate name).
In order to understand your Sales Orders, you need to understand how to get the information out of the Sales Order Processing (SOP) System. Each type of database requires its own set of parameters to define how RED can connect. Once set, RED tailors what you can do with that connection from then on.

NOTE: RED, by default, does not write to the source system and so a read-only login is all that is required. This logon can even be tailored to specific sets of data should the need arise.

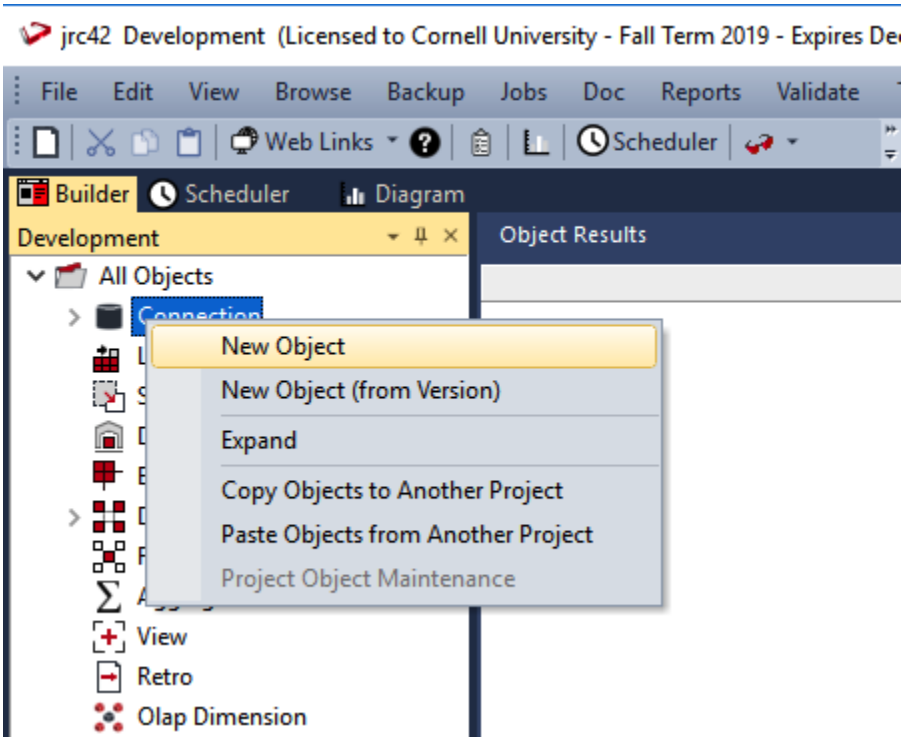
Start by creating **two connections.**

- One to define the target warehouse.
- A second connection for your first Source system (in this case Sales).

When RED is first installed, you are asked to specify where the RED metadata should be held. Since this is also the location for your data warehouse, the connection **DataWarehouse** already exists. You can create a new data warehouse connection or update this one at any time, but there can only ever be one active data warehouse connection at the same time.

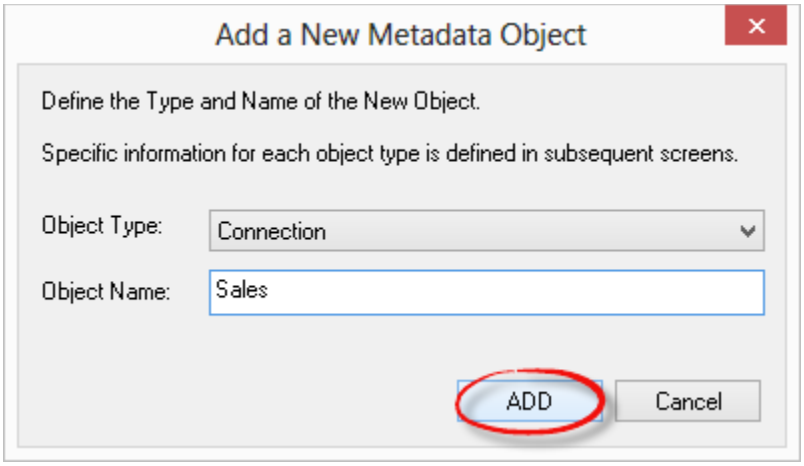
Now, bring in your first source and create a connection to the Sales Order Processing system using the database connection:

- In the object explorer on the left pane, expand **All Objects.**
- Right-click on **Connection** to bring up the option to create a New Object (in this context, a new Connection):



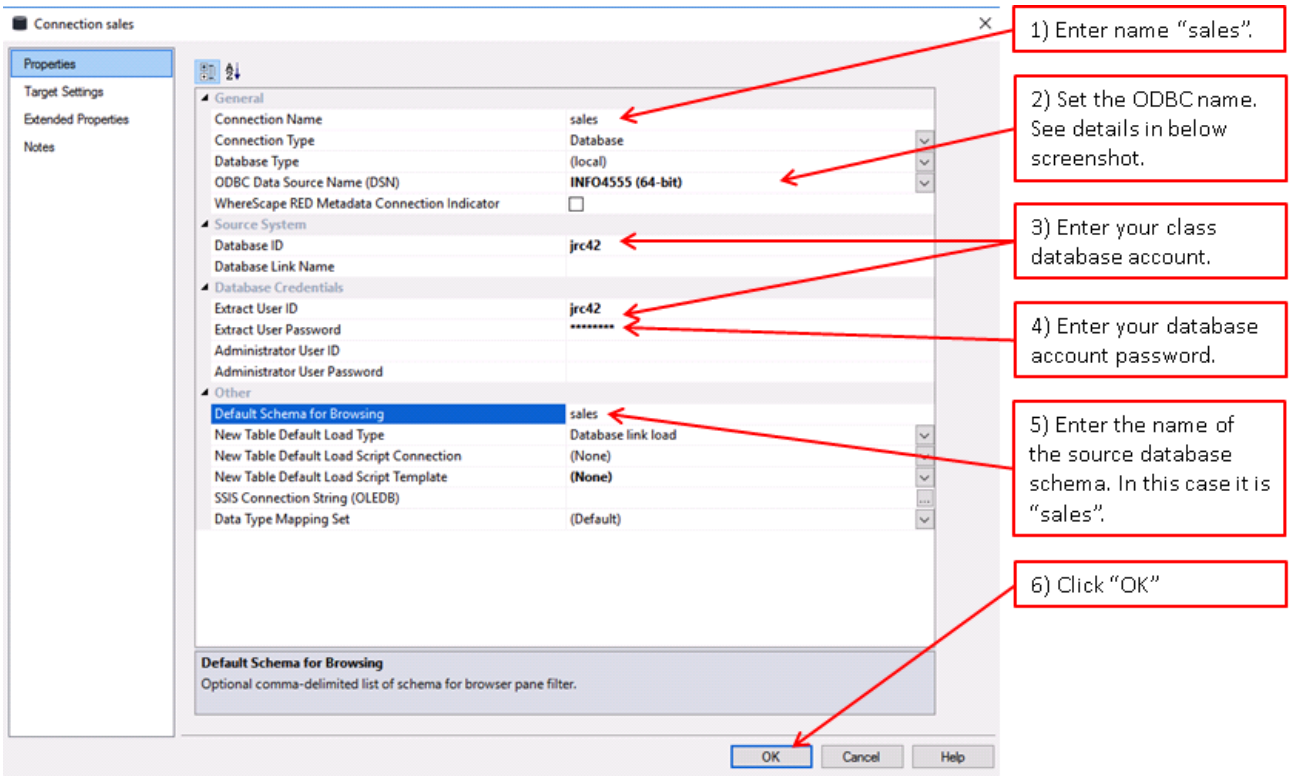
Note: If you need to change the type you can, but because you started on **Connection**, the New Object dialog defaults to an object of type Connection.

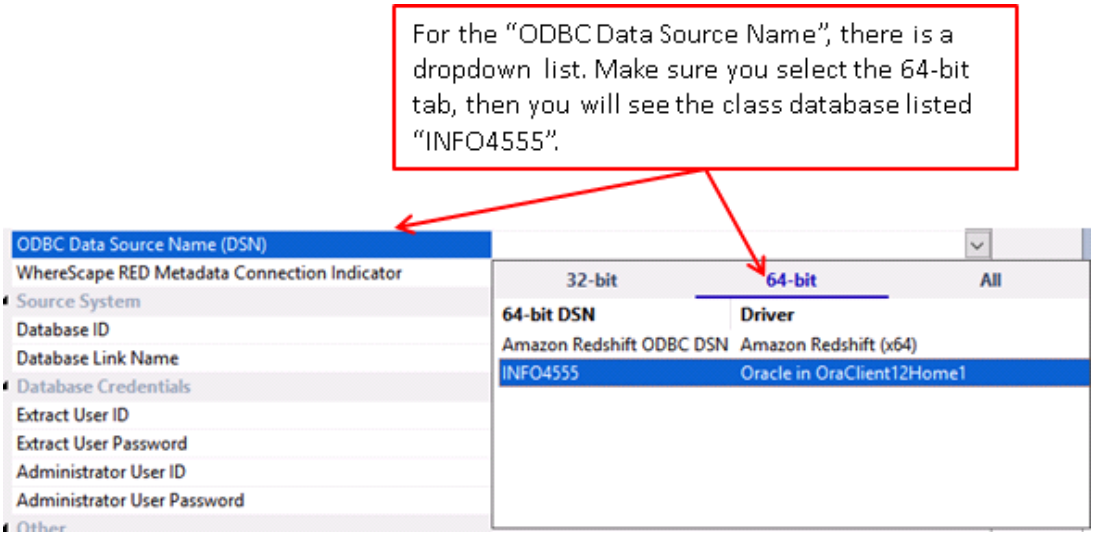
Type **Sales** as the connection name and click **ADD**.



This dialog box has entries for every option you might set for the chosen **Connection Type** – in this case you will want to see a **Database** type connection. Notice as you switch the Connection Type, your fields change to reflect the information relevant to that type.

- Here, select **Database** connection type for this Sales connection, other connection types will be explored later.
- For ODBC Data Source Name (DSN), enter the class database **INFO4555**.
- For the **Extract User ID**, enter the user ID and password with access to SELECT from the source system tables to extract data - in this case **your netid database account**.
- For "Default Schema for Browsing" enter **SALES**. This is the schema we want to view.
- Under "new Table Default load Type", select "Database Link Load" from dropdown.
- Click **OK** to complete the Connection.



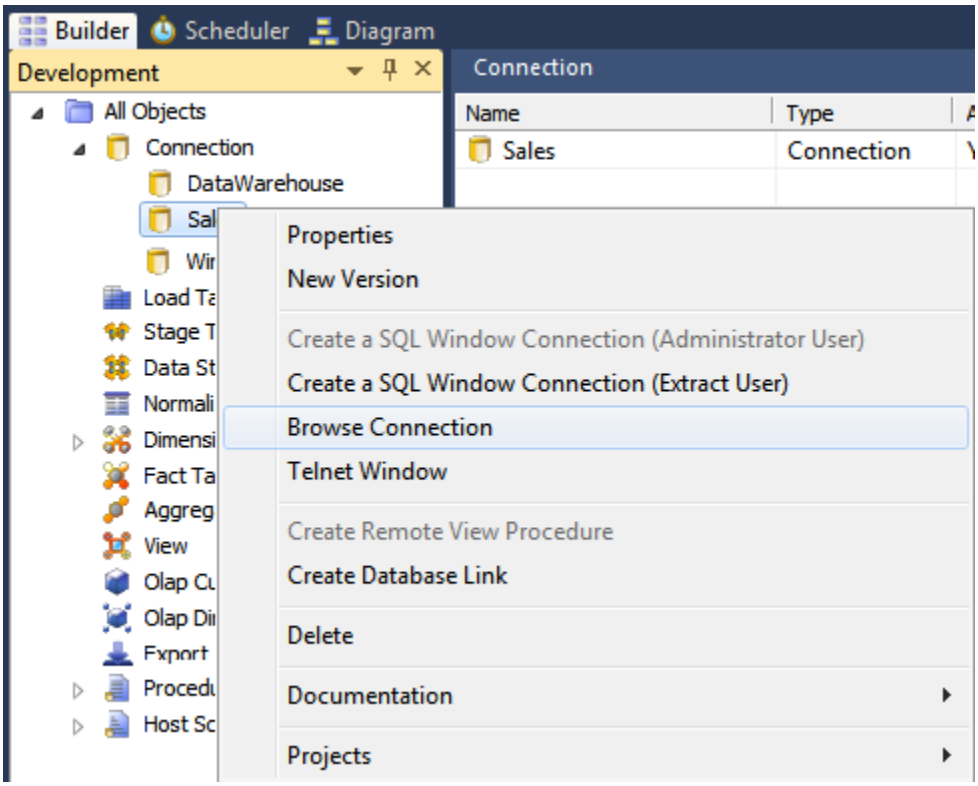


In this exercise, this is all you need to do right now, however if you were in a more complex environment, your DBAs may specify some of the other fields to be populated.
For example, if your DBAs have specified a specific username and password for you to use (as opposed to using the network logon accounts) you can enter these in the Extract User details.
Also, the **Default Load** Type allows you to pre-set the way your **Load Tables** are loaded using this connection. This will be explored in more detail in the next section.
In the Notes section, you can add any relevant information that will pass through to the documentation. This is particularly useful if there is anything unusual or specific that must be remembered when using this connection – keeping this in the notes means that any other developer(s) will also understand and act accordingly.

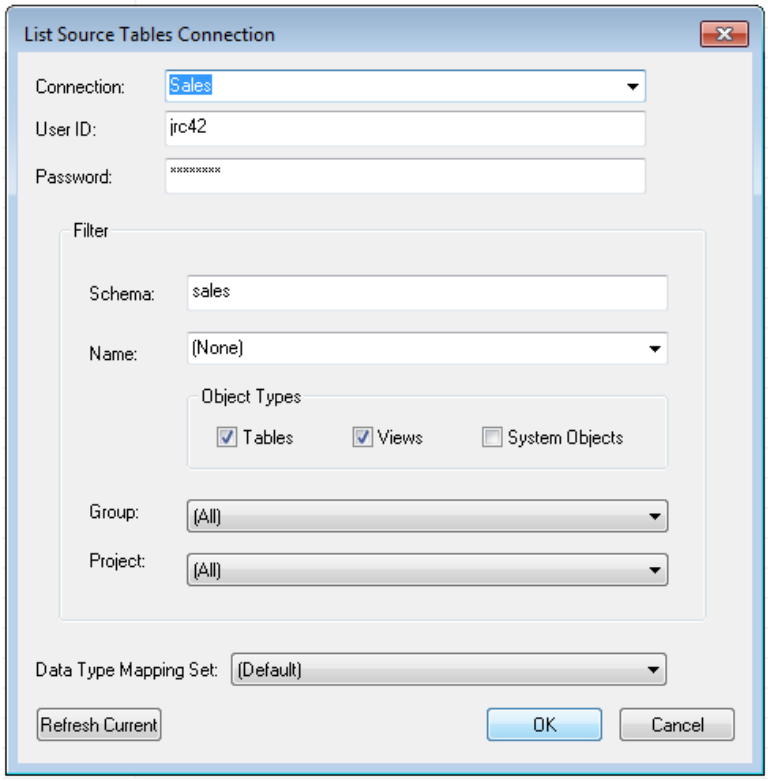
Browse a Connection

With a usable connection to the Sales system, you can see the data you have access to:

- From the left-hand object explorer pane, expand the Connection folder, right-click on the Sales connection and click on **Browse Connection**



Your connection details should be listed by default, otherwise enter your **User ID** and **Password** and enter **Sales** for the **Schema**. Click **OK**.



The right hand pane (Source browser tab) will update to reflect the content of the selected connection.



Double-click on the **order_header** table to see what your connection can determine about the columns and their associated data types.
Now you know how the table is structured, but you don't always have the luxury of having access to the database specific front-end tool to explore the data further to get a better feel for the information at hand.

NOTE: Depending on the connection you are using, the underlying source does not always share information about the individual columns. In this scenario, RED can still access the table and read the records, but you may need to tell RED the column details manually.

Right-click on the **order_header** table in the Source Browser and select **Display** Data.
The middle work pane changes to show a sample of data from the chosen table.

Data from Source Table order_header

order_id	order_number	order_type...	order_stat...	transaction...	base_curre...	sales_rep_id	payment_m...	sales_tax_f...	ship_to_ad...	bill_to_addr...	sold_to_ad...	order_date
25	GYZWP0B...	DEBIT	L	75	75	106	3	N	3160	4567	6582	2010-11-19...
26	TPRL0PQZ...	DEBIT	K	86	86	62	2	Y	3527	150	9492	2009-07-17...
28	OGPMJ0U...	CREDIT	K	63	63	756	3	Y	7396	5426	3051	2010-02-27...
124	SGOTTBJTI...	CREDIT	R	57	57	718	4	N	7382	6713	261	2009-10-14...
125	SUTPXKA0...	CREDIT	G	46	46	146	1	N	7155	447	2088	2011-05-31...
126	WYZKYZBE...	DEBIT	B	47	47	214	1	Y	7538	3403	5972	2011-10-18...
127	IXTQSZCO...	DEBIT	B	64	64	342	3	N	4066	1769	4935	2009-07-19...
159	DYWXVHV...	CREDIT	G	43	43	755	3	N	2715	2900	4615	2011-02-03...
160	EBOSCDVR...	DEBIT	G	72	72	460	4	Y	2304	4120	8273	2009-09-22...
161	TJ9HIIKWC...	DEBIT	E	39	39	502	3	N	10242	8543	6457	2007-04-15...
128	DWBDOH...	CREDIT	K	52	52	315	3	N	9356	7265	10109	2011-10-11...
129	Q0TWLIWE...	CREDIT	C	37	37	607	1	Y	536	9395	1595	2008-06-08...
130	SMN9BPTS...	CREDIT	B	77	77	307	2	Y	9589	3679	3308	2009-12-23...
131	UXZEKGH...	CREDIT	K	2	2	295	3	N	5156	6571	3511	2009-04-10...
162	NDZCZNO...	CREDIT	L	51	51	650	2	N	2467	7769	5880	2011-08-28...
163	BGBAU0FN...	CREDIT	B	43	8	594	1	Y	8541	7444	3731	2011-02-27...
164	AWFQI9KX...	DEBIT	C	63	63	318	4	Y	10190	478	10289	2008-02-01...
27	IULLUKYQ...	DEBIT	E	28	28	57	2	Y	10269	3175	458	2007-02-08...
29	RRIFEFZW...	CREDIT	R	30	30	341	2	N	8576	4282	8832	2007-11-30...
30	TYNKNSA...	DEBIT	R	30	30	73	2	Y	1836	6372	4156	2011-10-29...
31	DOYSZPP...	DEBIT	K	44	44	571	3	Y	2494	4490	3743	2010-11-06...

If the field / column names are more ambiguous, this can help us identify the data you are looking for within the underlying database.

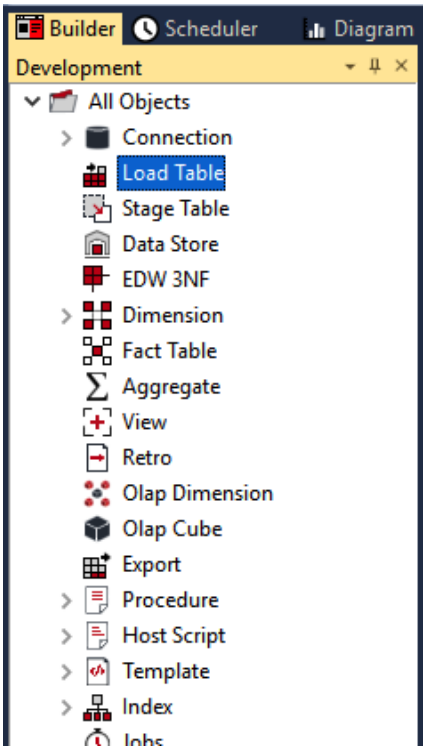
Load a Database Table

There are many types of Load Tables and most will be covered throughout the course. In the first example this is kept simple but will be built upon as the course progresses.
With an established Connection to the Sales Orders database, you need to determine what data needs to be extracted out of that connection and into the data warehouse connection.

On Lesson 1, the table **order_header** seemed to hold the level of detail needed right now so you want to bring this table into the data warehouse to easily report on it. The fastest, simplest way to do this is to copy the table over entirely. Later in the course you will see how to be even more efficient with the extracts from the source systems.

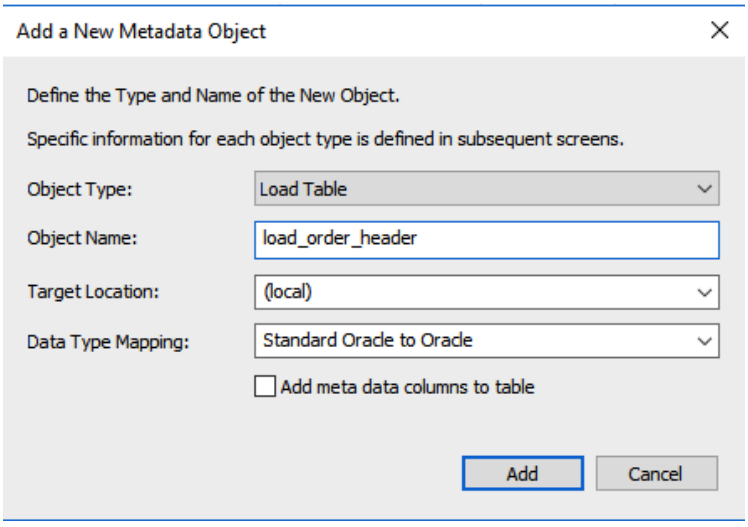
To create an extract against the **order_header** source table and bring the data into a Load Table in the data warehouse, set your working context to **Load Tables.**

- Expand the **All Objects** tree on the left pane and double-click on **Load Table.**
- If you had any Load Tables already defined, they would display in the middle work pane now headed **Load Table.**

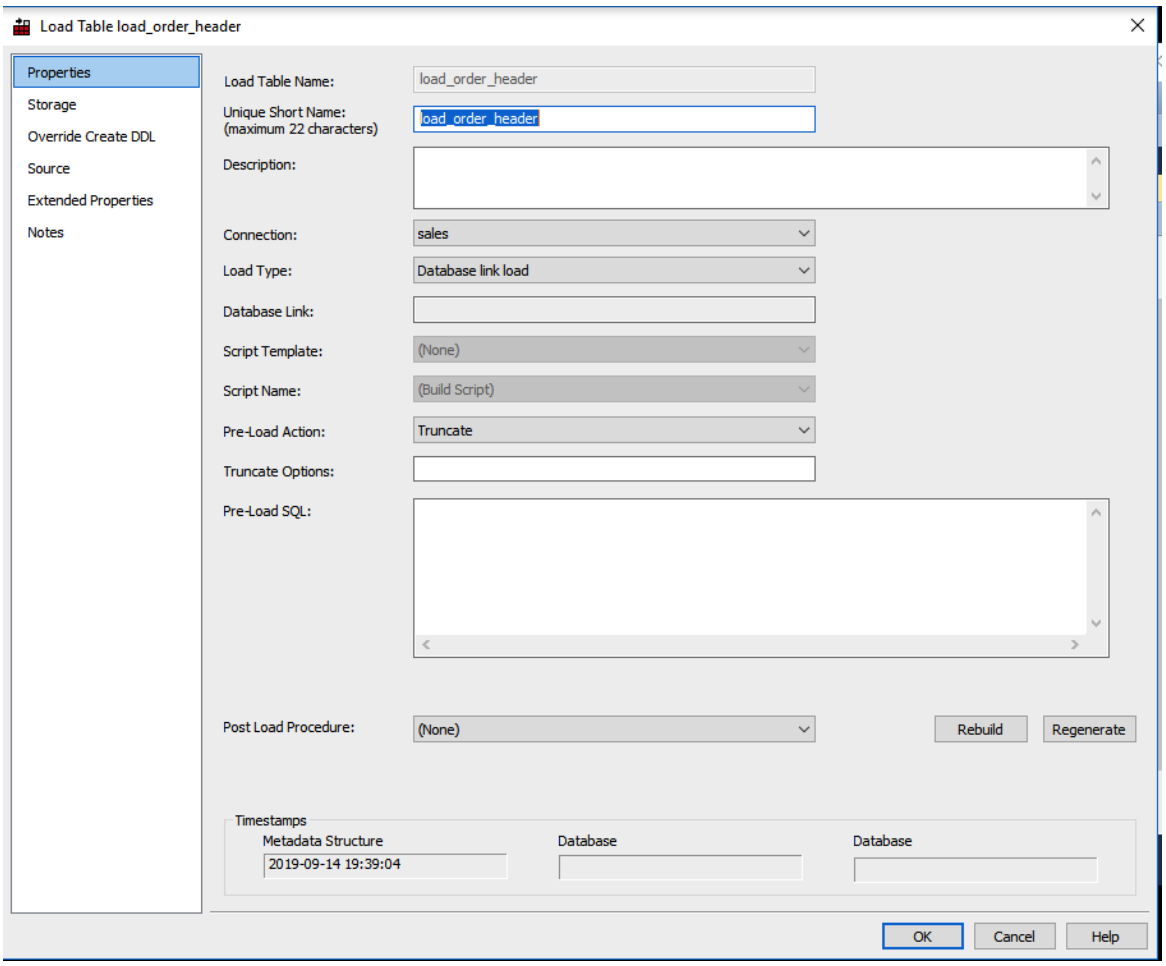


To create a Load Table definition of your **order_header** source table, drag the **order_header** table from the right-hand explorer pane into the middle work pane.

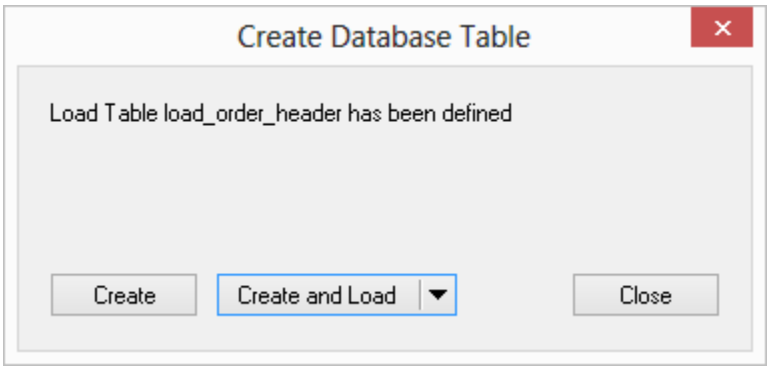
- As soon as you drop the table, RED recognizes a load table is being created and prompts you for the name of the new table. The suggested name **load_order_header** comes from your preferences and in this example for load tables, the original source name is prefixed with 'load_'.
- Click **ADD**.



The Properties dialog for this new load table is displayed, allowing you to change the definition – RED has already set everything you need so there is no need to change anything but you will later in the course. Click **OK**.

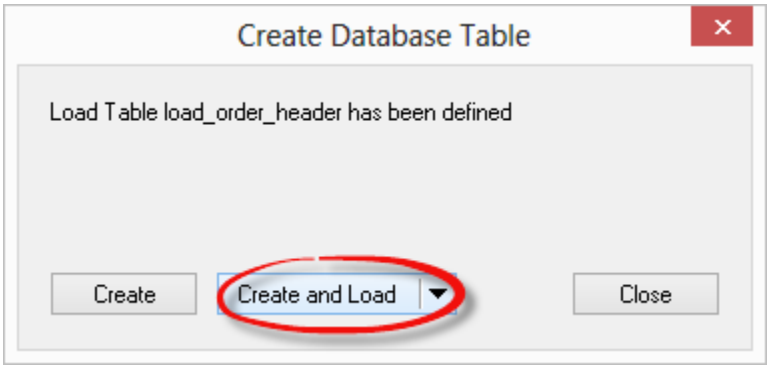


The final dialog in this task is simply for RED to understand how far you wish to take this load table at this point.



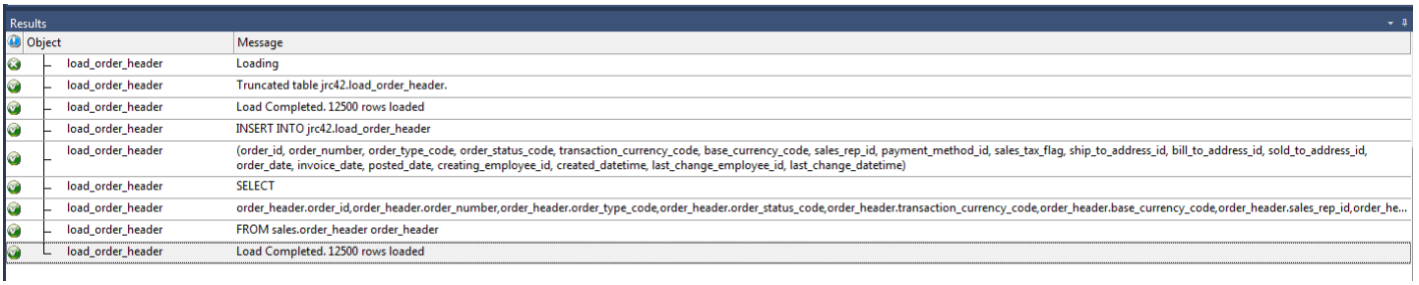
- **Create** will finish defining the table in the RED metadata but also create the (empty) table in the data warehouse. This again allows you to carry on working on the table and what happens to the data while you wait for an appropriate opportunity to load the data from the source system.
- **Create and Load** does the full define in RED. It creates the table in the data warehouse and load the data from the source system. Notice the drop-down to allow this to be processed via the Scheduler – this will be covered later.
- **Close** will simply define the load table in the RED metadata. You can modify it as required and once you are ready, create it in the data warehouse then load the data into it.

Click **Create and Load** to bring the table definition into the warehouse and populate it with the current data.



NOTE: For a large dataset, RED will pop up a confirmation screen showing the number of records that will be extracted, and ask if you wish to continue.

Notice the bottom **Results** pane shows what actually happened when you loaded the table from the connection.



The result pain should end with a “Load Completed” message, not an error.

Note that we used an ODBC load, which is not the fastest method, but easiest to deploy in our classroom lab environment.

RED Definitions – Connection Load Types

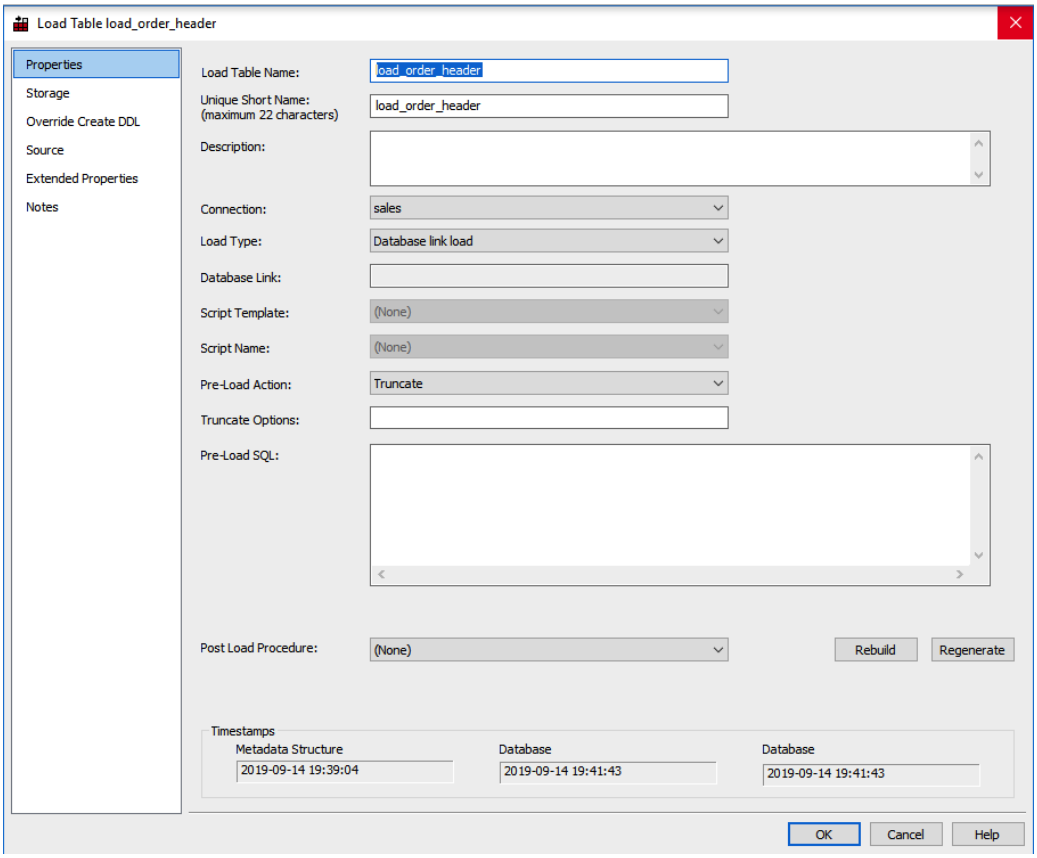
Database Link Load would have the data loaded from another database via a database link in the data warehouse.
Externally Loaded expects that the load table is populated by some external process, e.g. an ETL (Extract, Transform and Load) tool or EAI (Enterprise Application Integration) tool, putting the data directly into the load tables.
Integration Services Load uses the SQL Server Integration Services engine to run an SSIS package that RED auto-generates to extract the data from the source system and load it into the Load Table.
ODBC Load loads the data via an ODBC connection against the source and also writes each record using ODBC into the data warehouse connection.
Native ODBC is an alternative ODBC load where the source is again extracted via an ODBC read but then the data is written to a temporary file and the data warehouse bulk loader called to load it instead of by row inserts. This is typically much faster for larger data sets.
File Load uses a text file to source the required data. RED generates the required code to load the table each time the table is updated.
Script-based load again uses a text file load where a UNIX or Windows system script is executed to create the file on the source system. The script is executed each time the table is updated. The script can be modified if required.
XML File Load is a specific type of file load where the XML file(s) hold the data in XML format in a Windows directory.

Next, you will see what else RED did as part of Loading the **load_order_header** table.

- In the left-hand object explorer pane, expand the **Load Table** section and double-click on the **load_order_header** table to open up the Properties dialog.

Properties tab

The Properties dialog for each type of Load Table will be different based on the connection being used.
There will be a different number of sections down the left as well as more or less options in each section as you look at the different connections and load types used.



Properties Details – Database Load Table

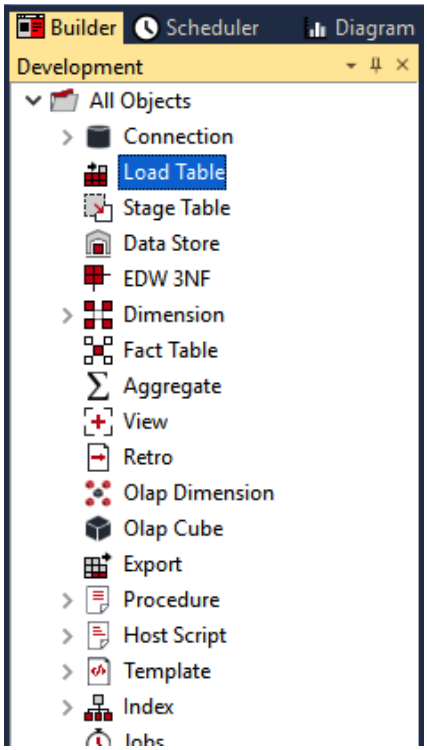
On the first section, Properties, you will notice the Connection you chose, plus the Load Type reflecting the default you set in the Connection itself. Changing either of these will re-configure the dialog box accordingly.
Load Type is set by default to the default Load Type for the 'Connection' being used. As it is currently set to Database link Load, the database link and script name are grayed out as they are out of context.
Pre-Load Action states what RED should do to the Load Table when someone re-loads it. The default is Truncate as in empty the table but leave the structure of the table intact for re-use. If it was necessary, you could change to a 'Execute Pre-Load SQL' Pre-Load Action which would allow you to, for example, delete certain sets of data from the load table before running. If you were to re-load the table, you would find that the first entry in the Results pane would be 'Table load_order_header' truncated given by the option you choose here.
NOTE: Whatever SQL you enter must be capable of running in a batch.

Post Load Procedure is the name of a Stored Procedure known to RED that you wish to run at the end of the load.

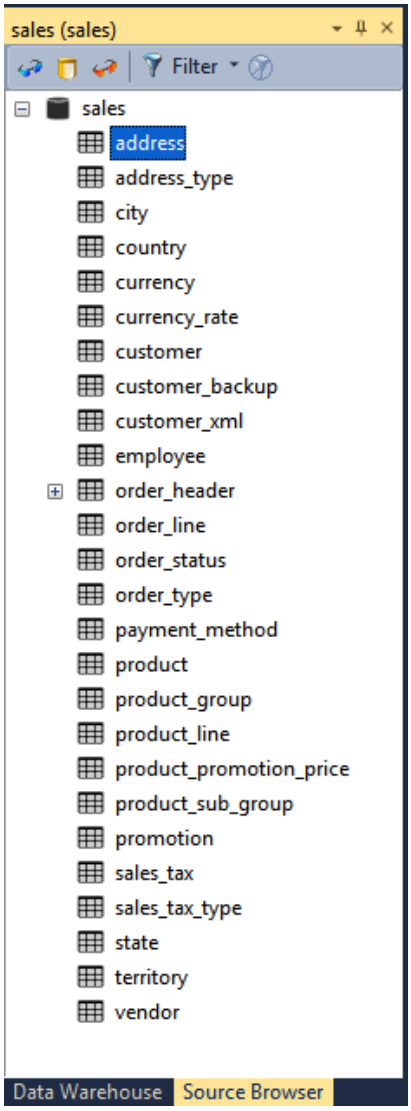
Along the bottom of the Properties section are the key timestamps. RED keeps track of changes and with Versioning (discussed later) you can roll back to a previous instance of any object. Versioning happens on most objects by default but these can be configured in the preferences.

The next step will be to **load** the **address** table which contains the customer code. The load table **load_order_header** does not have it and you will need it.

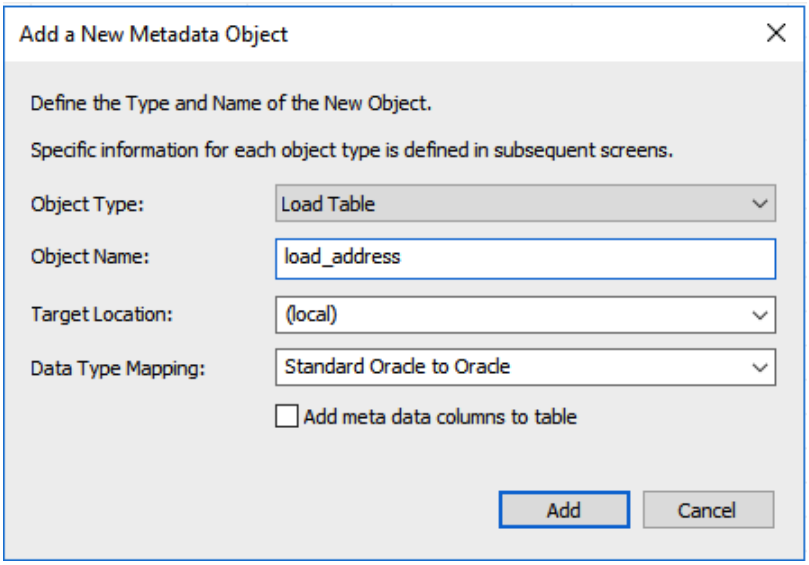
- Double click on **Load Table** in the **All Objects** tree on the left pane. This will set context to "Load Table".



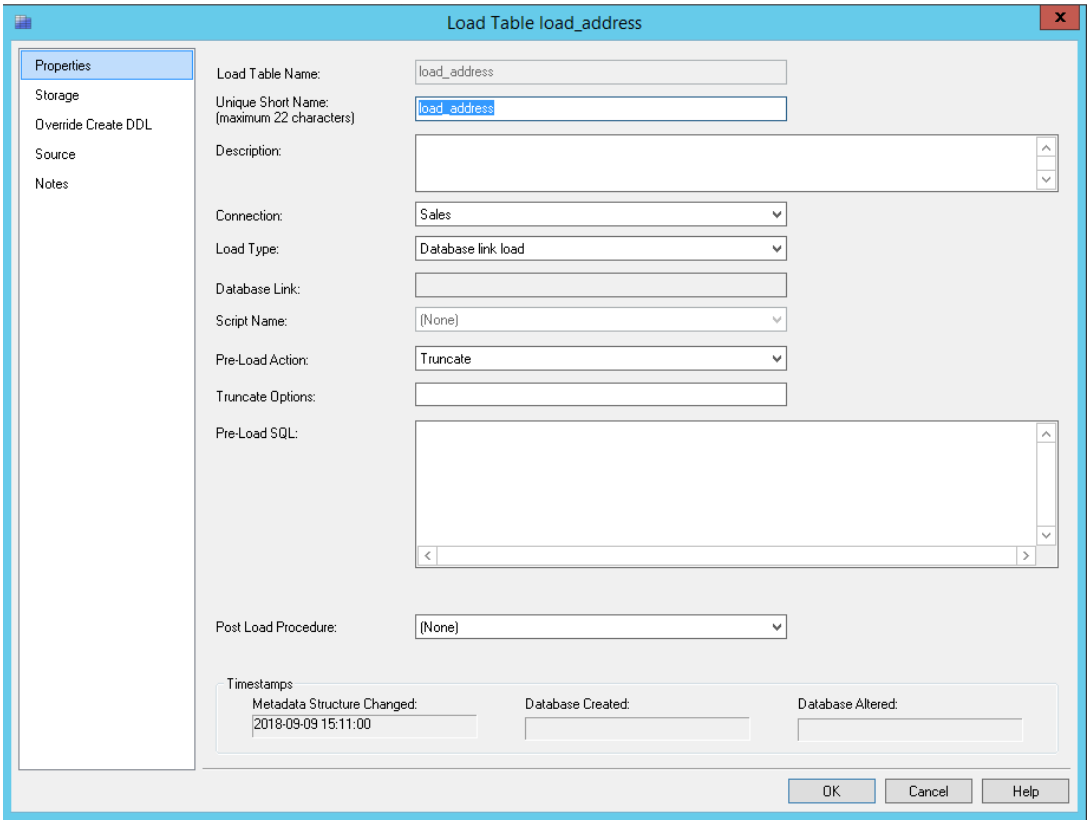
Drag the **address** table from the right-hand connection explorer pane into the middle work pane.



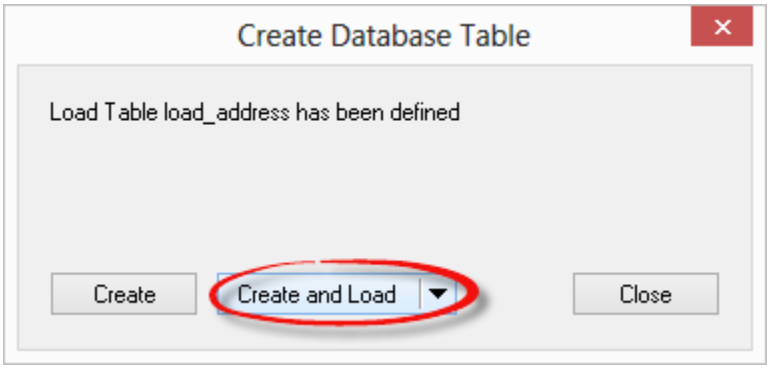
Accept the default name of **load_address** and click **ADD**.



Click **OK** on the Properties screen.



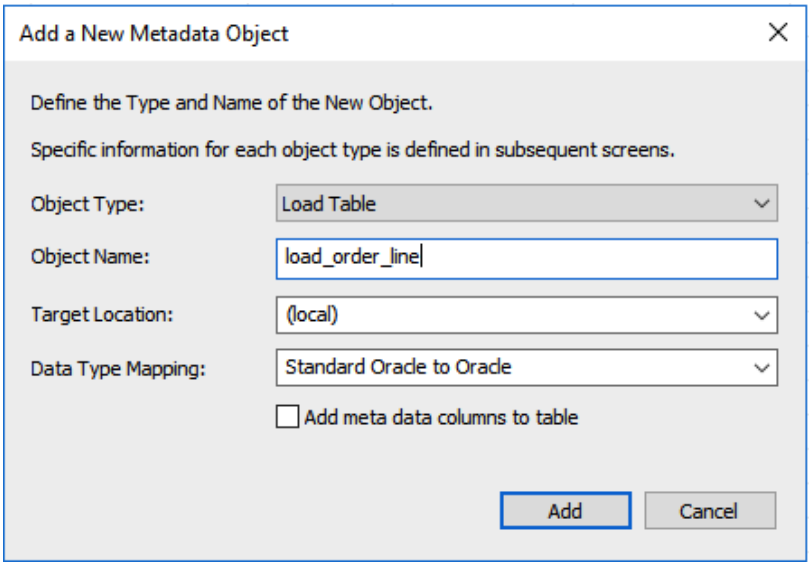
Click **Create and Load**.



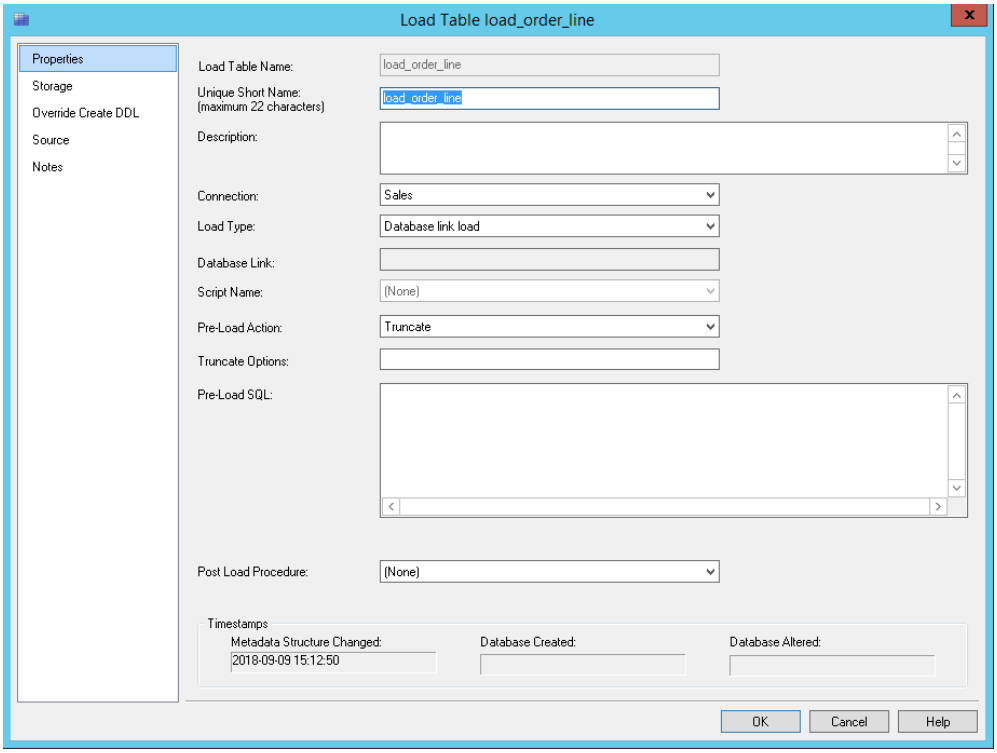
After confirming the table created and loaded successfully via the results pane, continue on to create another load table.

Set the context of your work pane to **Load table** by double-clicking on **Load Table** on the left pane explorer.

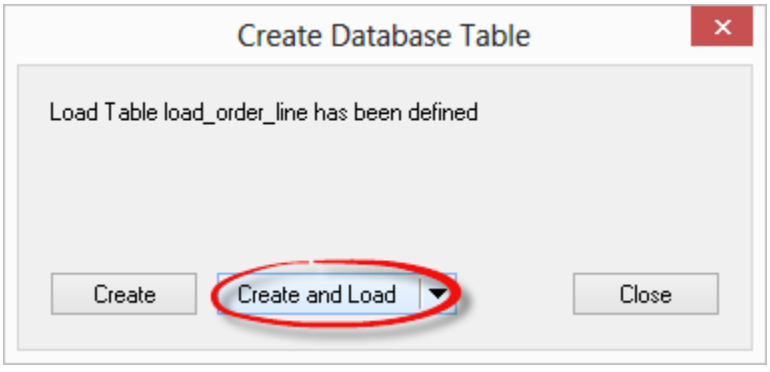
- Drag the **order_line** table from the right-hand source connection browser to the middle work pane to start creating a load table, **load_order_line**.



Clicking **ADD** pops-up the Properties dialog for the new load table, allowing you to modify how it is defined if necessary. Click **OK** to accept all the defaults on the Properties dialog.



Click on **Create and Load** to allow RED to save the definition, create the table in the database and extract the order line detail from the sales system into the data warehouse.



NOTE* This one may run a few minutes.

You now have three tables in your data warehouse regarding Sales; **order_header**, **order_line** and **address**. These tables were extracted from the source in the least resource hungry way to ensure that

the source system is unaffected by copying this data. However, these three sets of data need to be merged into one. It was something you were reluctant to do on the source, but now you know the data warehouse will perform this task efficiently.

Combine Multiple Sets of Data-Oracle

So far you have only explored load tables copied from the available sources. Now you need to be able to manipulate one or more of these extracts into a new set of data to meet the business needs.
You need to be able to determine which products are the best sellers each month and who are your top customers.
Even though the product data is in one table and the customer data is in the other, these tables share a field for the order number - **order_id**; and this in fact what is used to join the tables together.
To join tables together you need to create a new Stage table where you can apply business logic and merge in all the required information.

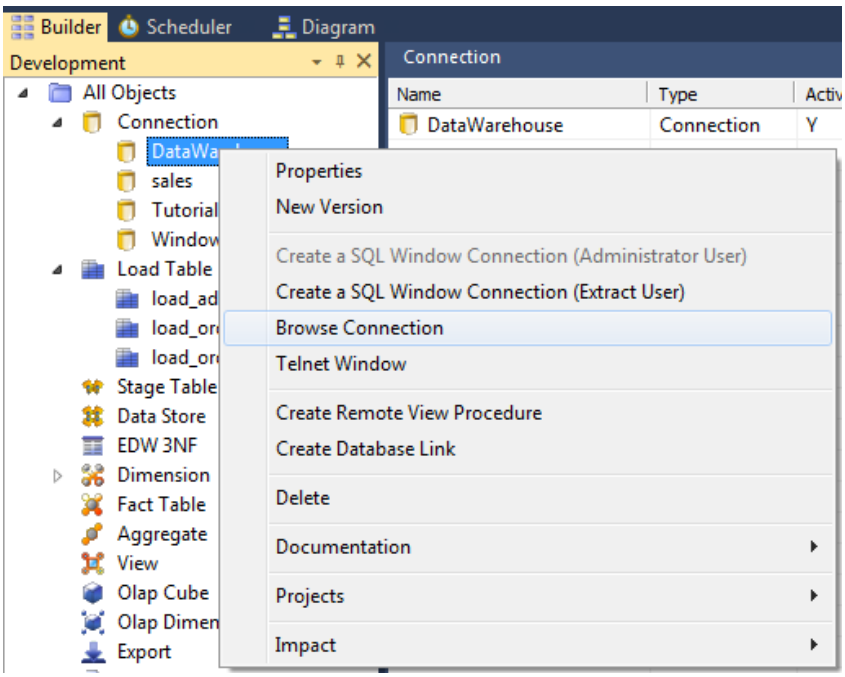
NOTE: As with most things in RED, there are a number of solutions to this 'problem'. Here you are going to create a new **Stage table** holding the content of both tables, but you could have easily created a load table that performed this merge on the source. Fundamentally the choice will be driven by the availability of resources and / or any requirement to replicate the source system(s).

RED Definitions – Stage Table

Stage Tables are the powerhouse of RED. These are used to present clean, structured, business defined sets of information to the areas of the data warehouse for the business to report against.

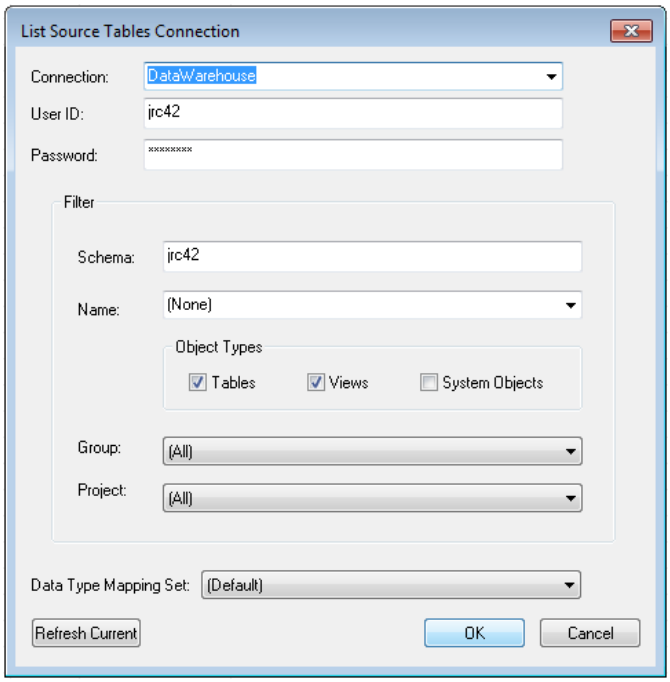
In the **All Objects** tree on the left pane, expand the **Connection** group and right-click on the **DataWarehouse** connection.
The **DataWarehouse** connection is your information store that you have been creating. Now you can see what it contains so far.

- Select **Browse Connection** to open the data warehouse in the source browser on the right-hand side of the screen.

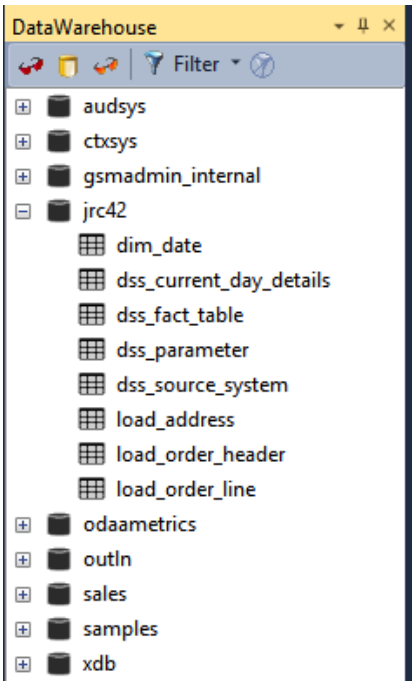


TIP: There are actually two tabs at the bottom of the right-hand pane; Data Warehouse which always shows the connection defined as the data warehouse connection, and Source Browser which lets you see any other connection defined in RED.

As before, the **List Source Tables Connection** dialog box appears, allowing you to adjust how much of the connection you want to explore. In this case you want to see everything so enter **your netid** as the Schema and click **OK**.



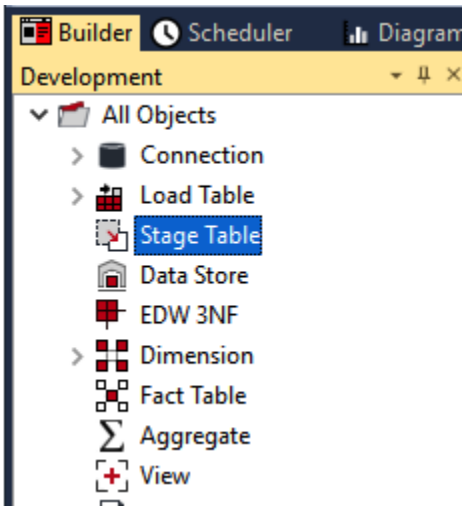
The **Data Warehouse** tab refreshes displaying all tables currently defined in the data warehouse, including your load tables that you created earlier plus one new table, dim_date, that you have not created.



NOTE: dim_date is a system table that RED creates automatically as every data warehouse will have a requirement to analyze data across dates at some point. You will look at this in more detail later.

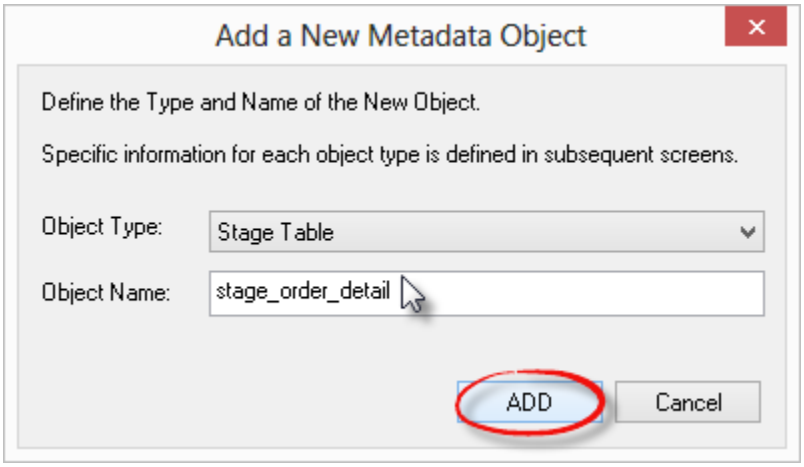
The data you need does exist in the load tables, but in separate load tables so you need to create the **Stage Table** to merge these sets of data together.

- In the object explorer in the left-pane, double-click on **Stage Table** to set the work pane context to **Stage Tables**.



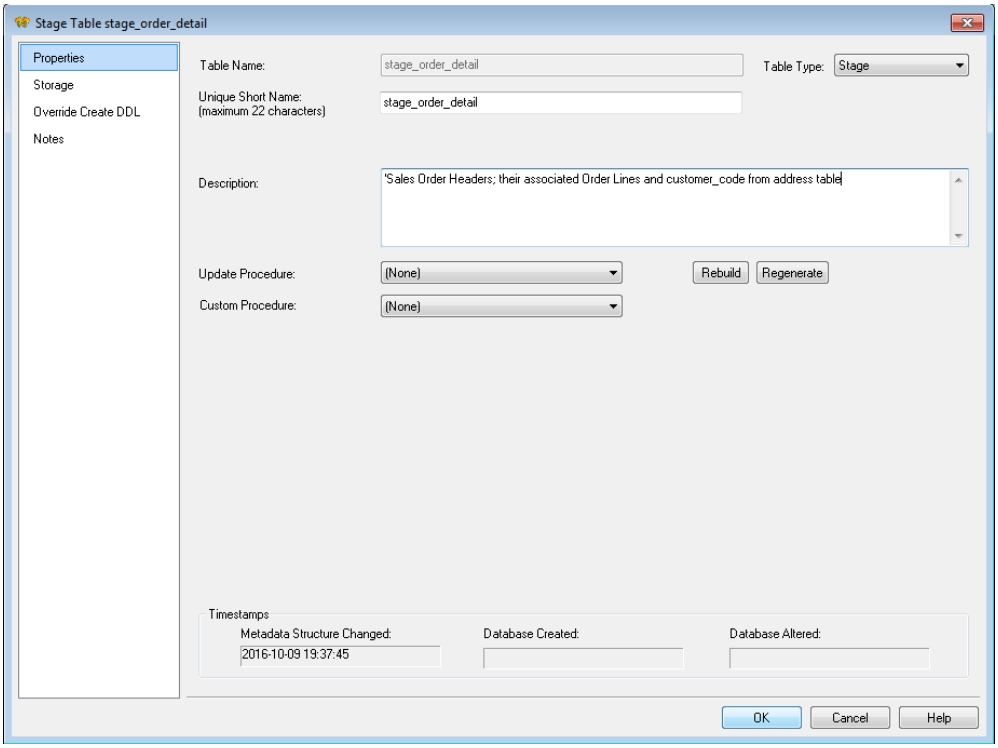
You want every field from the first two tables, so drag the **load_order_header** table from the data warehouse connection explorer into the middle work pane.
RED understands this action and creates a Stage table from your load table, setting the defaults according to your preferences.

- Type **stage_order_detail** as the new table name and click **ADD**.



In the **Description** field, type 'Sales Order Headers; their associated Order Lines and customer_code from address table' so that anyone looking at this table will understand why it exists.
This description will also flow through to the documentation.

- Notice that there are a lot less options to specify for a stage table. Click **OK**.



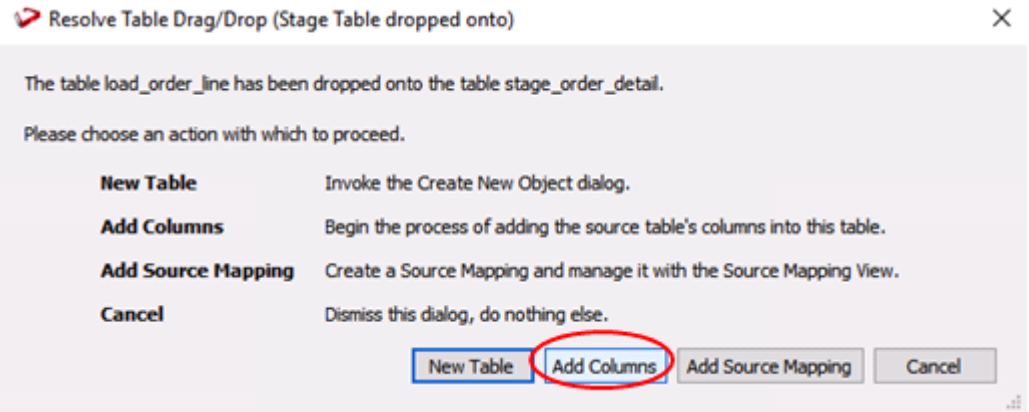
- **TIP:** For longer comments, use the Notes tab. Any Descriptions and / or Notes will always be taken into the documentation – this will save a lot of time later when it comes to documenting your solution.

- At this point, the stage table is simply a direct copy of your load table but behind the scenes, RED will expect to handle this table differently as it is a stage table and not a load table.

- **NOTE**: Load tables can only extract data from source systems, while stage tables can only feed from the data warehouse tables.

The 'missing' data will need to be added next to fulfill your requirement.

- With the new **stage_order_detail** as the context of your work pane, drag the **load_order_line** from the right pane onto the middle work pane.
- RED recognizes that this may be a mistake and you are not really wanting to add all the columns of this table into the stage table and prompts the option to create a new table from the fields of this load table instead of merging the two tables together. In this case you want to add in the columns of the second load table into the first so select **Add Columns** to **merge** load_order_line into the stage_order_detail table.

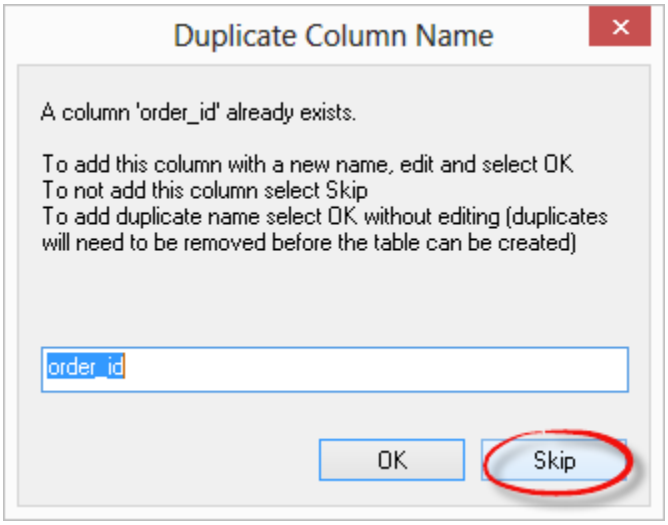


RED identifies that there are columns that are shared in both tables and prompts for you to determine how to handle them; for example: order_id is simply the key so you do not need to bring it in twice.

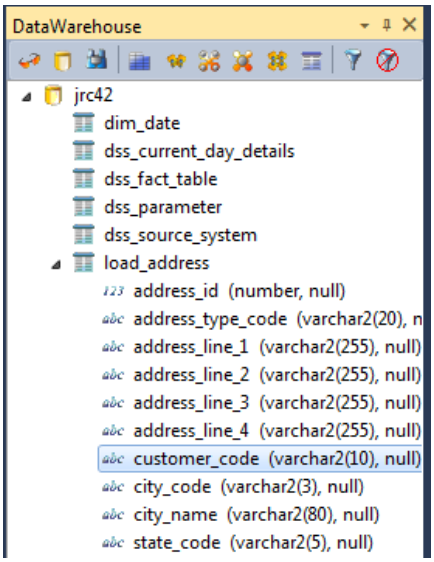
TIP: There are occasions where you may need to see both fields to be sure, so simply add it in with a more qualified name e.g. line_order_number (the header version is already in your stage table and could have been renamed prior to merging these tables, if required).

- Click **Skip** to ignore the second version of the **order_id** field.

- Click **Skip** to ignore each of the **four** identical columns that follow as well.



With the stage_order_detail still as the context of your work pane, double click on **load_address** in the right-hand connection browser and drag **customer_code** onto the middle work pane.

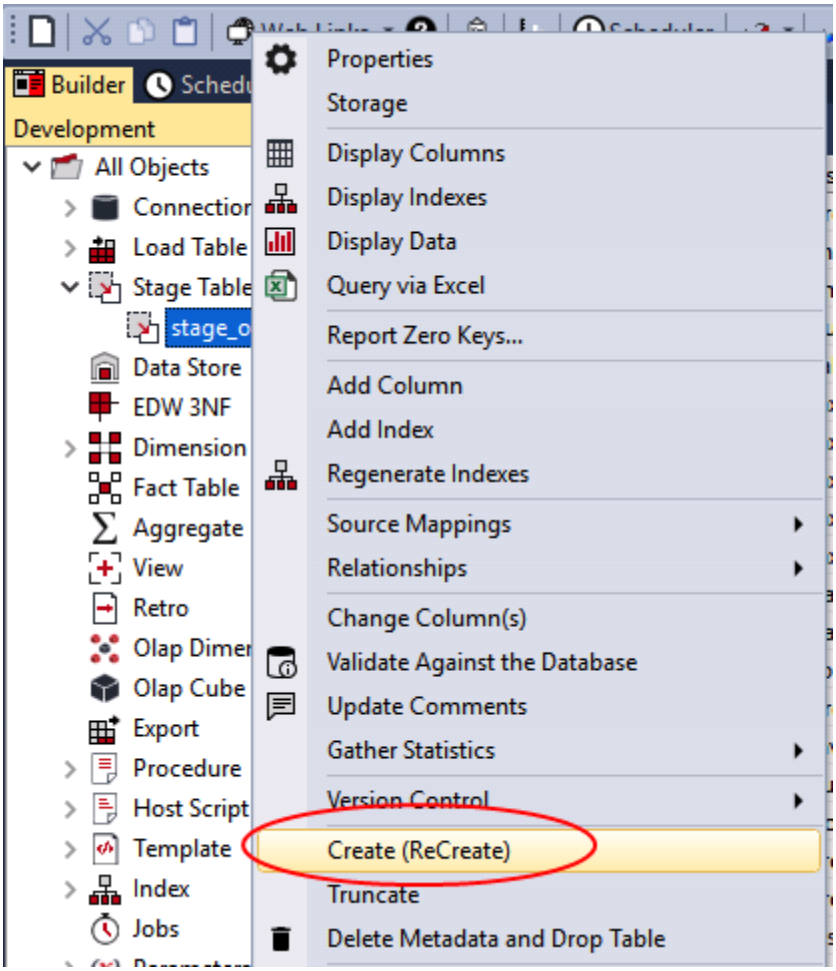


NOTE: Sometimes you will have the same field coming in from disparate sources and they are unlikely to be the same thing. For example, you have two sales systems after a merger and you have two customer_code fields – it is unlikely that customer_code '1' will be the same customer in both systems so you may wish to keep the fields separate or transform them in some way to make them unique across the new integrated system

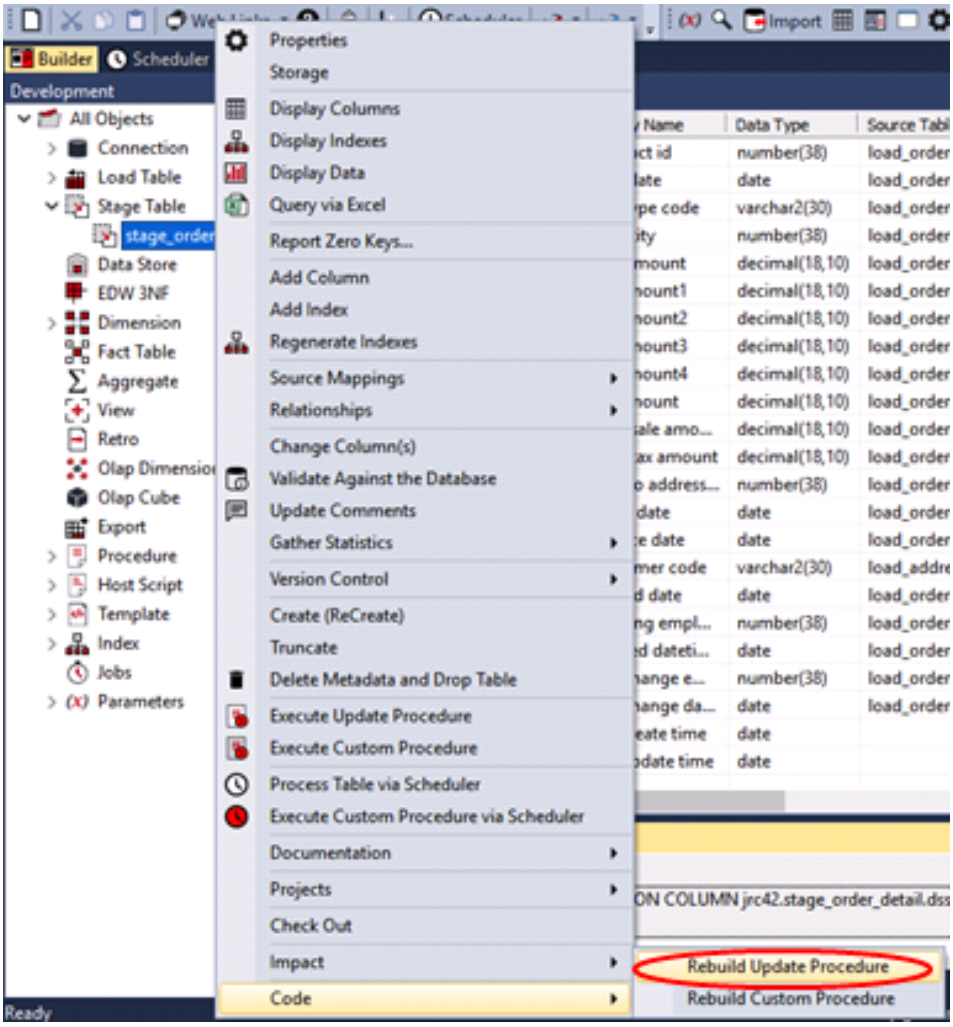
---

There is now a table defined in RED that will merge the three load tables together but RED needs to know how the tables relate to each other.
To do this you will need to build the stored procedure that RED will write to refresh the table.

- Right-click on the **stage_order_detail** stage table in the object explorer in the left pane and select **Create (ReCreate)** to define the table in the data warehouse.

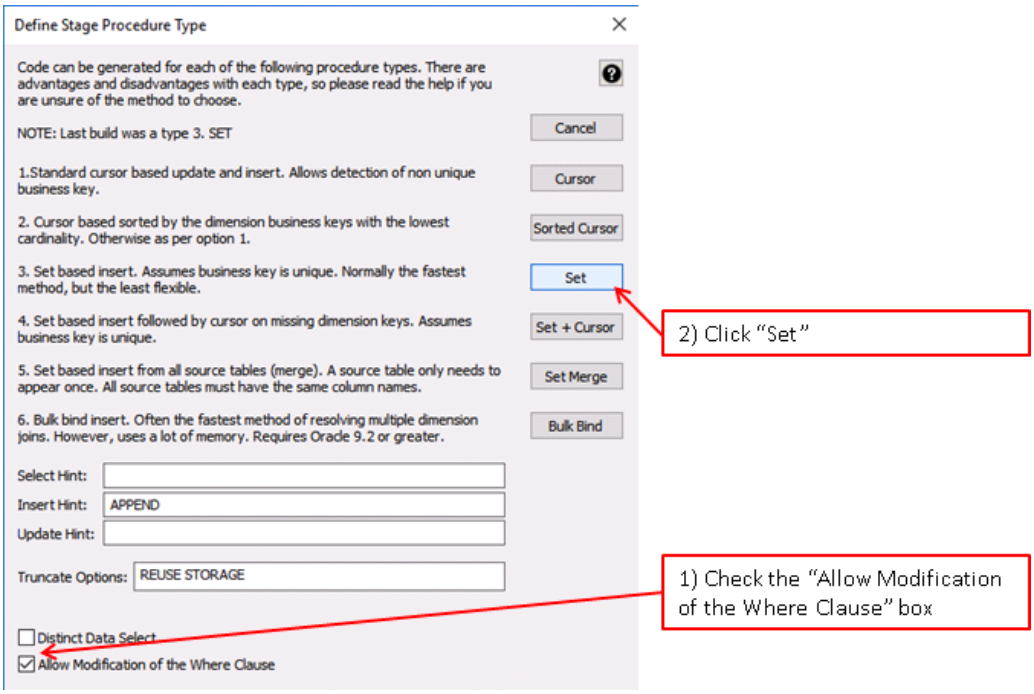


Right-click again on the **stage_order_detail** stage table, select **Code and** then **Build Update Procedure**.

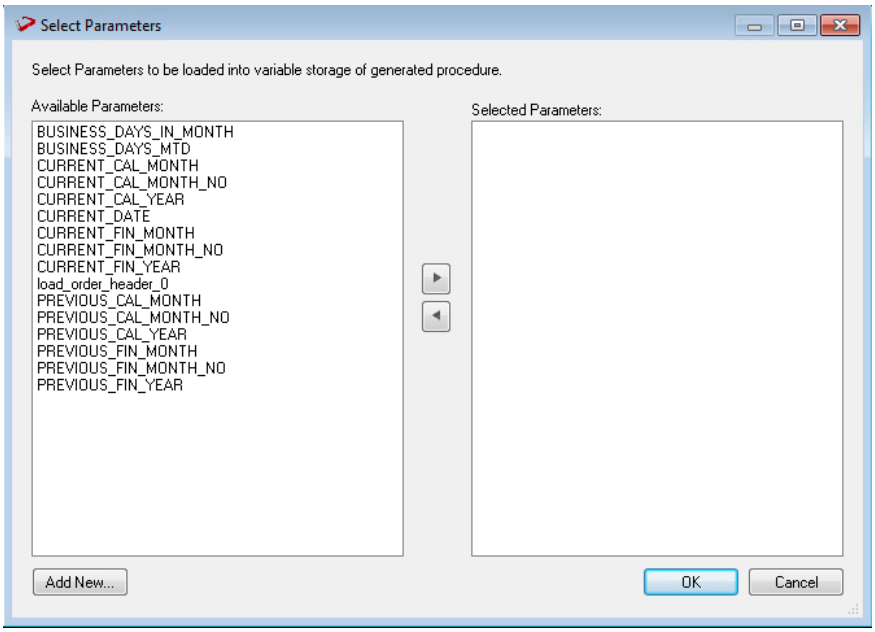


This brings up the **Define Stage Procedure Type** dialog to select the type of update you need.

- See the *Definitions table* below for the different types of Stage Procedure options.
- In this current example, you have nice clean data in your load tables and you would prefer a faster load that exploits the power of the data warehouse so click on **Set**.

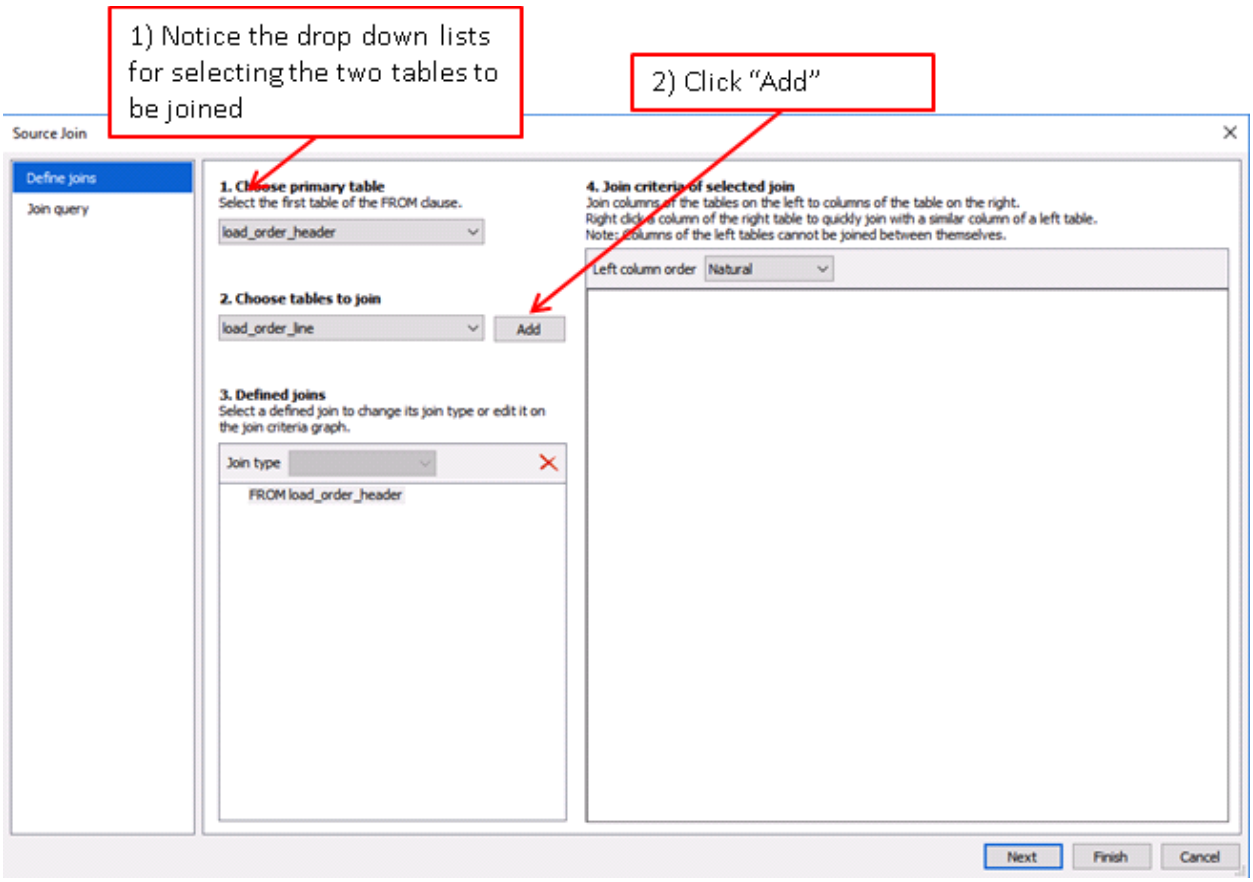


Parameters do not need to be used so click **OK** without selecting any parameter to use in this build. Parameters will be discussed later in the course.

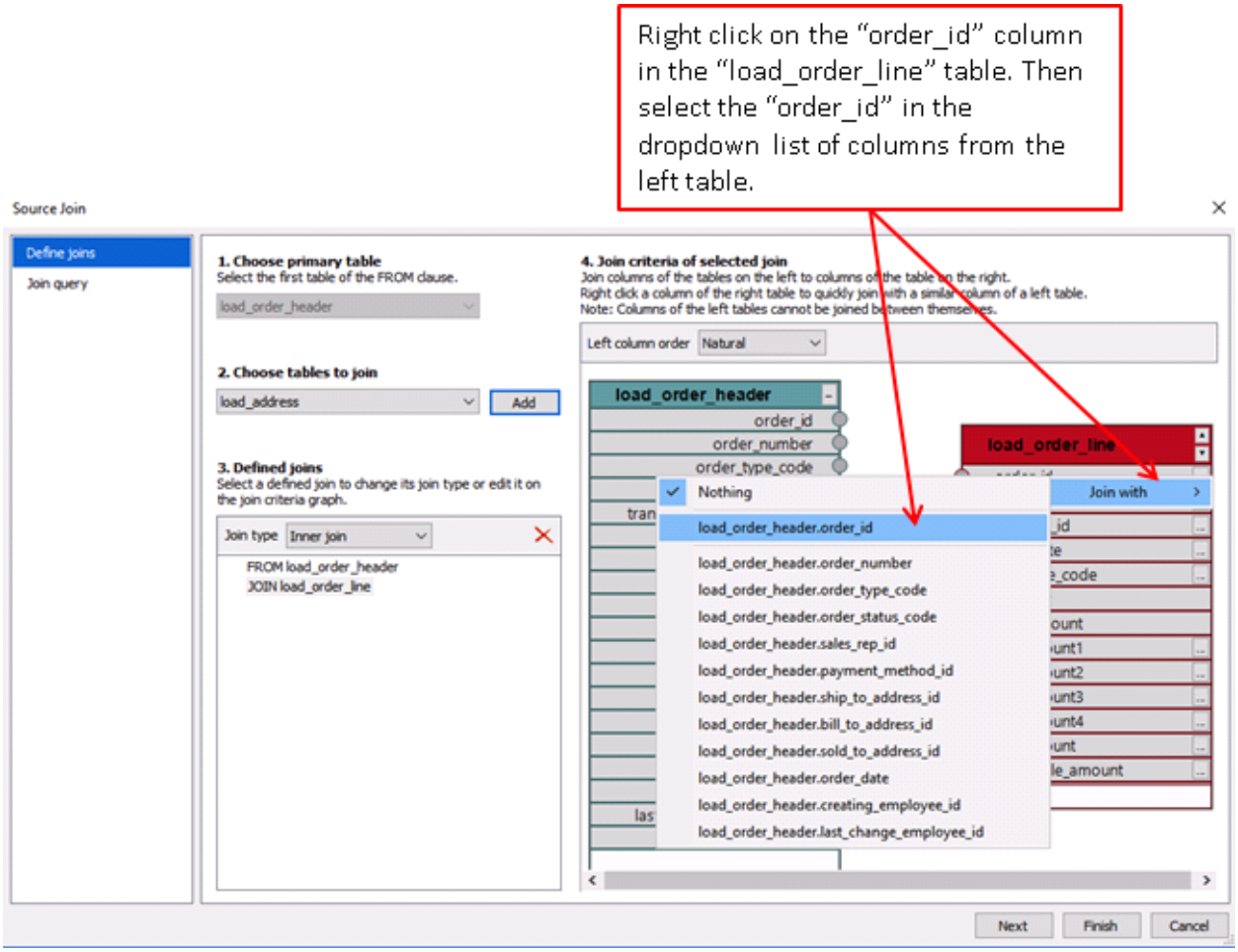


The **Source Join** panel is where you can define how the source tables relate to each other.

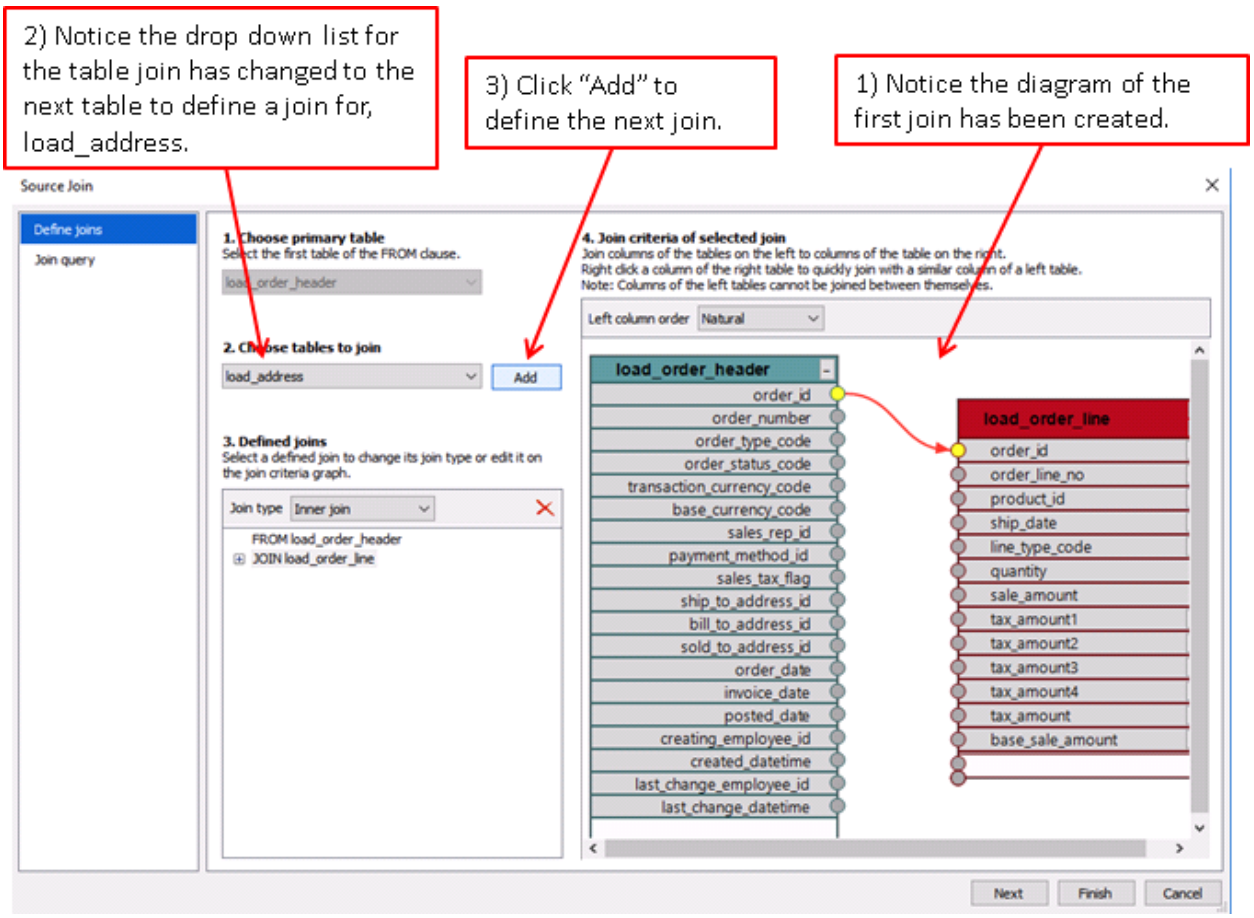
- Drop down list 1 is for picking the primary table. In this case WhereScape identified load_order_header as the primary table. The drop down list 2 is for the table to join the primary table to, in this case load_order_line is the default. Click Add to define the column(s) to join these two tables.



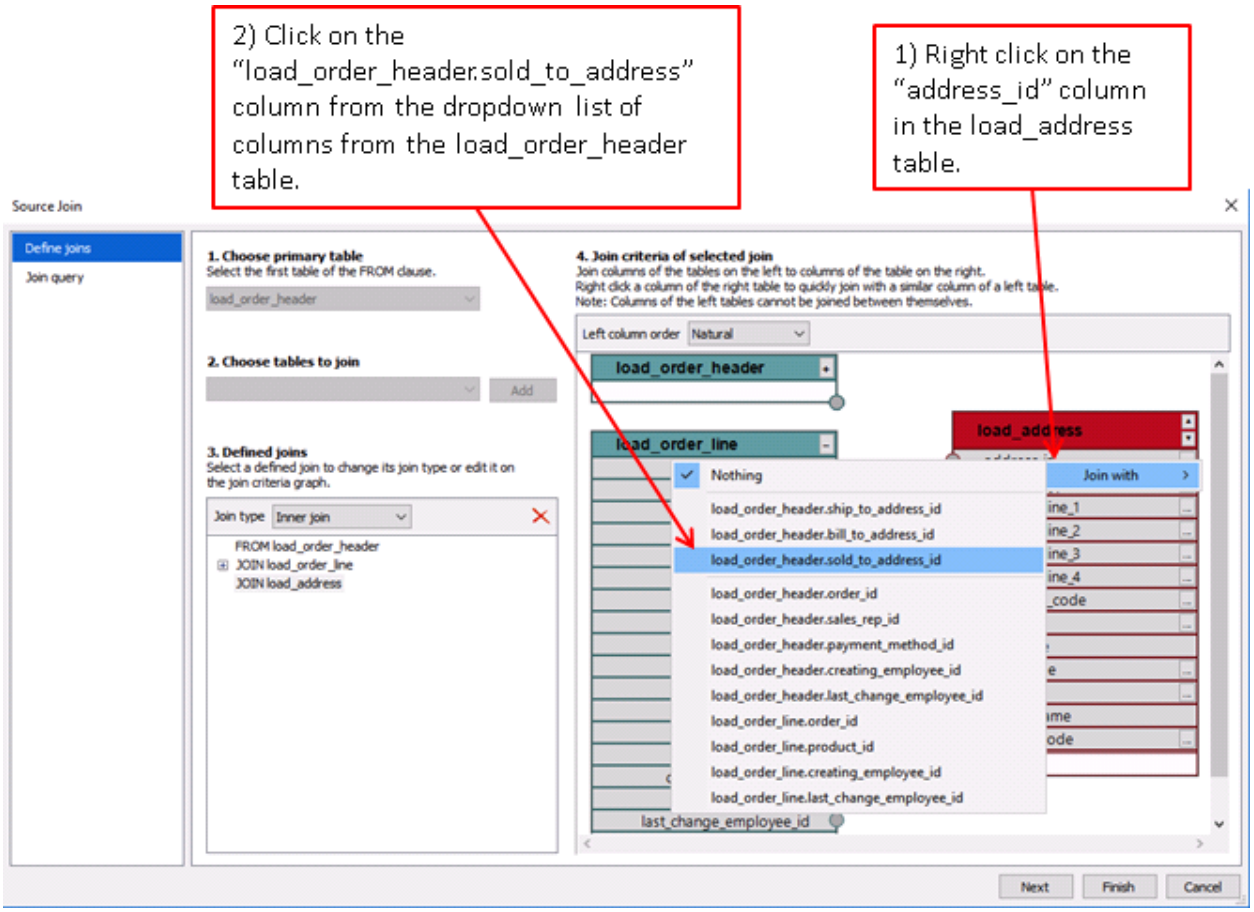
- Right click on the column order_id within the load_order_line table. From there click "join with". Finally, from the drop down list of columns from the load_order_header table, select "order_id".



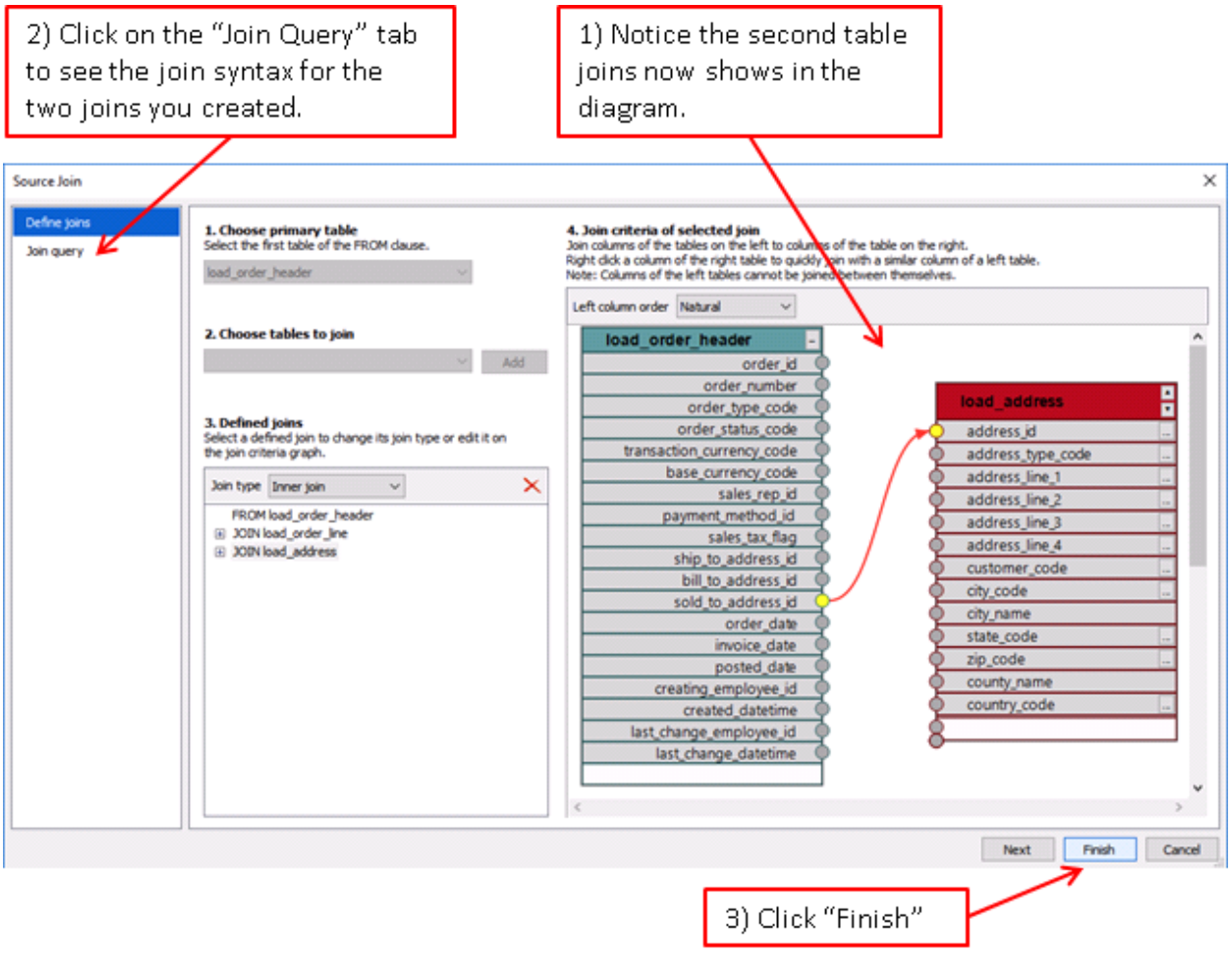
- You will now see a diagram of the first table join between load_order_header and load_order_line on the right. Notice that the next table to be joined "load_address" appears in dropdown 2. Click "Add" to define the column(s) for the join between load_order_header and Load_address.



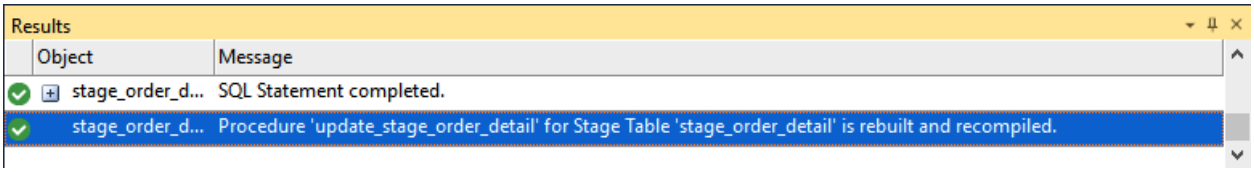
- Right click on the address_id column in the load_address table. Click “join with” and select the “sold_to_address_id” column in the drop down list of columns from the load_order_header table.



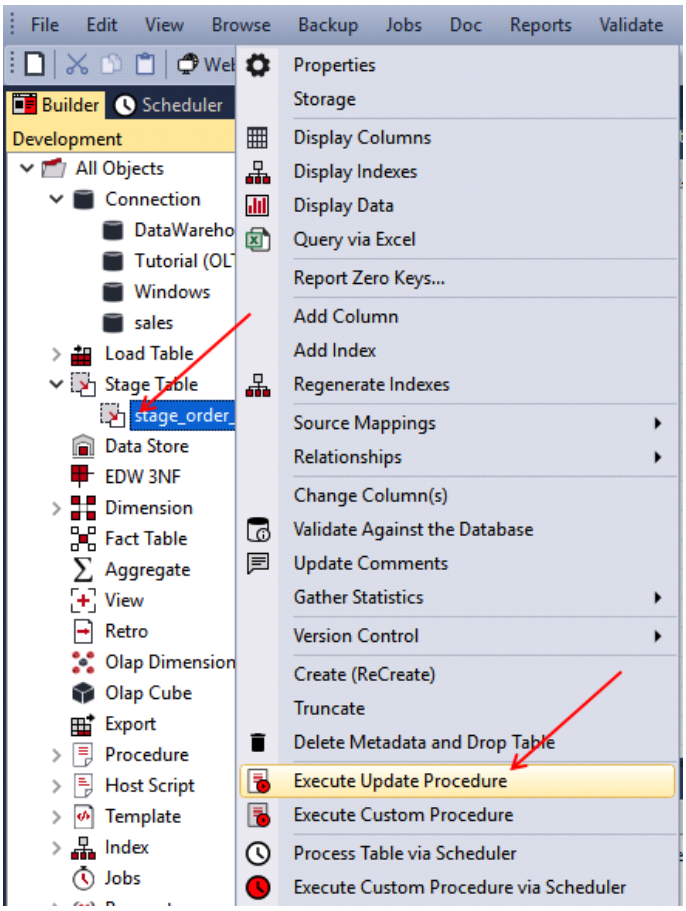
Now you will see the second join in the diagram. Try selecting the "join query" tab on the left. This will display the table join syntax for the joins you have defined. Be sure there are two joins defined. Finally, select "Finish" to build the procedure to populate the stage_order_detail table.



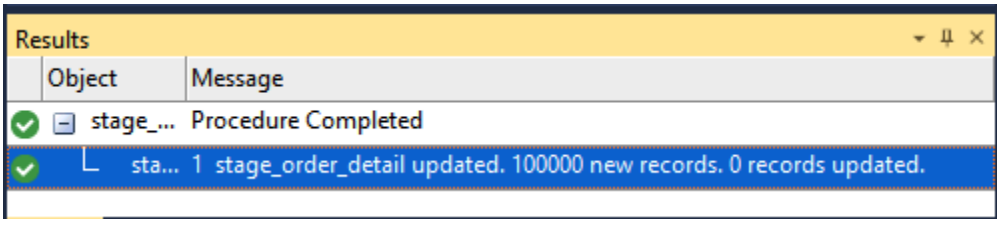
Notice in the **Results** pane: the stored procedure has been rebuilt and recompiled. It is ready for use.



Finally, populate the data into the table by right-clicking again on the **stage_order_detail** stage table and selecting **Execute Update Procedure**.

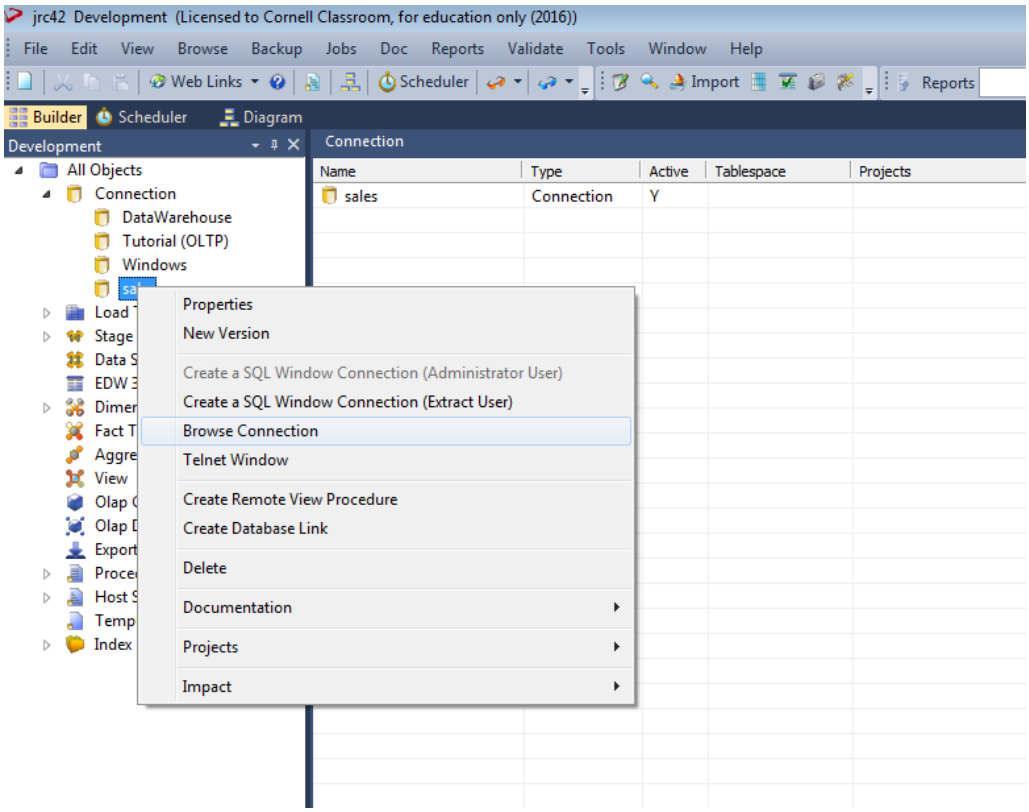


Notice the results in the Result pane below.

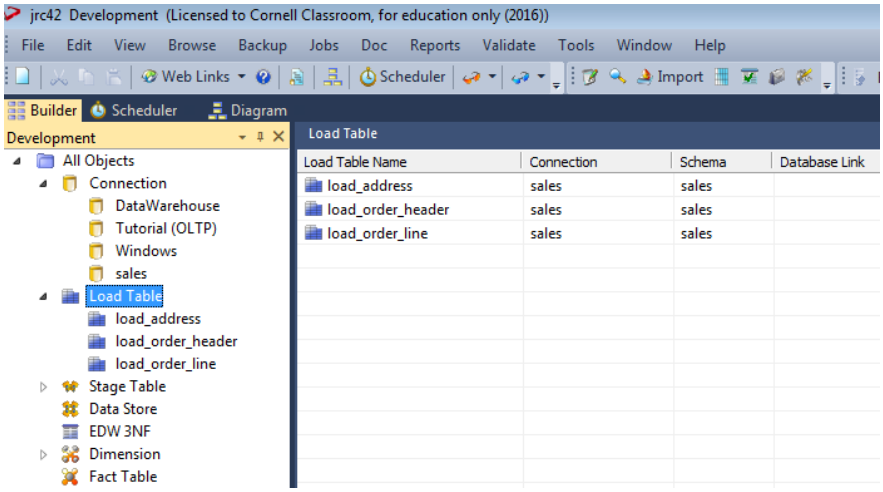


Now we will create another load table and associated stage table for customer data.

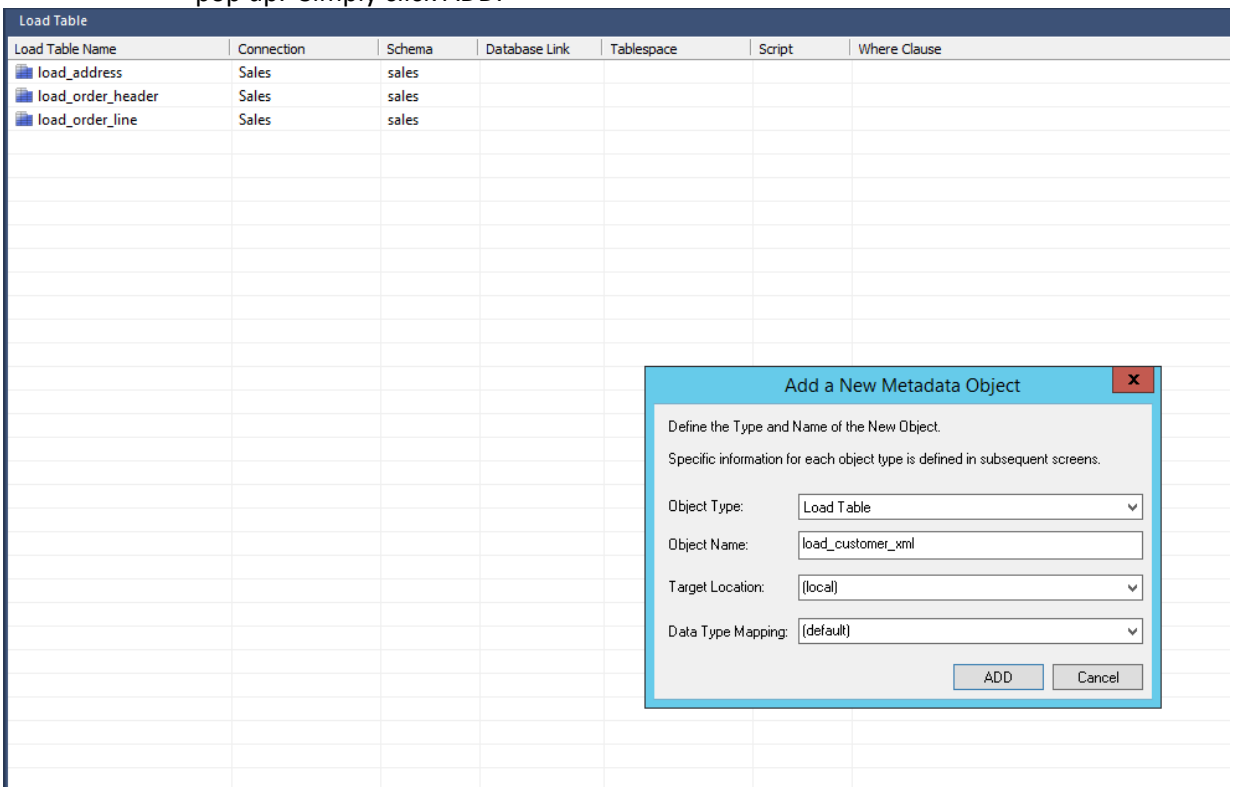
1) In the Builder pane, right click on the “sales” connection and select “Browse Connection”.



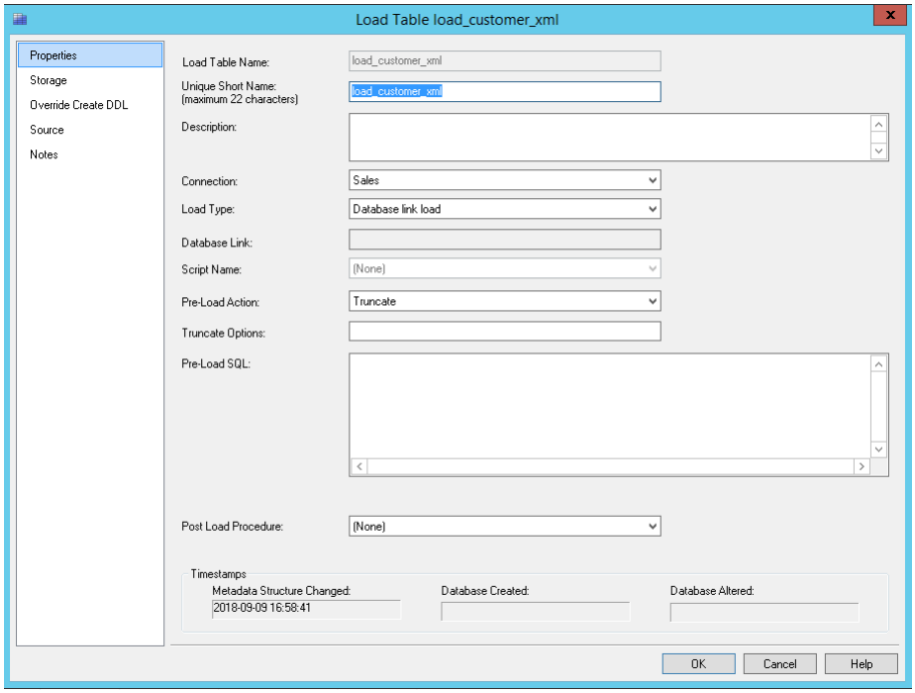
2) Double click “Load Table” under “all objects” in the builder pane. You will then see “Load Table” on the banner of the central pane.



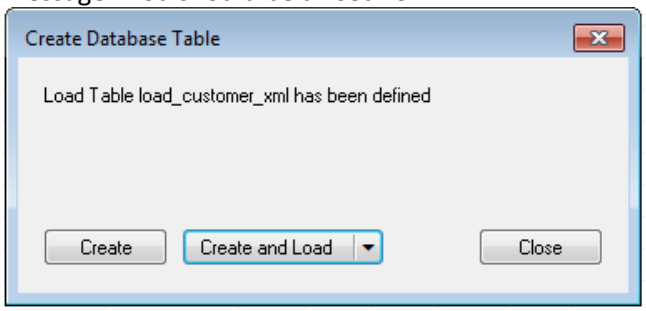
3) Drag the "customer_xml" table from the right pane, showing the sales connection, and drop in the center pain. You will then see the "Add a New Metadata Object" window pop up. Simply click ADD.



4) Click “OK”



5) Click “create and load”, then check the results pane below for a “Load completed” message. You should be all set now.

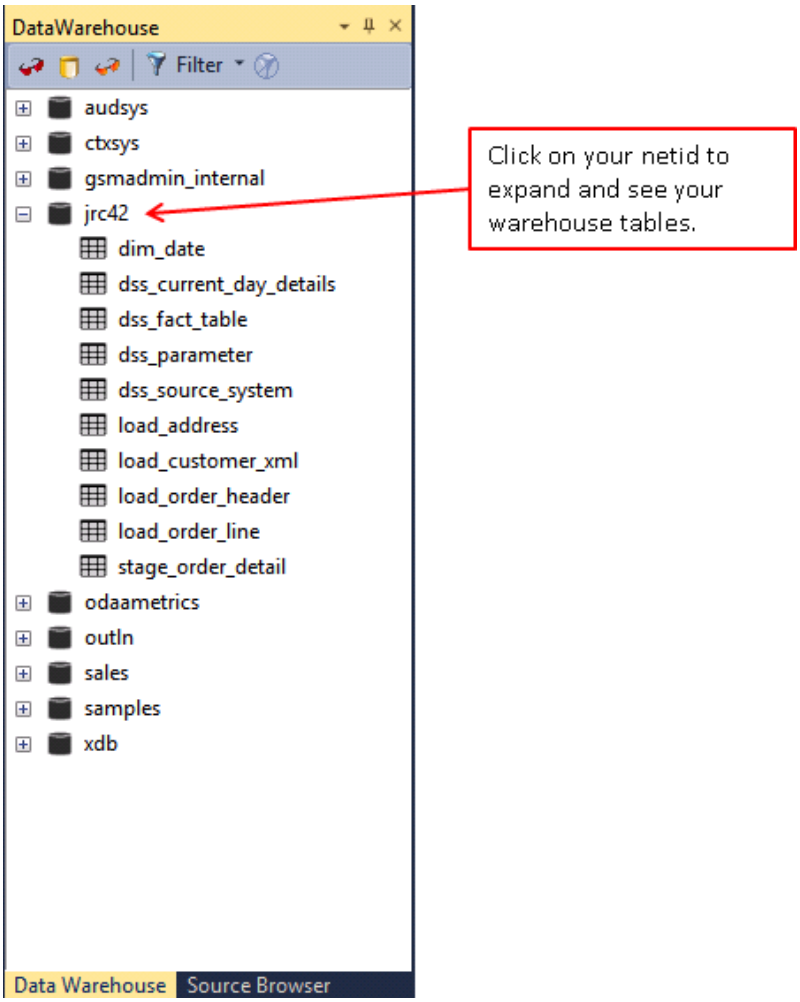


Make the Data Meaningful

With your Dimension Tables, you could populate directly from the load tables themselves but instead you are going to use a staging table to help prepare the data before it is pulled into the dimension. It also gives you the example to cleanse the data and add in calculations that will be useful to the business so staging is always good practice.

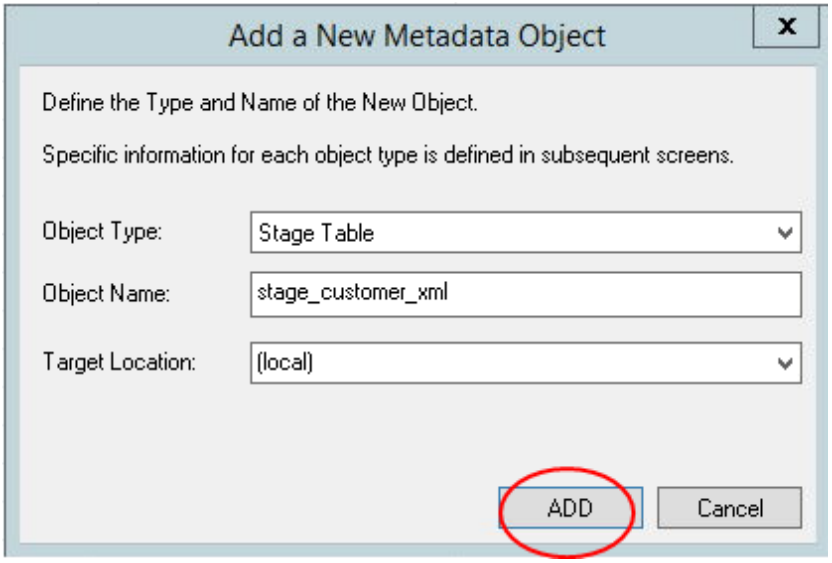
Creating a stage table from one or more (RED) tables is an easy process, so as a start for your new stage table create it as a copy of the original load table.

- In the **connection** browser, select the **Data Warehouse** tab.
- Click on the first **red glasses** icon to refresh the connection and re-browse its content.



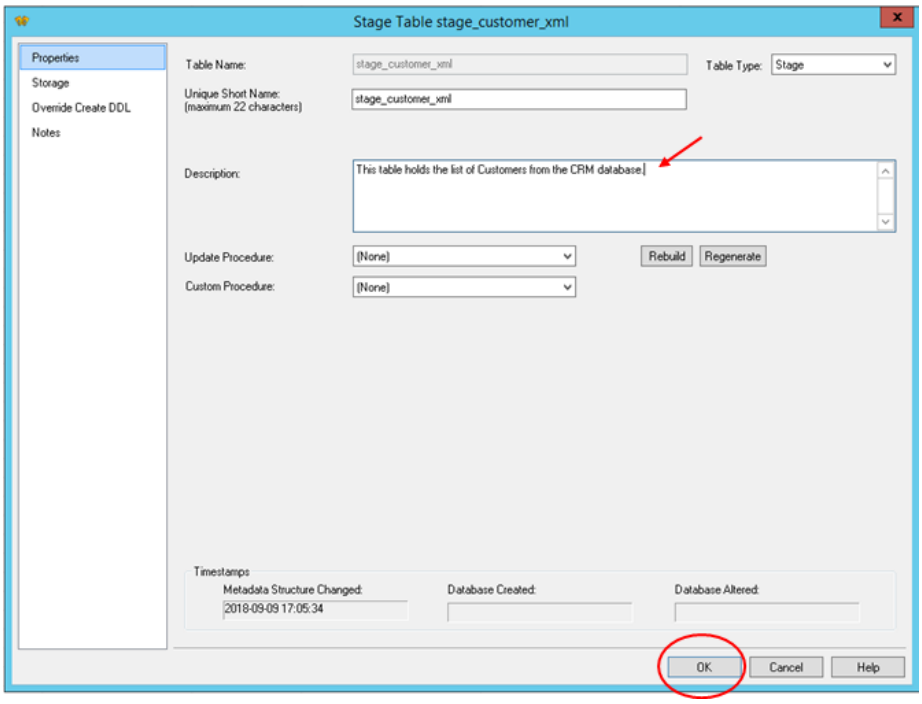
Notice that a few more tables have been created as you progressed through the course.

- Set the work context to Stage Tables by double-clicking on the **Stage Table** group on the left pane - notice the work pane now shows all stage tables created to date.
- From the data warehouse connection browser on the right, drag the **load_customer_xml** table into the middle work pane to start defining the new stage table.
- Accept the default name of **stage_customer_xml** and click **ADD**.



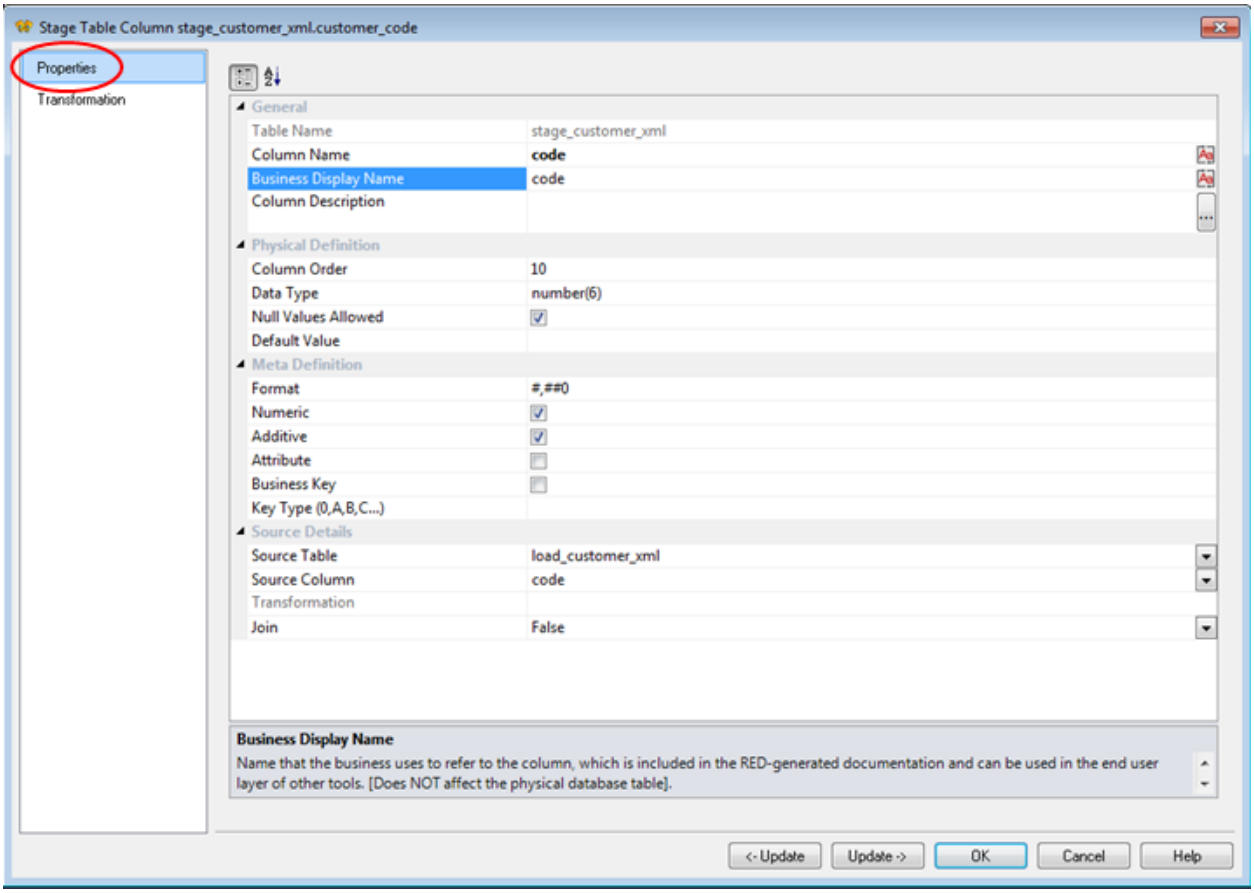
In the Properties dialog, enter a **Description** for other developers to understand why this table exists.

- Type 'This table holds the list of Customers from the CRM database' and click **OK**.



This time the table is only defined and RED presumes you want to do more so does not offer to do anything further. Looking at the new table, the column names are not appropriate for the business, so you need to rectify that now:

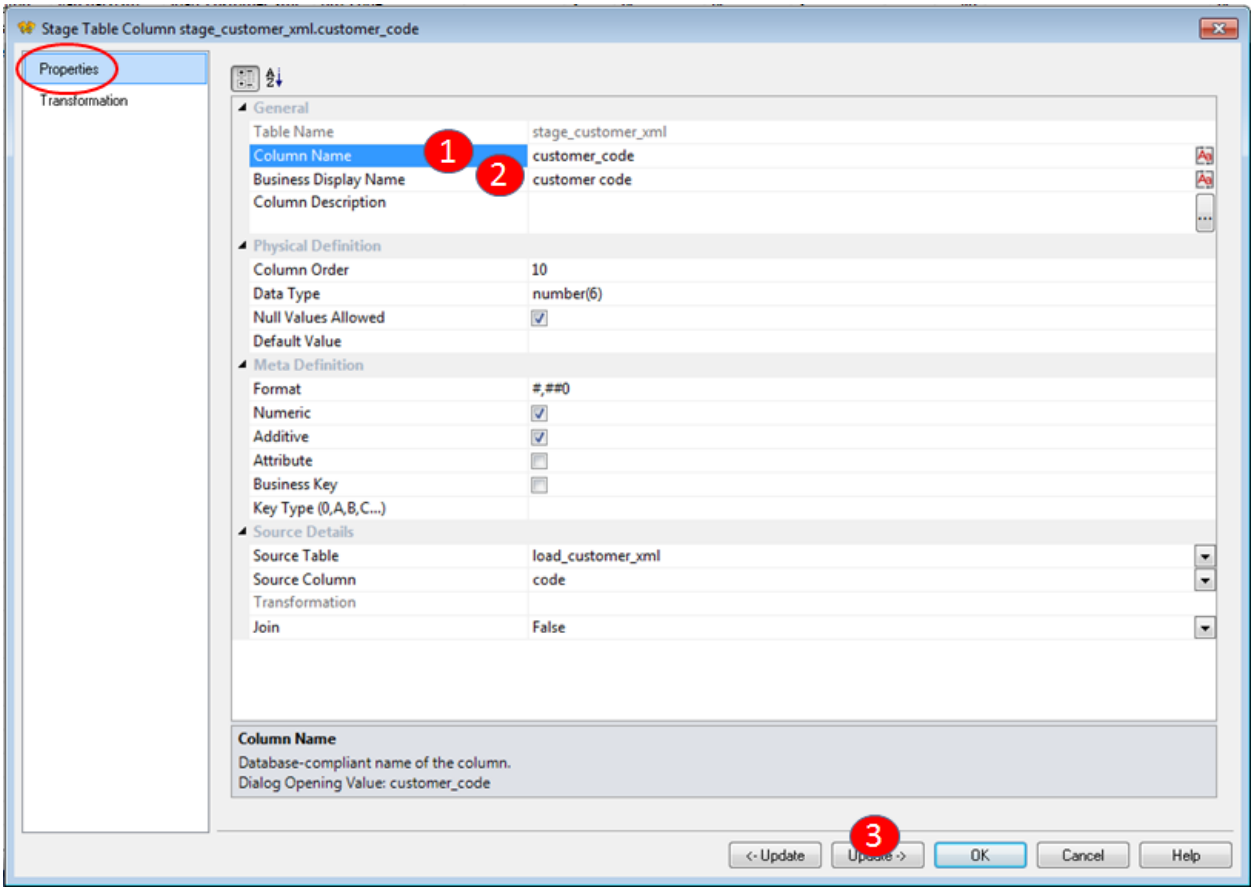
- Click on the table **stage_customer_xml** in the stage table group on the left pane - this sets your context in the middle work pane to be the columns of the selected table.
- Double-click on the column **code** to open up its Properties dialog.



The Business does not refer to Customers as **code**, so change the **Column Name** to **customer_code.** This will affect the structure of the table, but that will be addressed shortly.

- Change the **Business Display Name** to **customer code** so any third party products that read this metadata can use business terms instead of field names in the database. It also ensures that colleagues understand what this column is called in the business itself.

TIP: At this point, you now want to change further columns. You could click on **OK**, then open each column's property dialog one at a time. However, RED has two fast navigation buttons **<-- Update** and **Update ->** which will save, close, and open the previous or next column respectively. This is very handy when updating multiple columns in the same table.



Click the **Update ->** (next) button and update the remaining the fields as below:

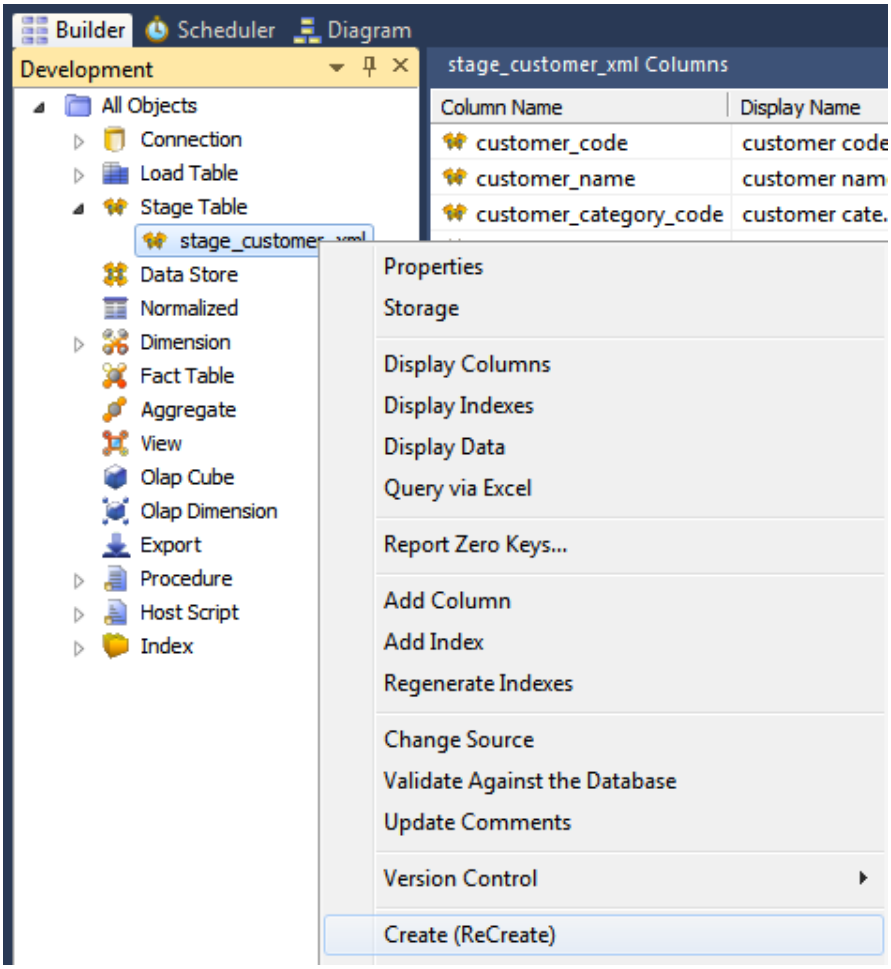
Original Name	Column Name	Business Display Name	Column Description	Nulls allowed
name	customer_name	customer name	The official business name of the customer.	X
cat_code	customer_category_code	customer category code	The category code of the customer.	X
grp_code	customer_group_code	customer group code	The group code of the customer.	X
subgrp	customer_subgroup_code	customer subgroup code	The subgroup code of the customer.	X

This gives you exactly the field names you wish to use moving forward. Wherever you drag and drop these fields into further tables, RED will base the new field(s) on the details specified here i.e. the information given will propagate through the data warehouse.

TIP: You could have done this earlier in the Load Tables instead, but **best practice dictates the Load Table should reflect the original source**.

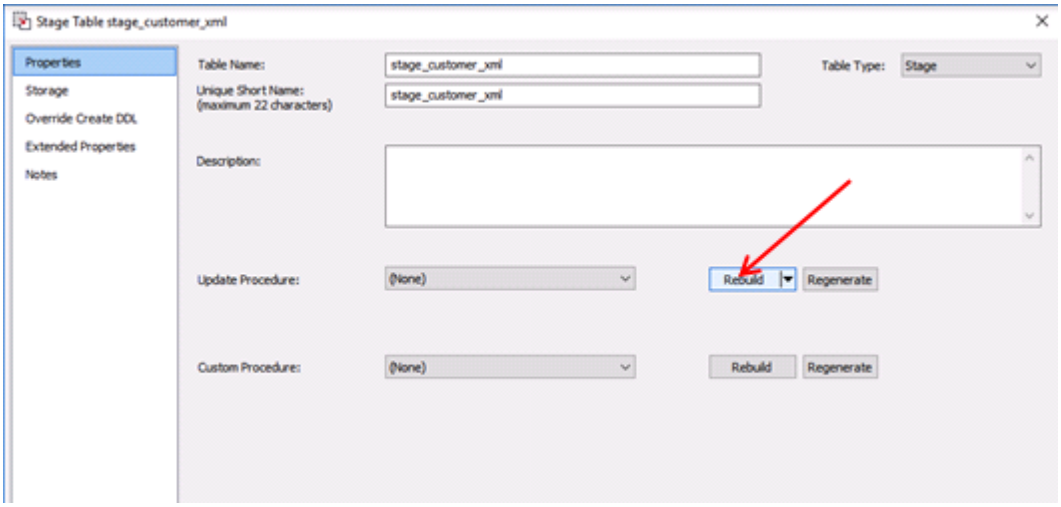
Since this table hasn't been created in data warehouse yet, create it and build a stored procedure to update it.

- From the object browser, in the **Stage Table** group, right-click on the **stage_customer_xml** table and select **Create (Recreate)** to define the table in the data warehouse.

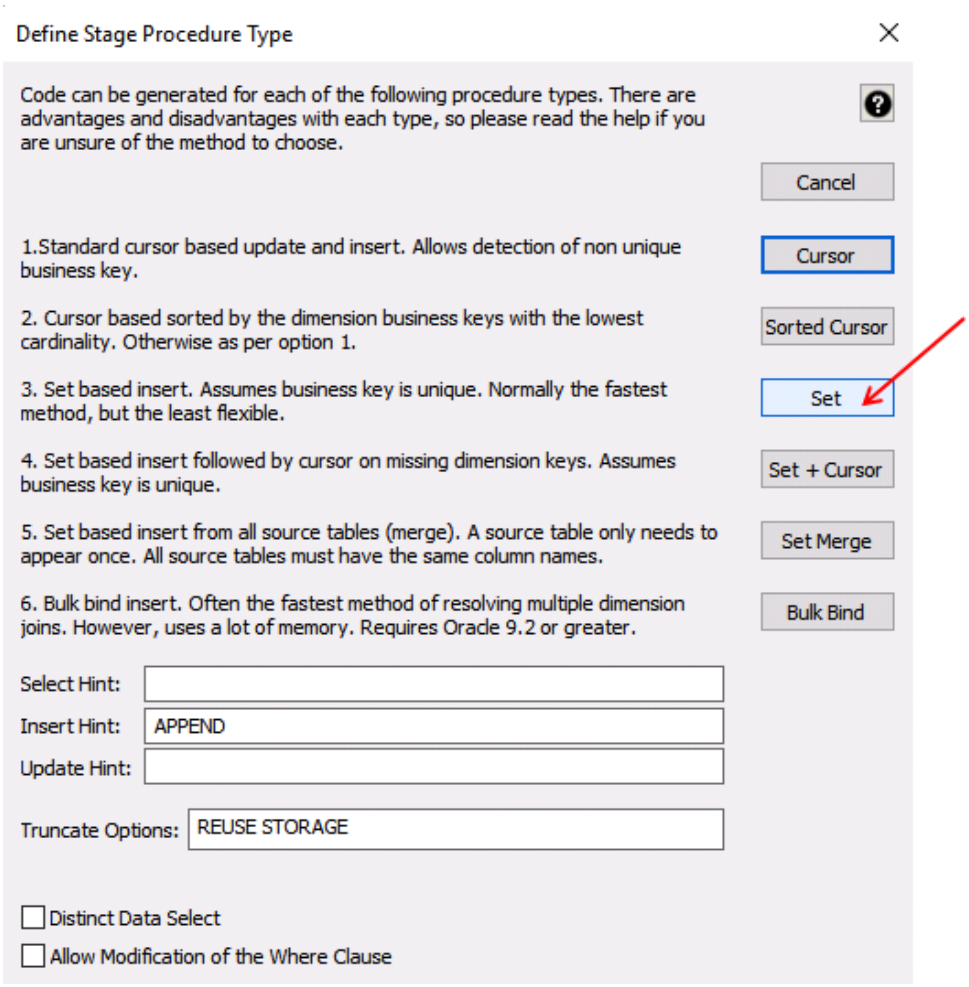


Double-click on **stage_customer_xml** to open up the Properties dialog.

- Click on **Rebuild** next to **Update Procedure** to launch the wizard so RED can learn what it needs to do to refresh this table.

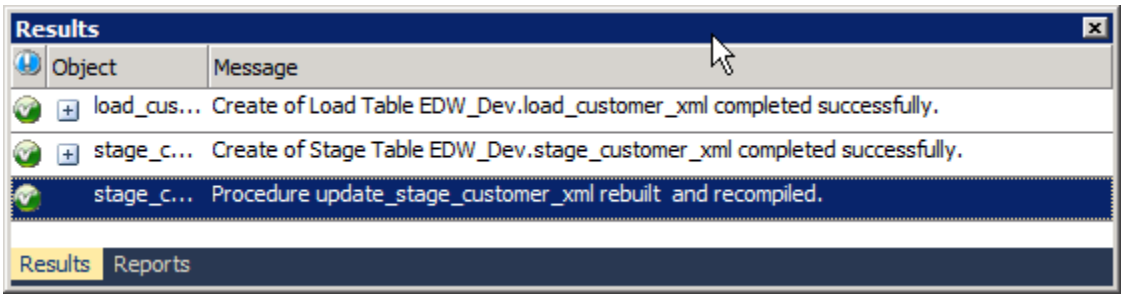


For the moment this is a simple stage table so select **Set** as the fastest update method.



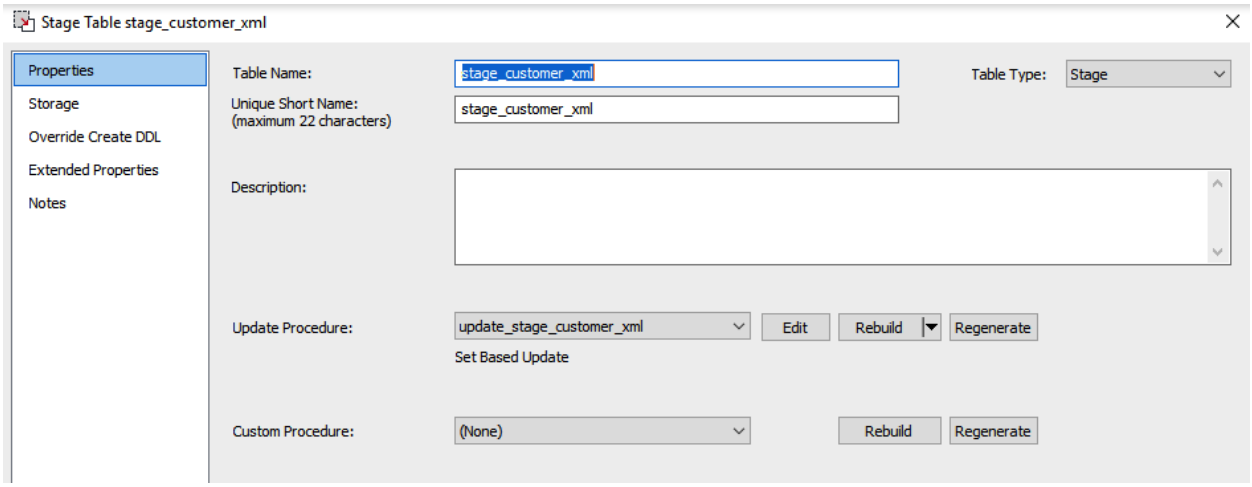
You do not need to use any of the Parameters yet so click **OK** to skip this "Select Parameters" dialog without choosing anything.

- Look in the **Results** pane to see that the procedure was rebuilt and recompiled automatically.

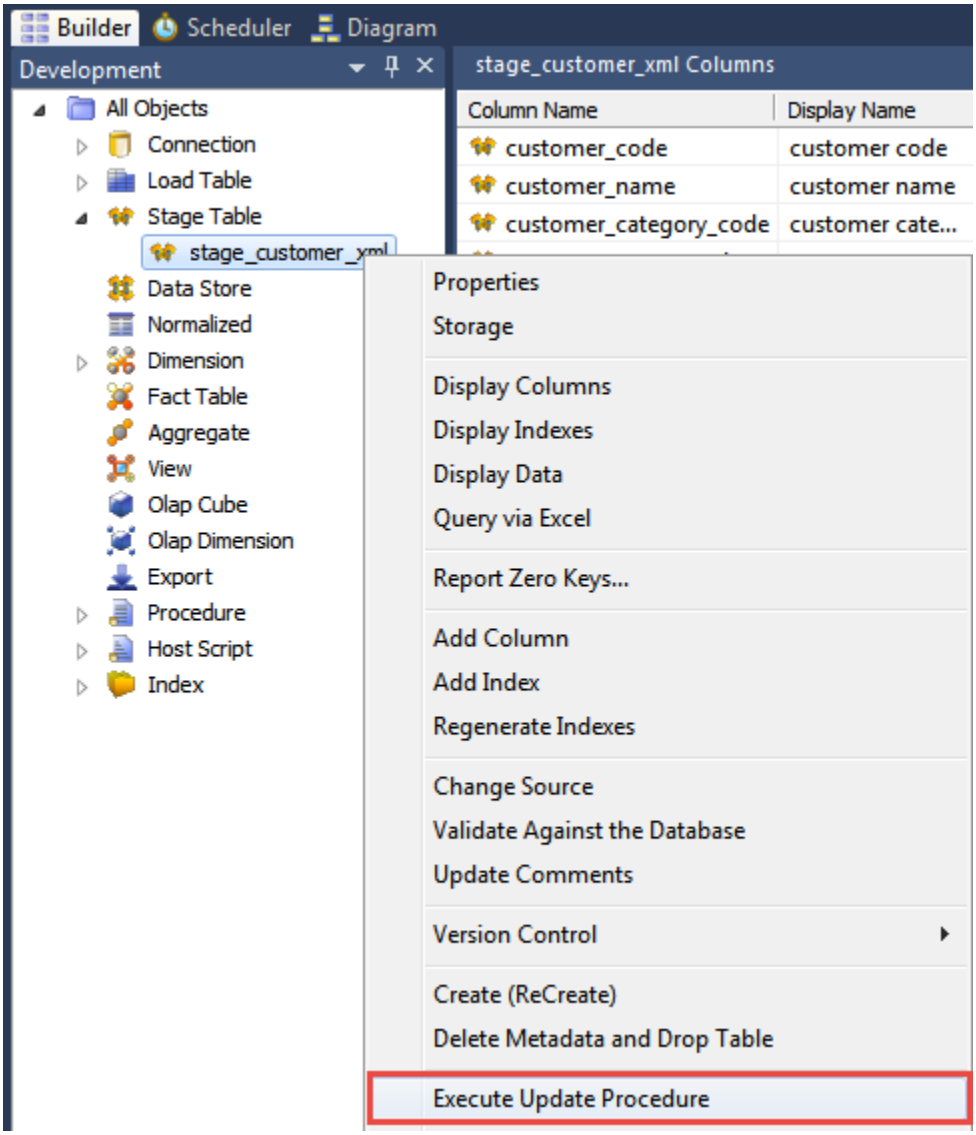


Re-open the **Properties** dialog to see the name of the new Update Procedure for **stage_customer_xml**.

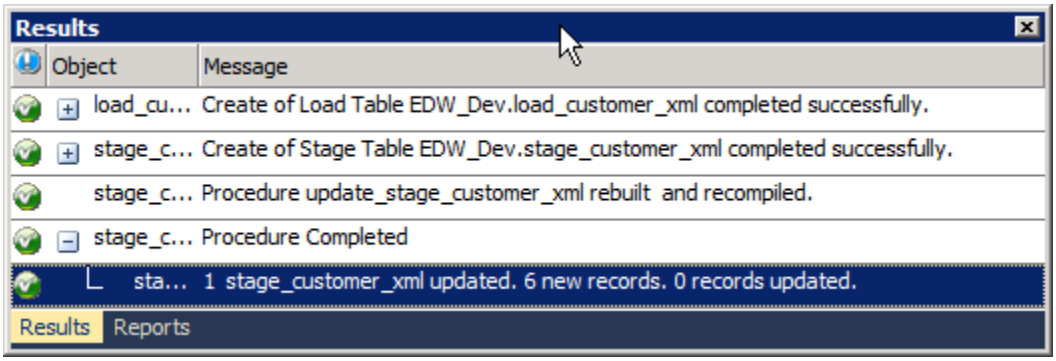
- This is the stored procedure created in the data warehouse that runs whenever this table needs to be updated – either manually, or automatically on a scheduler.
- Close the **Properties** dialog.



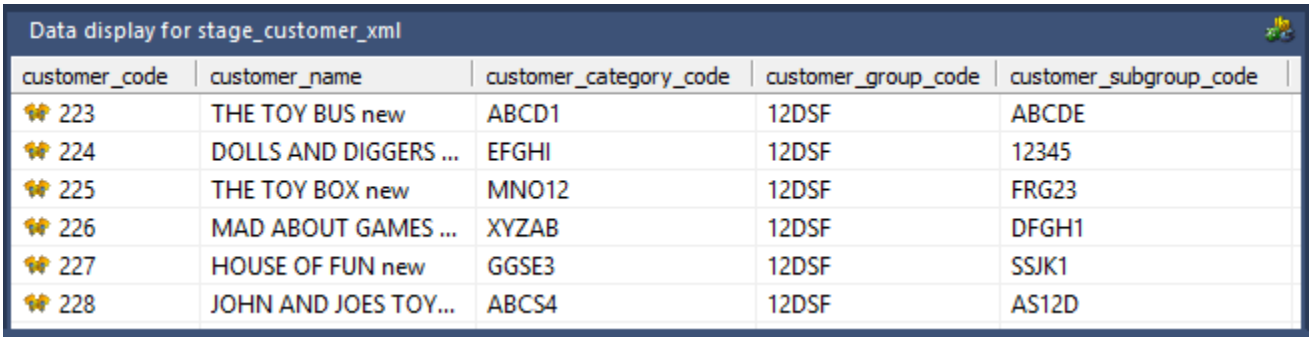
Next, populate the table with the Customer information you need by right-clicking on **stage_customer_xml** in the object explorer in the left pane and selecting **Execute Update Procedure**.



Check the **Results** pane for the success message and to see how many records were processed.



Right-click on **stage_customer_xml** and choose **Display Data** to see the contents of the new stage table.



customer_code	customer_name	customer_category_code	customer_group_code	customer_subgroup_code
223	THE TOY BUS new	ABCD1	12DSF	ABCDE
224	DOLLS AND DIGGERS ...	EFGHI	12DSF	12345
225	THE TOY BOX new	MNO12	12DSF	FRG23
226	MAD ABOUT GAMES ...	XYZAB	12DSF	DFGH1
227	HOUSE OF FUN new	GGSE3	12DSF	SSJK1
228	JOHN AND JOES TOY...	ABCS4	12DSF	AS12D

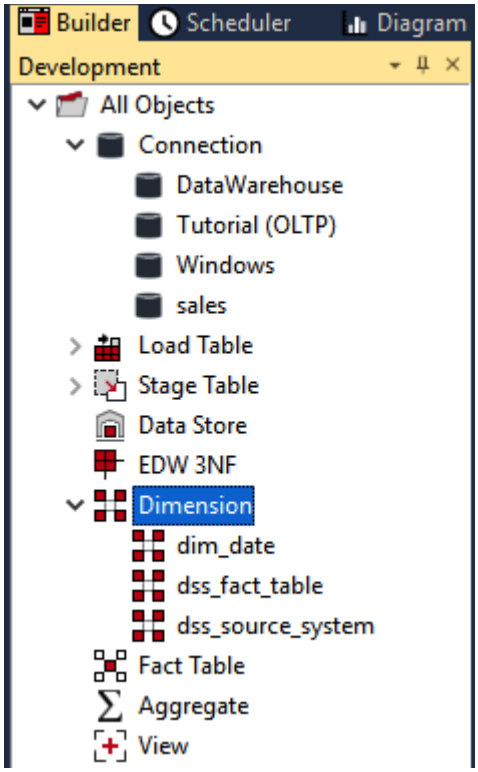
Finally, you have a **Stage Table** of the Customers within your organization in a business context. However, what you really need is a single source of Customer information within your organization - one where you see all variants of the same Customer (i.e. what the customer's identity is in each of the original source systems) and all appropriate information regarding each customer.
For that you will need a **Dimension.**

Create a Dimension

Your stage_customer_xml table holds the Customer information mapped in a business context from your load table which holds data extracted from the source system itself. This way the stage files become your working tables holding transient data to be loaded into further tables.
A dimension table will keep all of the Customer information and update it as required. It is designed to be a more persistent store of the Customer information. The information required has already been prepared in the stage table so creating the dimension itself will be very simple. Dimensions, just like stage tables are fed from the data warehouse so all the knowledge you require is stored on the one platform i.e. that of the data warehouse.

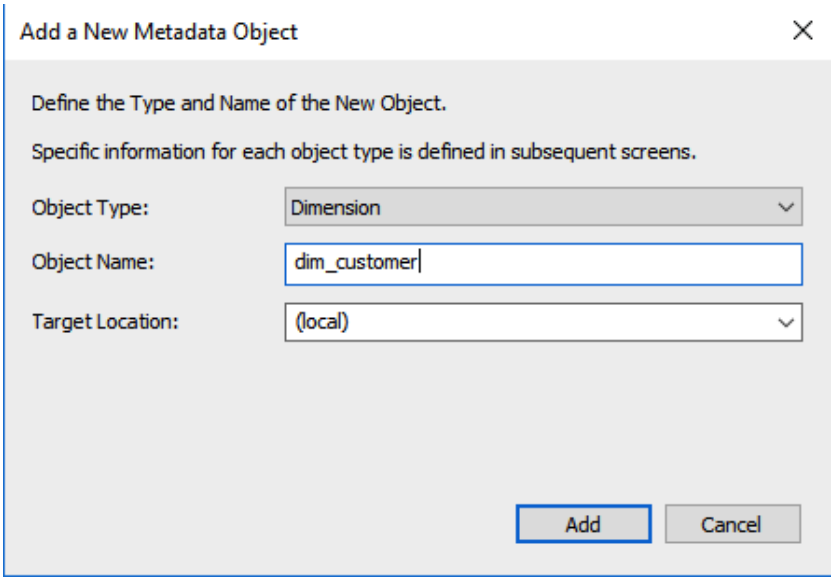
In the **Connection** Browser, select the data warehouse tab and click on the glasses to refresh the connection. You will see: "browse source tables/panes in right pane" when you hover over the glasses.

- Set the context of the work pane by double-clicking on the **Dimension** group of the **All Objects** tree in the left pane.
- The work pane will now show all dimensions. Even though you haven't created any yet, dim_date appears in the list.
  This is a pre-built dimension that RED creates for you and that will be explored in the next sections.



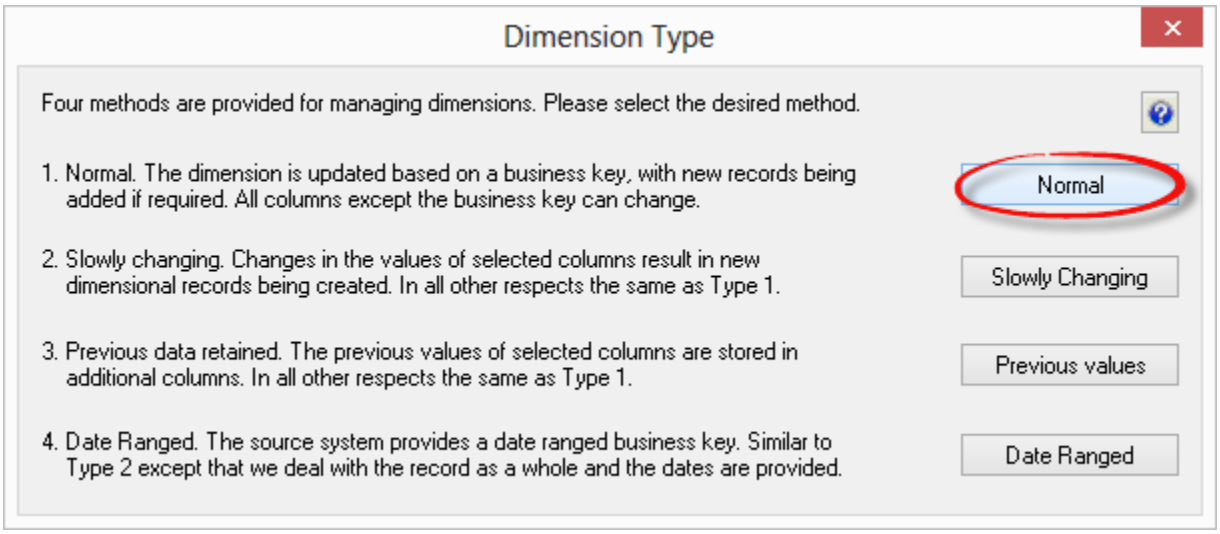
Refresh the data warehouse browser again on the right pane and drag the **stage_customer_xml** table into the middle work pane to create **dim_customer**.

- Update object Name to "**dim_customer**"
- Click **Add**



A dialog appears to start the definition with the fundamental question of how much history you wish to track for each Customer in this case:

- For the moment the **Normal** dimension type is good enough. Leave tracking until later and select N**ormal**.



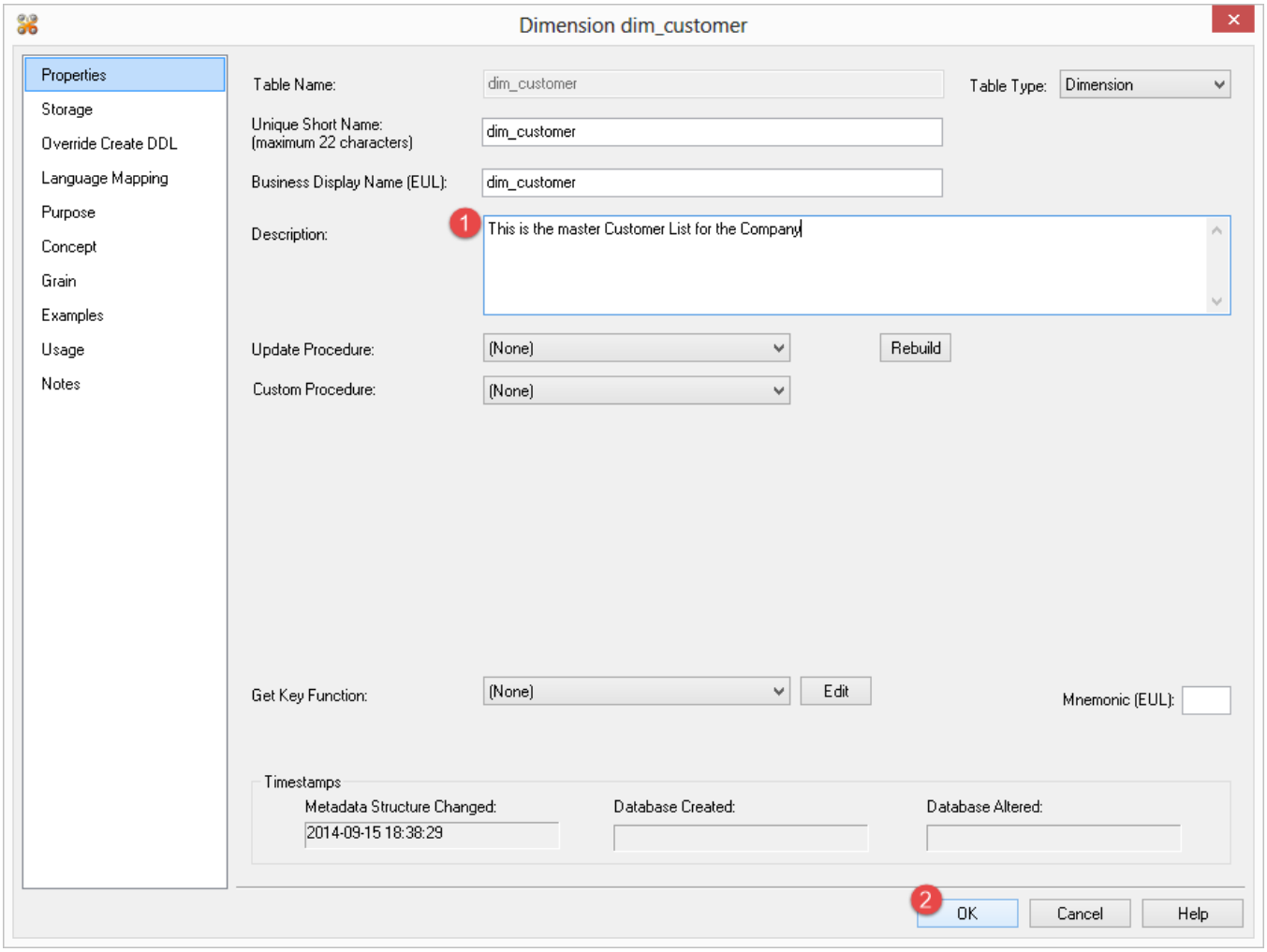
The Properties dialog of the table appears but notice that this time you have many more sections to consider in this essential part of your data warehouse.
As one of the critical success factors of any data warehouse is understanding what it holds, these extra sections are all geared towards gathering the IP of the data warehouse build:

- Purpose – what is the point of having this table – what challenge it attempts to solve
- Concept – how does this play it's part in the bigger picture
- Grain – what level of detail does this table go down to

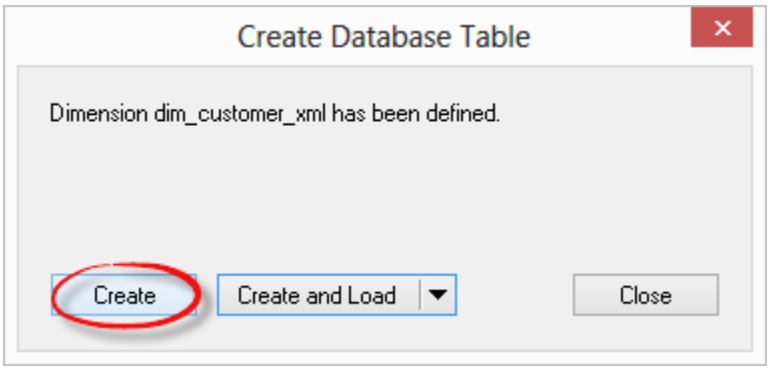
- Examples – working examples of what sort of information is in this table
- Usage – clarification of how this table should be used in relation to others
- Notes - any notes for the dimension table

NOTE: The names of the sections are completely at your discretion and can be changed in the generated documentation using Tools > Options menu, Documentation tab. By default, they are not enabled for load and stage tables, but can be if required. Up to 4000 characters of information can be stored in each field.

Set the **Description** to 'This is the master Customer List for the Company'. Click **OK** to accept the remaining defaults and define the table in the RED metadata.



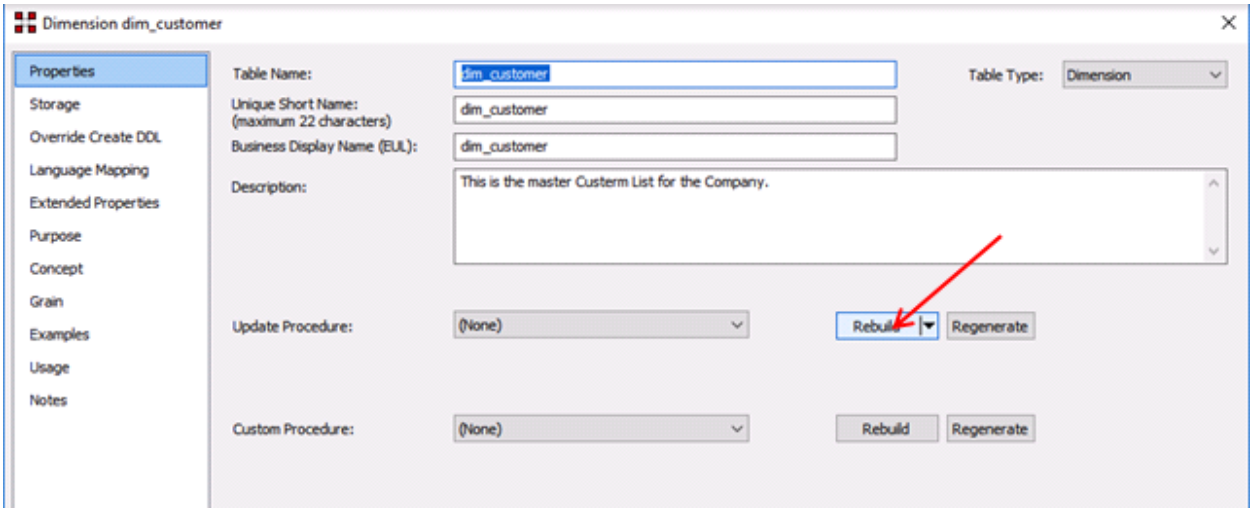
Click **Create** to define the table in the data warehouse.



With the table ready for data, RED needs to know how to populate the data in the table.

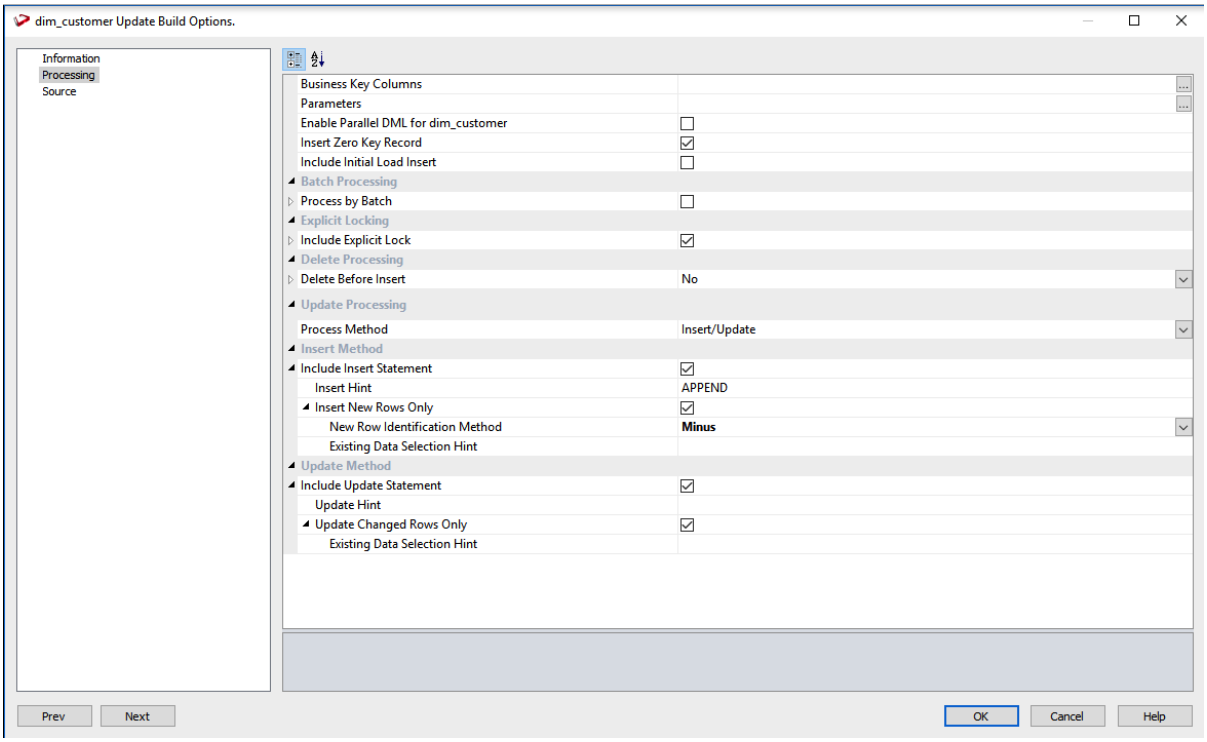
- Double-click on **dim_customer** in the Dimension group on the left pane to re-open the **Properties** dialog
- To the right of the **Update Procedure** drop-down box, click **Rebuild.**

TIP: You could have done this while the Properties dialog was open earlier.



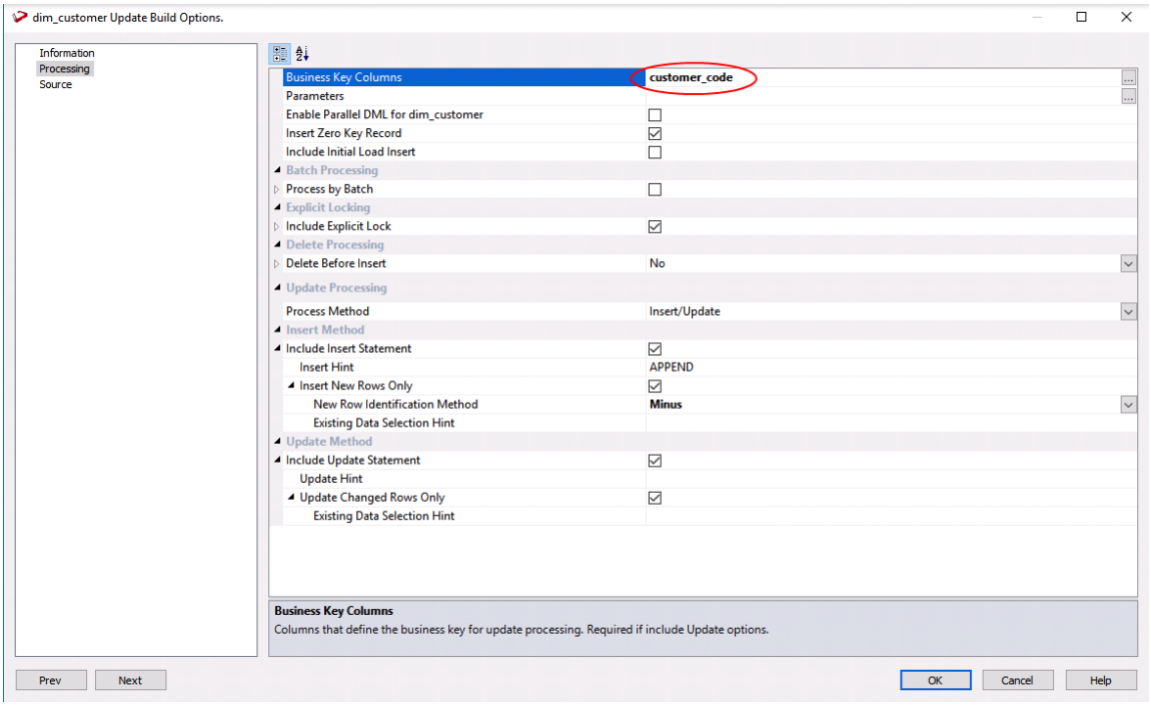
Earlier you saw you could benefit from using a **Set update** in your stage table so here select **Set** as well and click **OK**.

The **Dimension Update Build Options** dialog allows you to make the best use of the underlying database platform.



In the **Define Dimension Business Key Columns** dialog, RED needs to understand from a business perspective what column(s) constitute the unique identifier of, in this case the Customer information. The field **customer_code** identifies each unique customer in your dimension.

- On the Processing tab, click on the ellipsis button to the right side of the dialog to **Define Dimension Business Key Columns.**
- Double-click on **customer_code** to add it to the **Business Key List** and click **OK** to complete the definition of the Update Procedure for this dimension table.



NOTE: It is not unusual for multiple columns to be specified in order to determine the unique records.

The columns selected in the **Define Dimension Business Key Columns** dialog form the basis of the lookup in the **Get Key Function** stored procedure unique to dimension tables (at the bottom of the Properties dialog). Here you specified that the field **customer_code** was the business key in the transactions to uniquely identify the customer. Therefore, the get key function will scan the dimension on the **customer_code** column to determine the dimensional key (primary key) to use in the data warehouse to uniquely identify any variants of the business key. Furthermore, as numeric codes, these tend to be more efficient keys than regular business keys that tend to be alpha-numeric.

Right-click on **dim_customer** and select **Execute Update Procedure**.

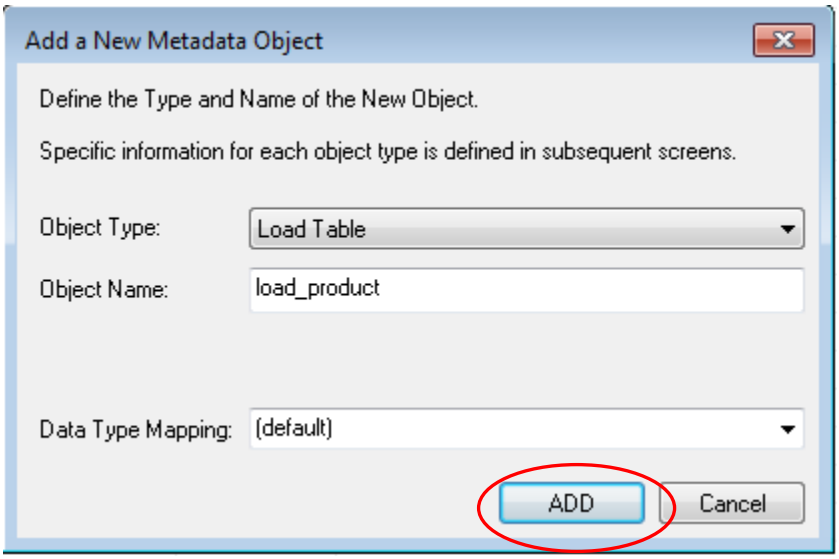
- Notice in the **Results** pane, the procedure has been built, compiled and executed, so have a look at the data now.
- Right-click on the **dim_customer** table and choose **Display Data** to see a sample of the contents of the table.

NOTE: There is no option to truncate the table on update of the dimension as there is with Load tables. This is because the nature of a Dimension table is that the data grows and is rarely removed. It is essential that the dimensions hold every dimensional item referenced in the

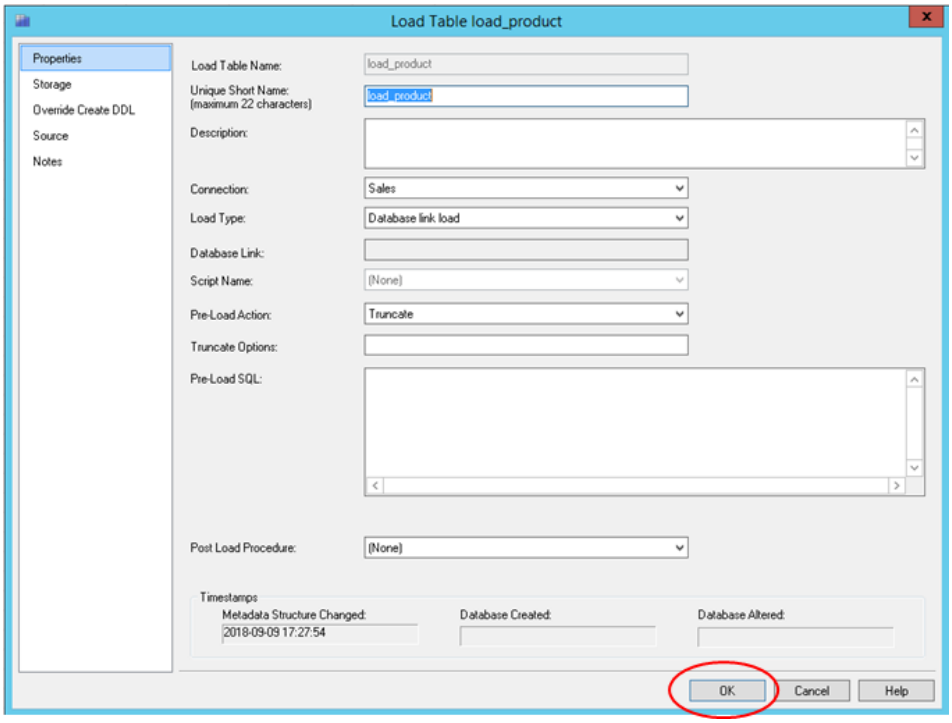
transactional data within the data warehouse. If this is not the case, then the transactions for these items could be lost or mapped out of their normal structure into an 'unknown' structure. Having the dimension hold just the current dimensional structure will result in records for historical dimensional items being mapped to 'unknown'. Furthermore emptying and rebuilding the dimensions COULD give the business keys of the dimension new primary / dimensional keys resulting in potentially wrong joins between the facts and the dimensions.

The same way you created a customer dimension, you will need to create a product dimension.

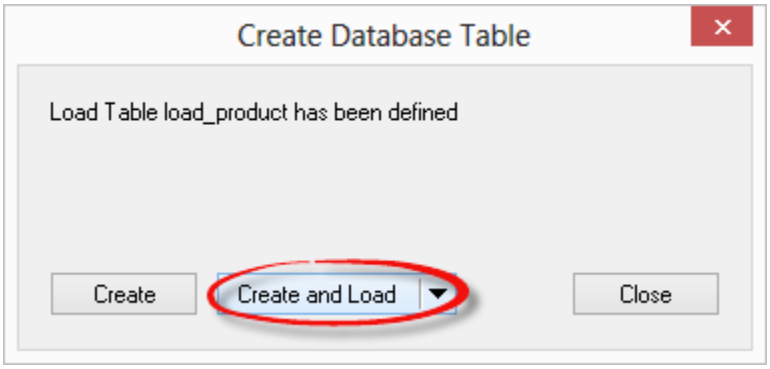
- Browse the **Sales** connection.
- In the left-hand object explorer pane, double-click on **Load Table** in the **All Objects** tree and drag **product** from the right pane into the middle working pane.
- Check that the object name is **load_product** and click **ADD.**



There are no edits to make on the properties page, so click OK.



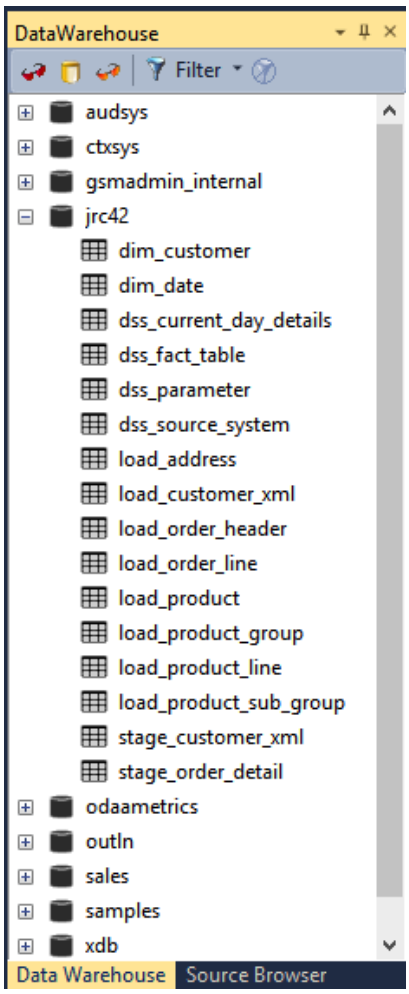
Click **Create and Load** to bring the table definition into the warehouse and populate it with the current data.



In the same way, create the load tables **load_product_line**, **load_product_group** and **load_product_sub_group**.

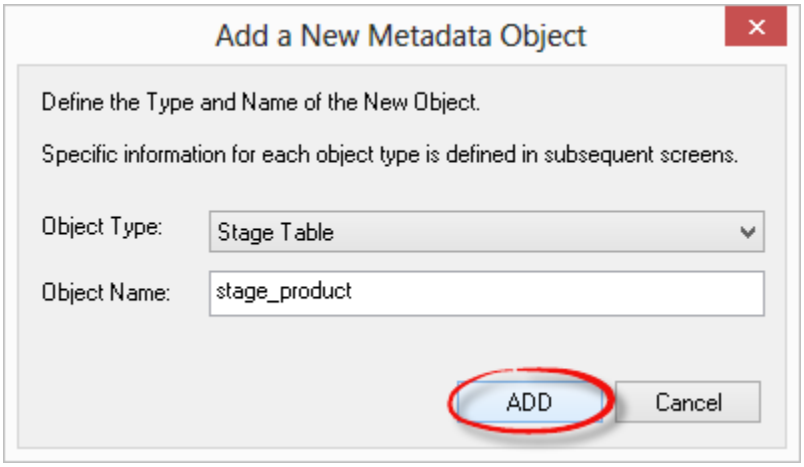
TIP: Don't forget to double-click on **Load Table** in the **All Objects** group in between each load.

Next go to the Connection Browser, select the data warehouse tab and click on the red glasses to refresh the connection.



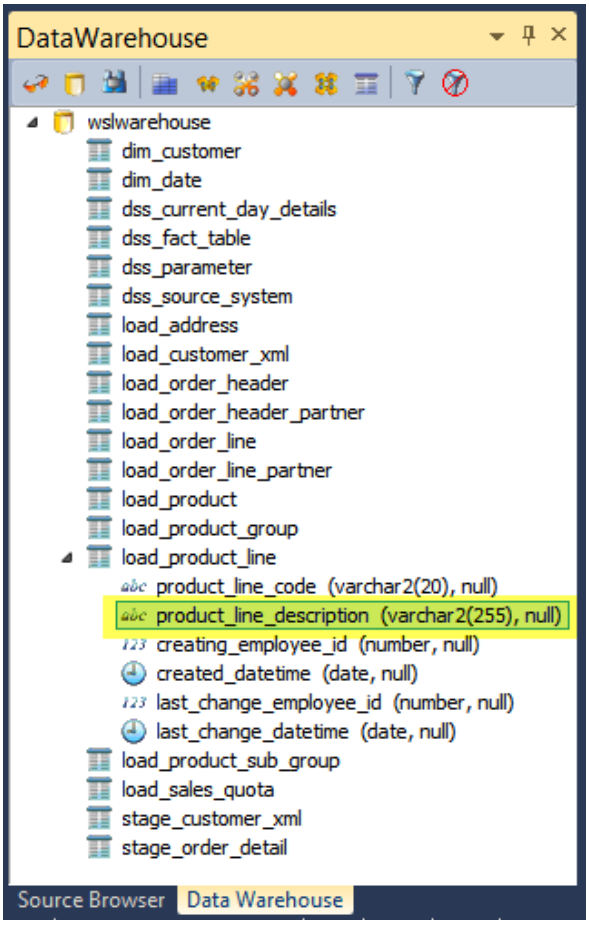
Set the context of the work pane by double-clicking on **Stage Table** in the **All Objects** tree and drag **load_product** from the right pane into the middle working pane.

- Check that the object name is **stage_product** and click **ADD**.
- Click **OK** on the **Properties** dialog.



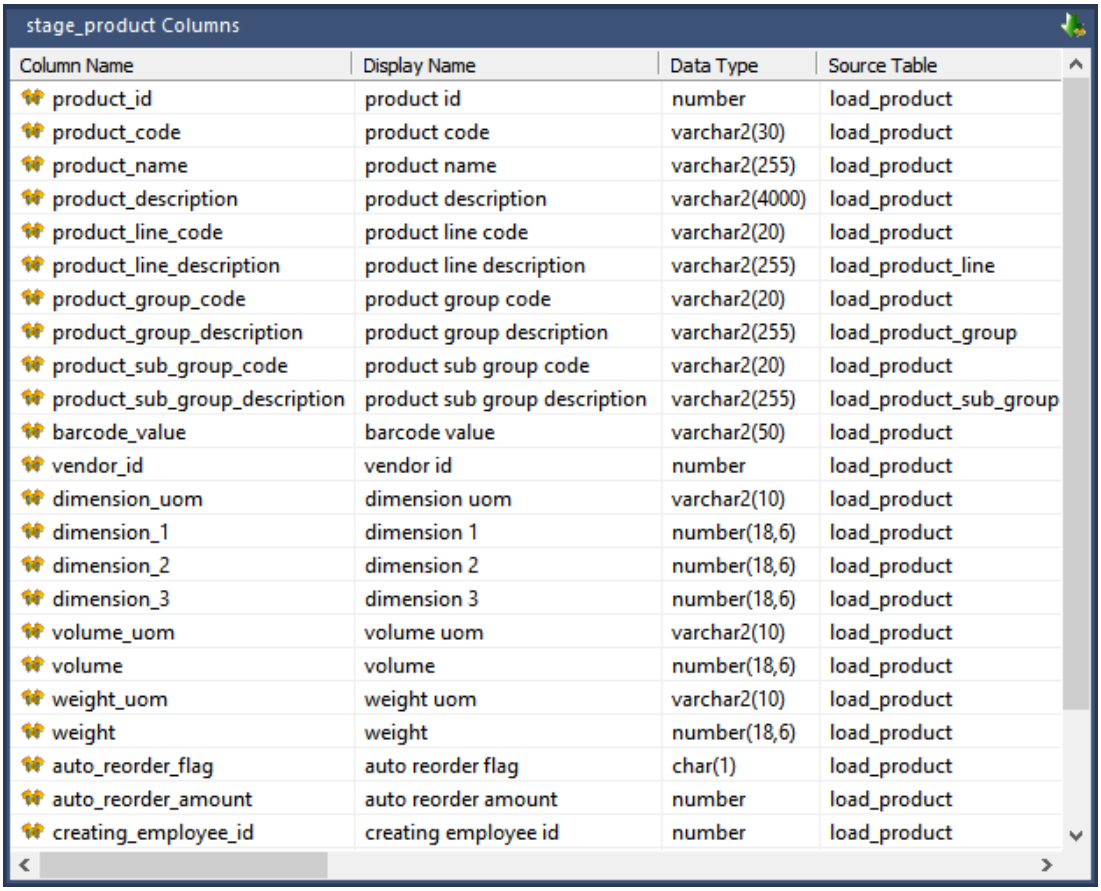
Double click on the "stage_product" stage table you just created in the previous step. This updates the middle working pane with the columns from the newly created "stage_product" table.

In the right pane, double click on **load_product_line** and drag the **product_line_description** column into the middle pane onto the column below **product_line_code**.



- In the right pane, double click on **load_product_group** and drag **product_group_description** into the middle pane onto the column below **product_group_code**.
- In the right pane, double click on **load_product_sub_group** and drag **product_sub_group_description** into the middle pane onto the column below **product_sub_group_code**.

Re-order the columns so that product details are first, then **product_line** details, then **product_group** and finally **product_sub_group**.



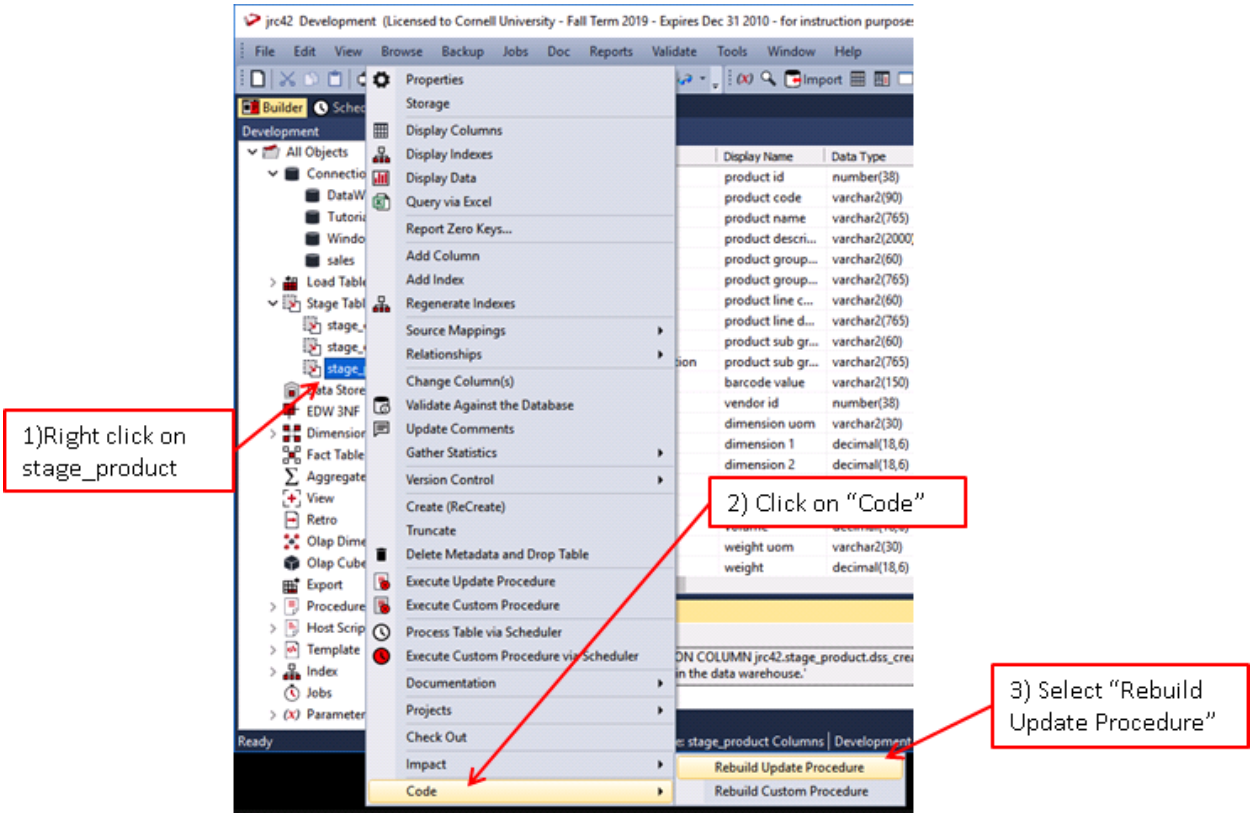
stage_product Columns

Column Name	Display Name	Data Type	Source Table
product_id	product id	number	load_product
product_code	product code	varchar2(30)	load_product
product_name	product name	varchar2(255)	load_product
product_description	product description	varchar2(4000)	load_product
product_line_code	product line code	varchar2(20)	load_product
product_line_description	product line description	varchar2(255)	load_product_line
product_group_code	product group code	varchar2(20)	load_product
product_group_description	product group description	varchar2(255)	load_product_group
product_sub_group_code	product sub group code	varchar2(20)	load_product
product_sub_group_description	product sub group description	varchar2(255)	load_product_sub_group
barcode_value	barcode value	varchar2(50)	load_product
vendor_id	vendor id	number	load_product
dimension_uom	dimension uom	varchar2(10)	load_product
dimension_1	dimension 1	number(18,6)	load_product
dimension_2	dimension 2	number(18,6)	load_product
dimension_3	dimension 3	number(18,6)	load_product
volume_uom	volume uom	varchar2(10)	load_product
volume	volume	number(18,6)	load_product
weight_uom	weight uom	varchar2(10)	load_product
weight	weight	number(18,6)	load_product
auto_reorder_flag	auto reorder flag	char(1)	load_product
auto_reorder_amount	auto reorder amount	number	load_product
creating_employee_id	creating employee id	number	load_product

Right-click on **stage_product** in the left pane and select **Create (ReCreate)**. The builds the newly defined stage_product table in the database.

Now you need to create the code to populate the new stage_product table.

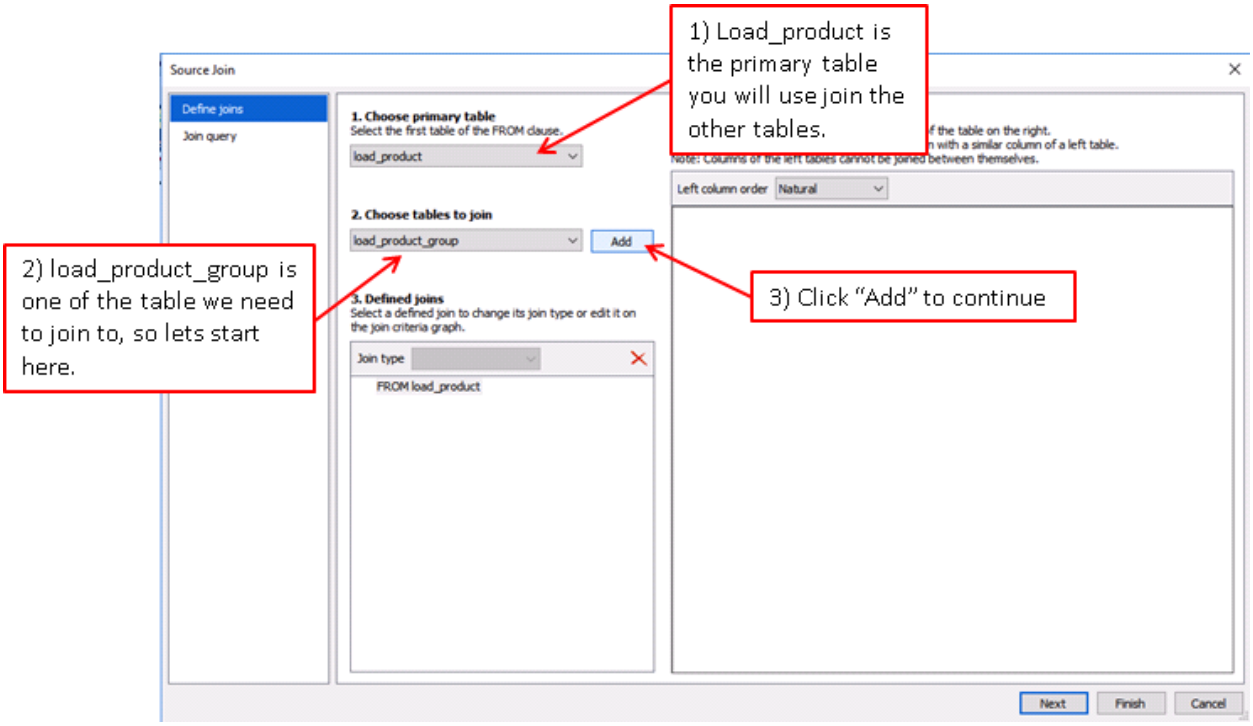
- Right-click on **stage_product** in the left pane, select **Code** and then **Build Update Procedure**.



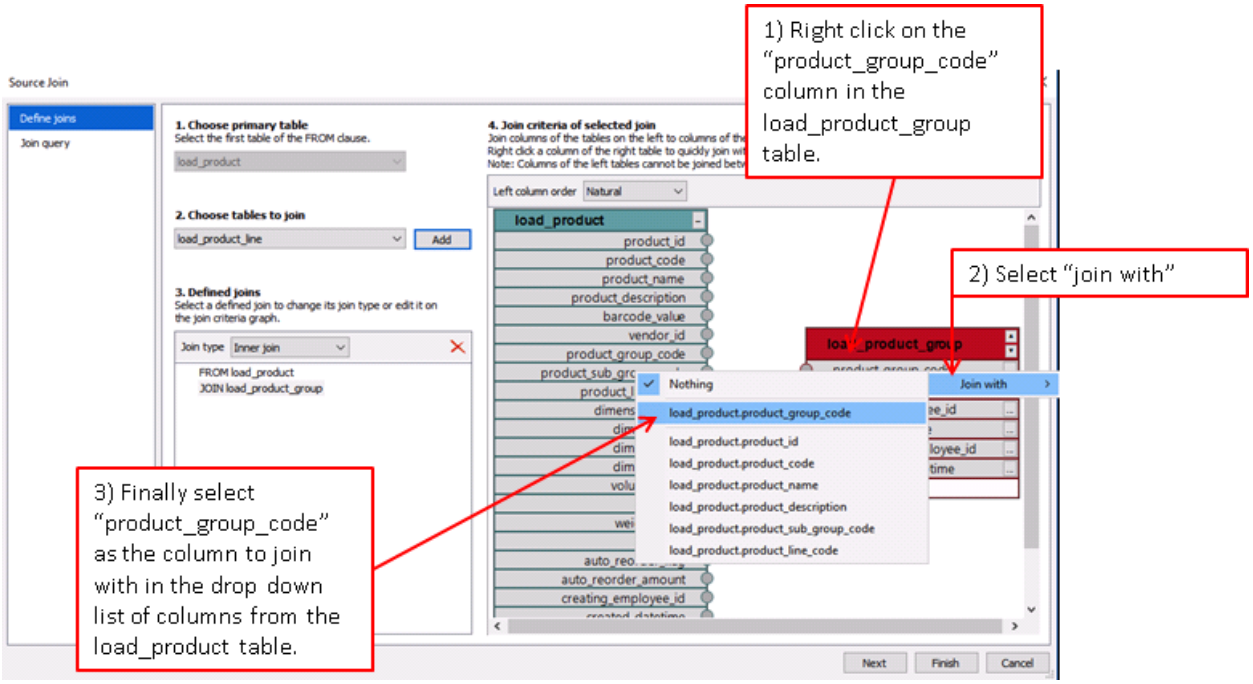
- Select a **Procedure Build Type** of **Set**.
- Click **OK** on the **Parameters** dialog without selecting anything
- Now you need to define the relationships (join paths) between the four load tables which will populate the new stage_procedure table.
- Here are the join relationships you will define in the next few steps.
    - Load_product.product_group_code = load_product_group.product_group_code (inner join)

- Load_product.product_line_code = load_product_line.product_line_code (inner join)
- Load_product.product_sub_group_code = load_product_subgroup.product_sub_group_code (right outer join)

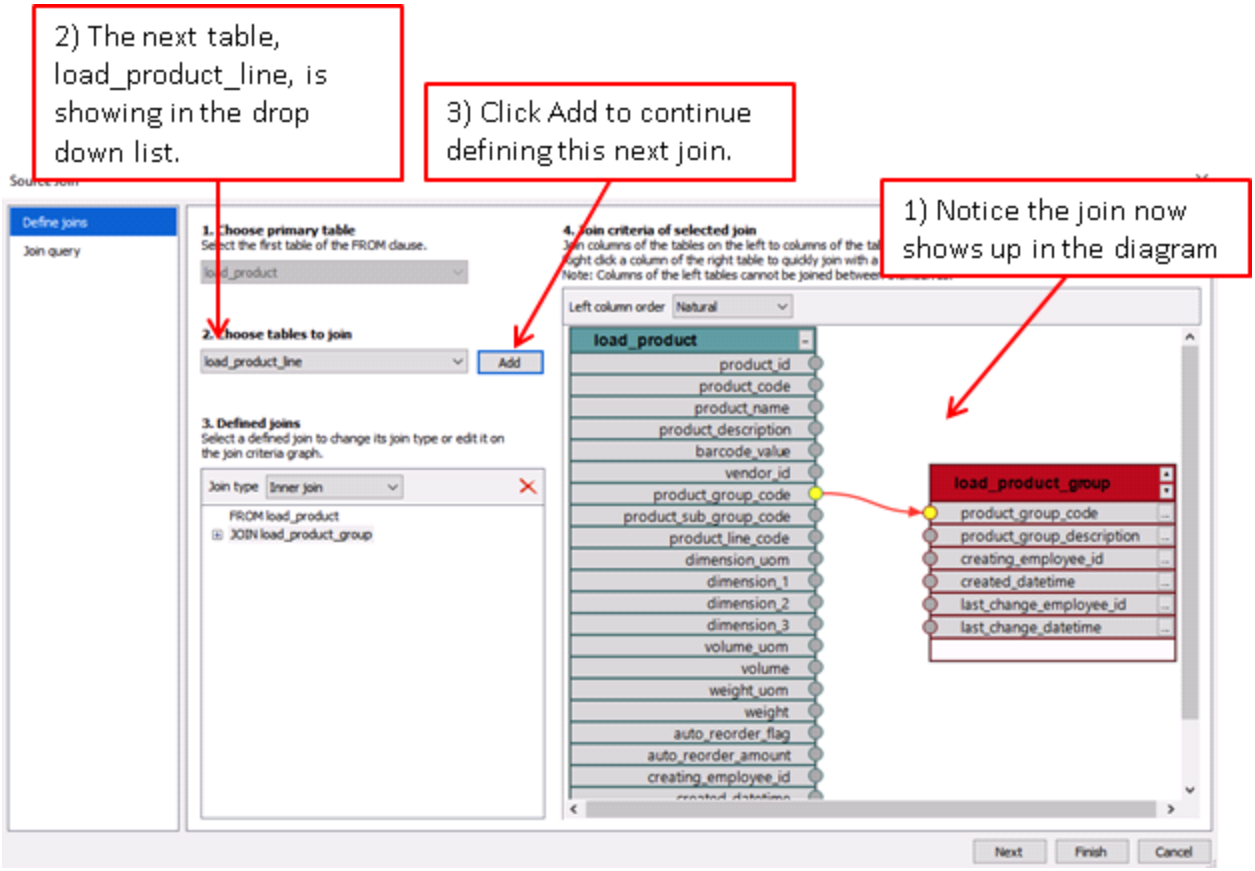
Begin defining the relationships between the four source tables, using the Source Join page.



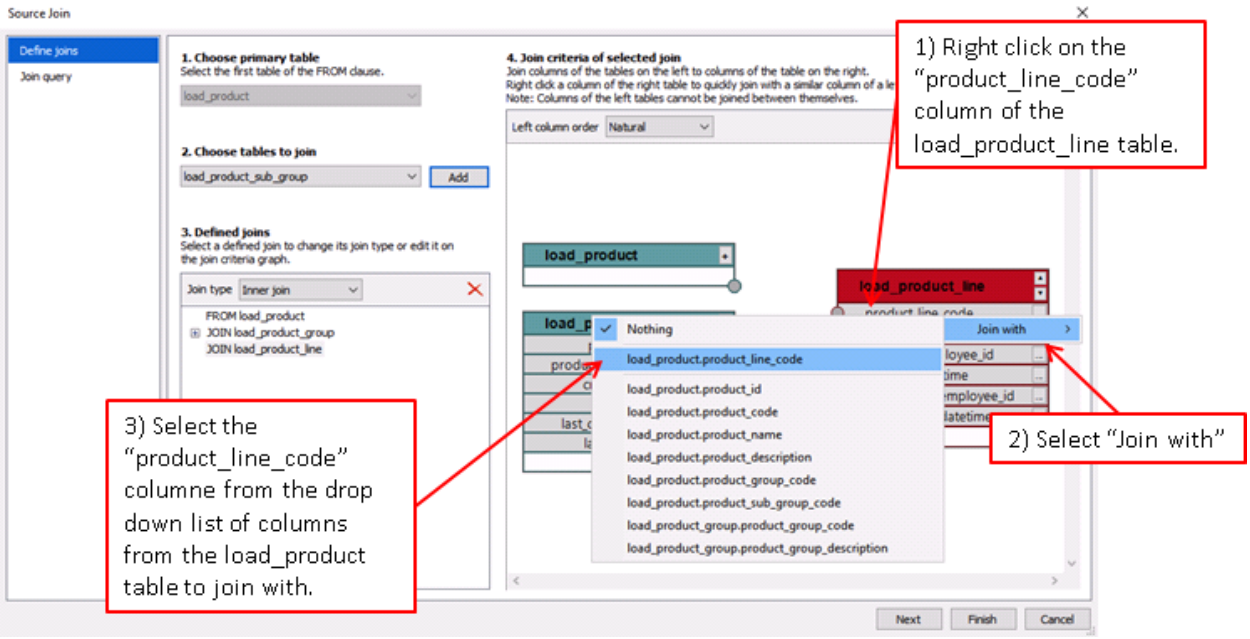
Define the join between the load_product and load_product_group with the below steps.



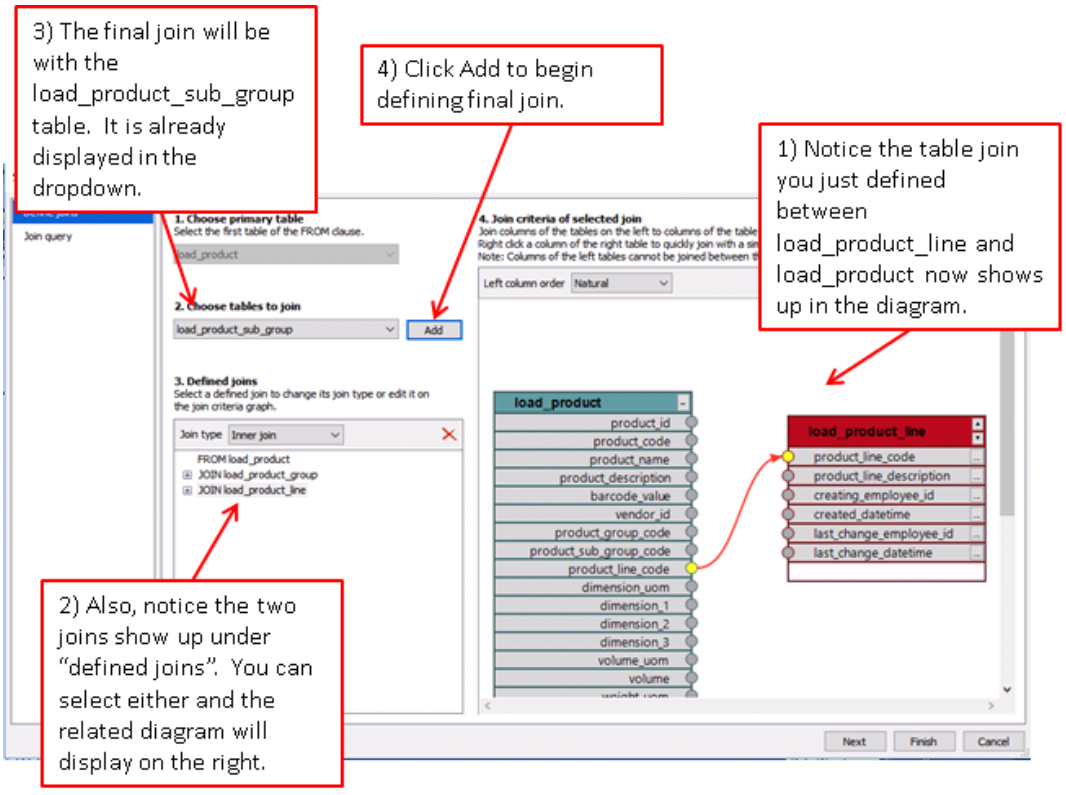
The first join now shows up in the diagram on the left. Now start defining the second join.



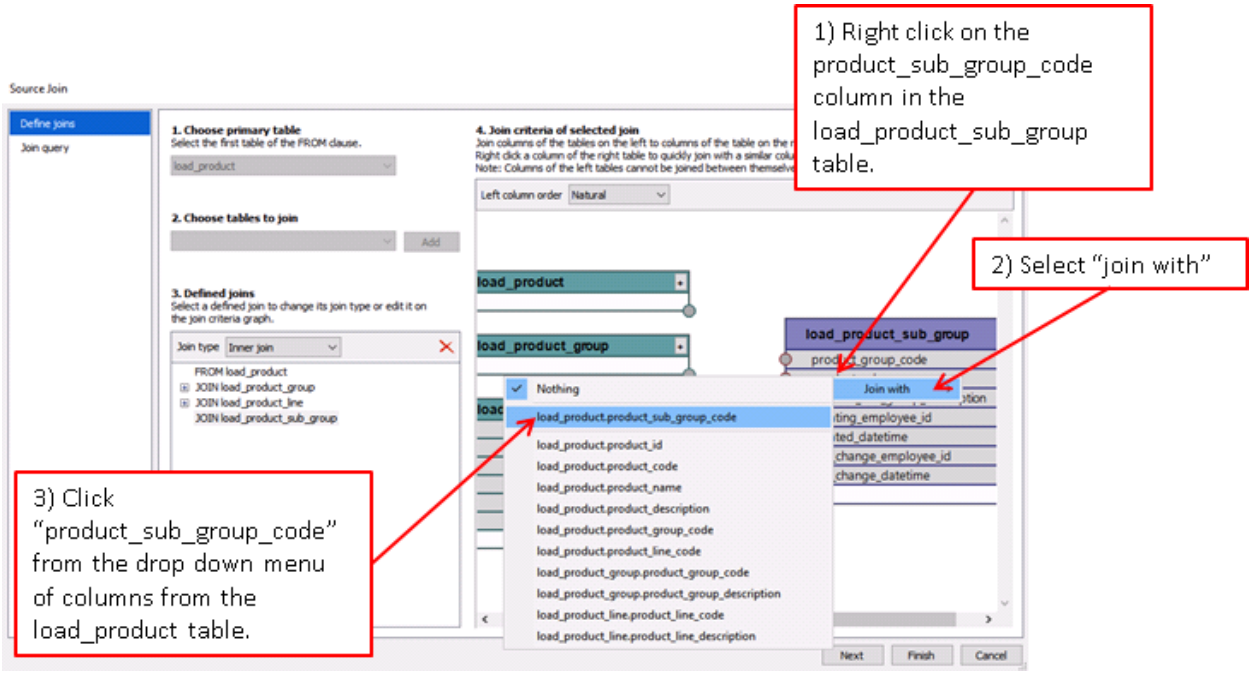
Define the join between load_product and load_product_line.



The first two joins have been defined. Begin defining the final join between load_product and load_product_sub_group.



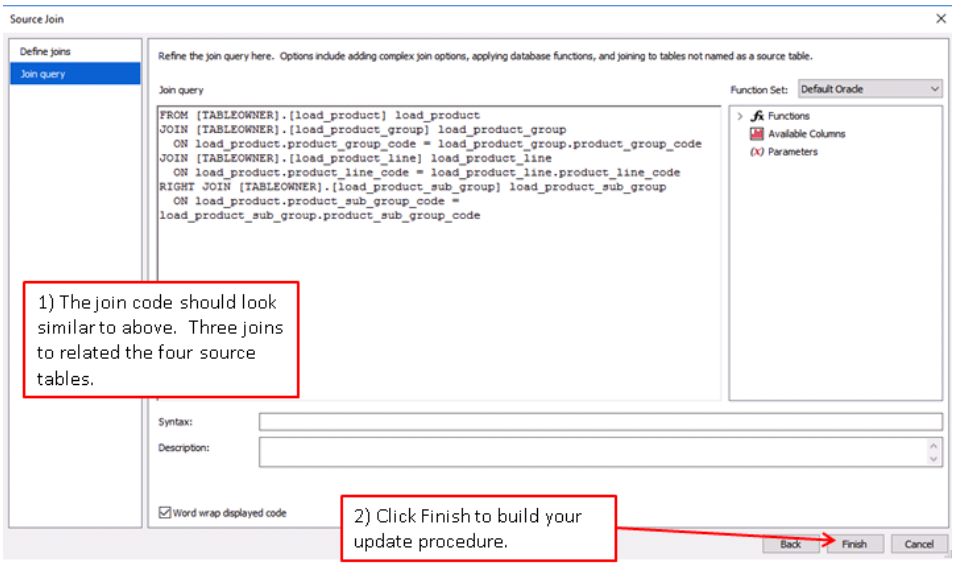
Define the final join.



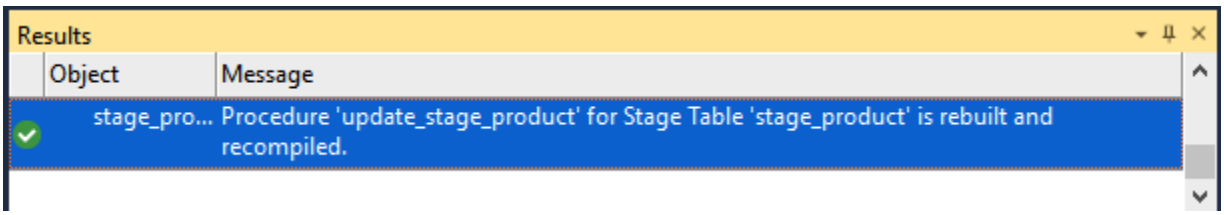
The last join you defined, between load_product and load_product_subgroup is an outer join, rather than the default inner join. Change this join to outer and verify the first two joins are defined as inner joins.



The join code is displayed. Should look similar to your code.

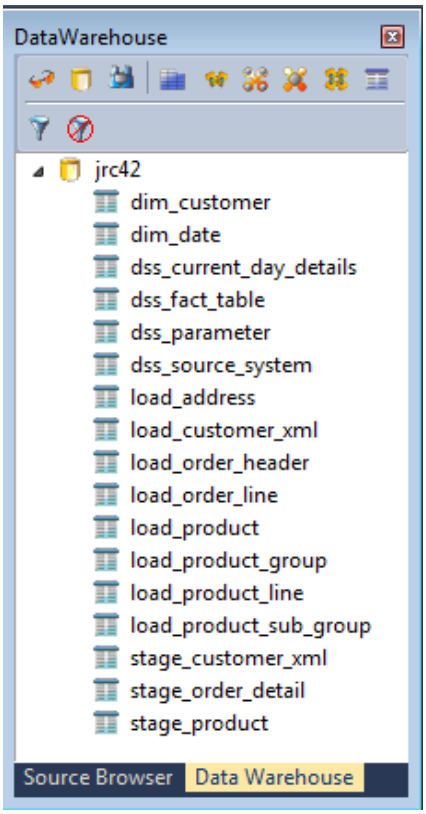


If your update procedure compiled correctly, you should see a success message in the results pane.



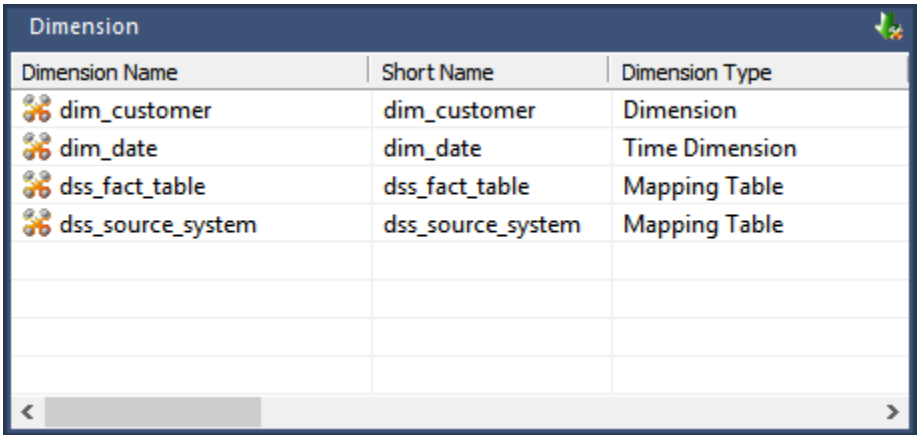
Right-click on **stage_product** in the left pane and select **Execute Update Procedure**.

- Right-click on **stage_product** in the left pane and choose **Display Data**.
- On the data warehouse tab click on the red glasses to refresh the connection.

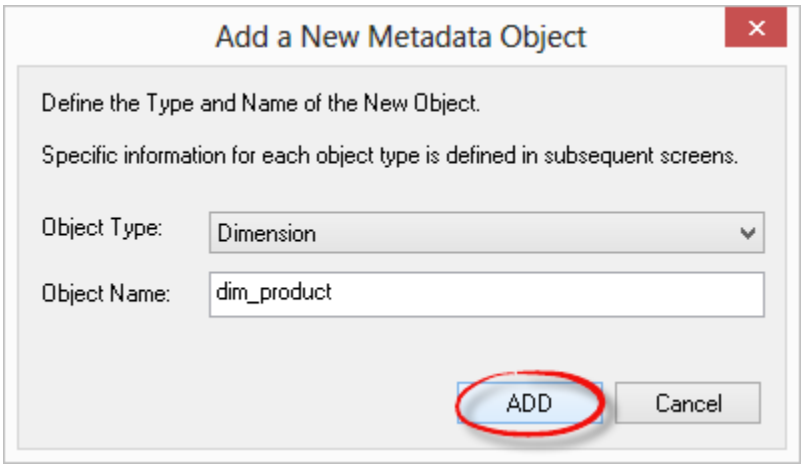


Set the context of the work pane by double-clicking on the **Dimension** group of the **All Objects** tree in the Object Browser window.

- The work pane will now show dimensions **dim_date** and **dim_customer**.



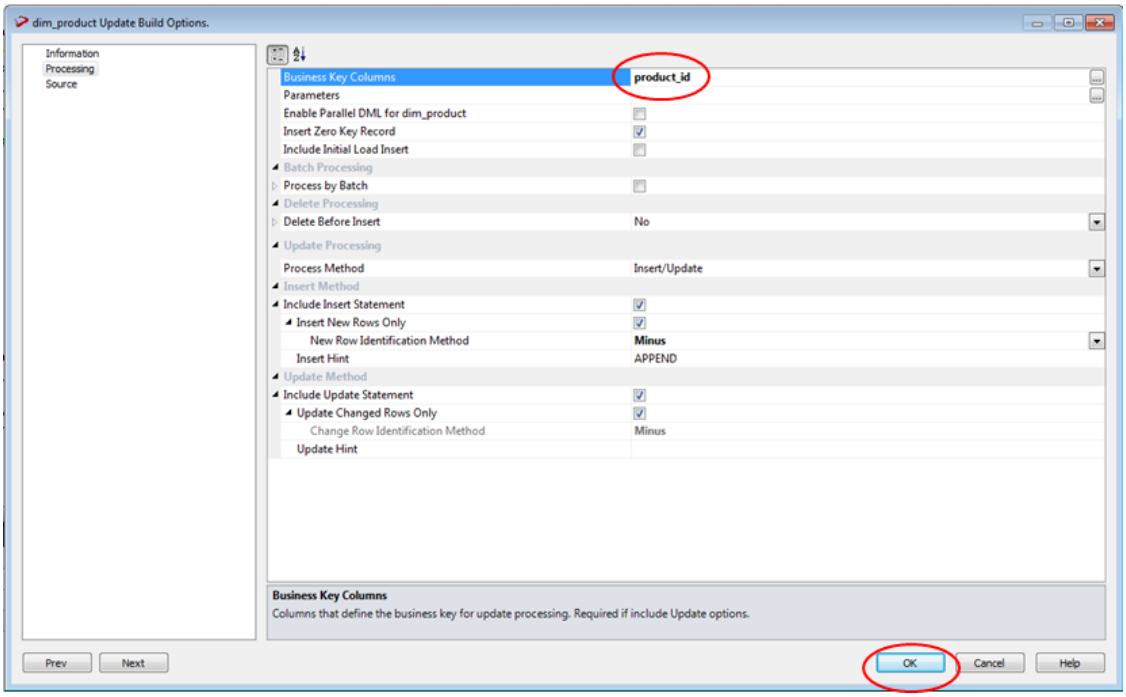
In the data warehouse browser window, drag the **stage_product** table onto the work pane to create **dim_product**. (If you don't see stage_product in the list, refresh by clicking the red sunglasses icon) Click **ADD**.



- In the dialog that follows, select a dimension type of **Normal**.
- On the **Properties** dialog, click the **Rebuild** button.
- Click **Create and Load**.

After Creating and Loading the dimension table, RED asks you to start defining the Update Procedure.

- Choose a **Procedure Build Type** of **Set** and click **OK.**
- On the Processing tab of the next dialog, click on the ellipsis button at the end of the **Business Key Columns** line.
- Select **product_id** as the **Business key** on the **Business Key Columns** dialog that pops-up next and click **OK.**



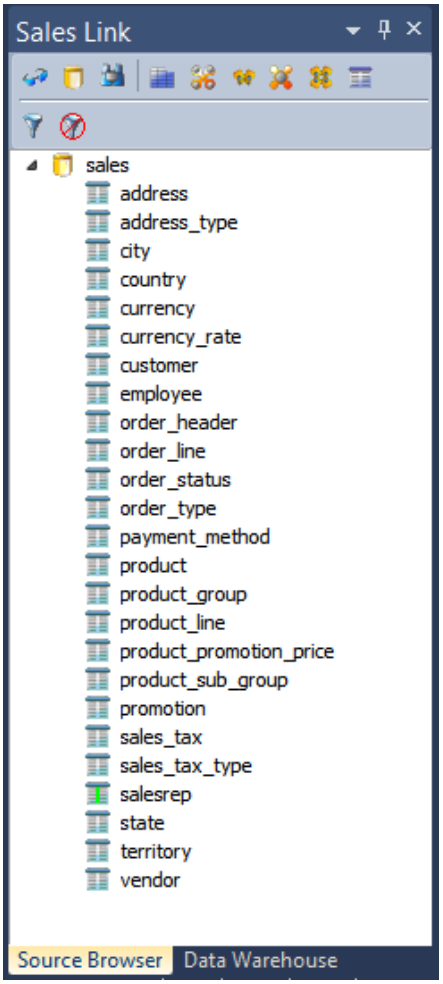
Finally, right-click on **dim_product** in the left pane and choose **Execute Update Procedure**.

- Right-click on **dim_product** in the left pane and choose **Display Data**.

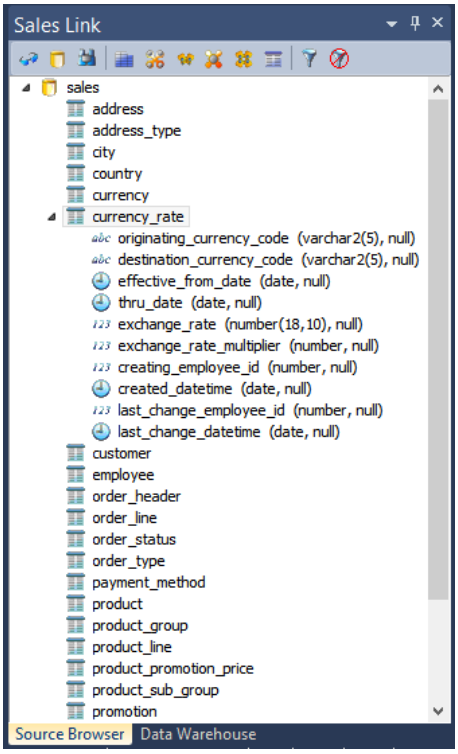
Create a Date Ranged Dimension Table

Often in a source system there can be dates that mark the effective dates of certain data. For example your HR system might only show the current position for an employee, or it might hold the dates for that particular position. If the latter is the case, you can map onto these fields using the dimension type **Ranged Dimension.**
Bring in the **currency_rate** table which holds the effective exchange rate for all periods of time for your data warehouse:

- Expand the Connection objects from the **All Objects** tree on the left pane and right-click on the **Sales** connection.
- Select **Browse connection.**

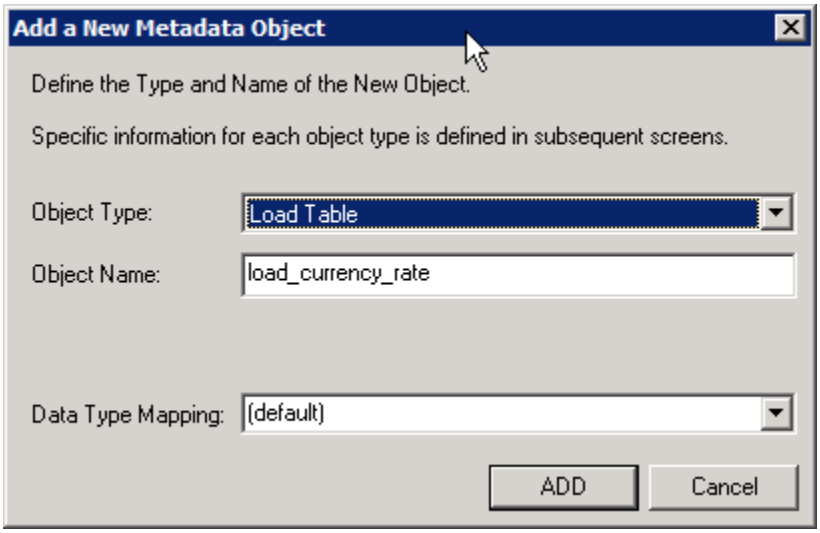


Explore the connection and double-click on the **currency_rate** table to see how it is structured.



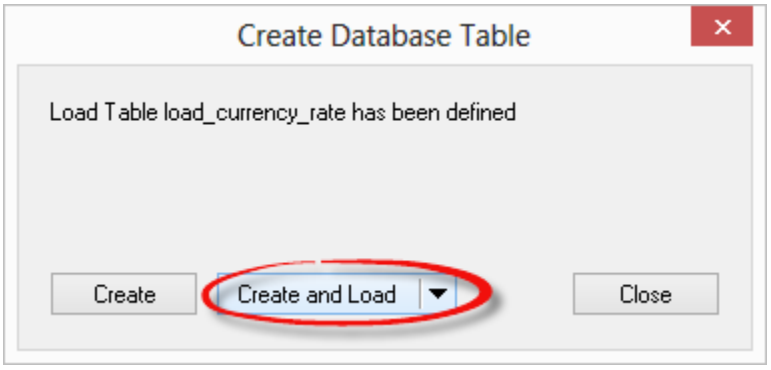
Notice the **effective_from_date** and **thru_date** columns – these are the columns you will use to range the dimension.

- Drag the table **currency_rate** into the middle, work pane to create **load_currency_rate** and click **ADD**.



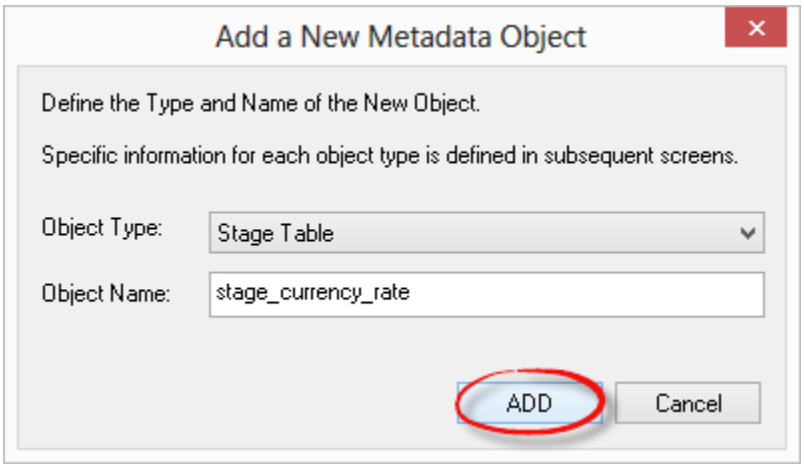
Accept all the defaults in the **Properties** dialog and just click **OK**.

- Click **Create and Load** to define and populate the load table in the data warehouse.



Next you need to create a stage table, **stage_currency_rate**.

- Double-click on the **Stage** folder on the left pane to set the work pane to the context to work on stage tables.
- In the connection browser on the right, click the **Data Warehouse** tab at the bottom and click on the red glasses to refresh the connection.
- Drag the new **load_currency_rate** table into the middle work pane to create a new stage table called **stage_currency_rate** and click **ADD**.



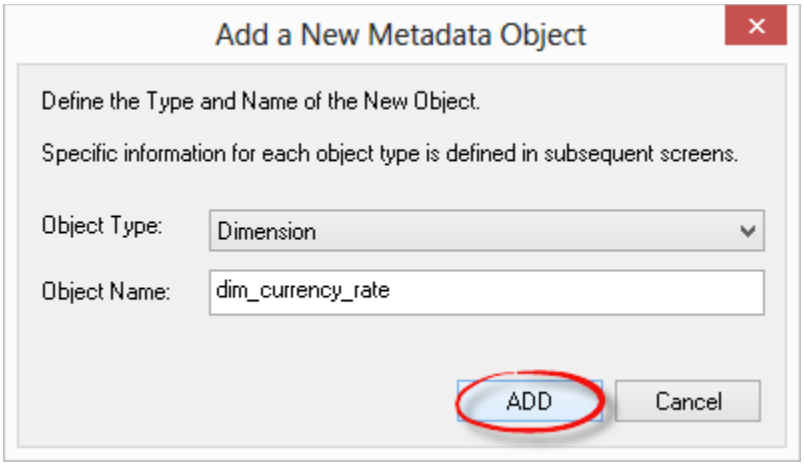
On the **Properties** dialog, click on the **Rebuild** button.

- Click **Create and Load** to define and populate the stage table in the data warehouse.
- Select **Set** as the **Stage Procedure Type**.
- Click **OK** on the **Parameters** dialog.
- Make sure your stage_currency_rate table loaded by displaying the data.

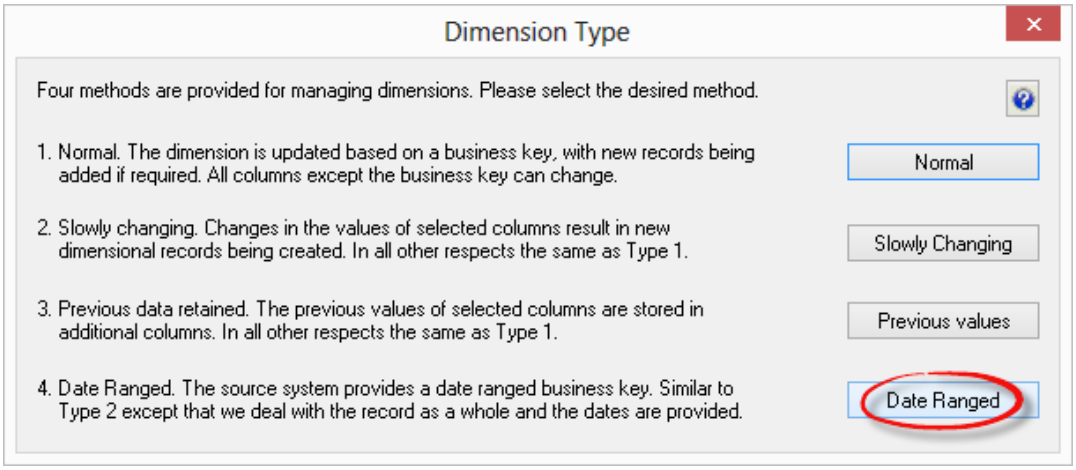
Finally, create a dimension table, **dim_currency_rate**.

- Double-click on the **Dimension** folder in the object explorer on the left to set the work pane into the correct context to work on dimension tables.

- In the connection browser on the right, click on the red glasses to refresh the data warehouse connection.
- Drag the new **stage_currency_rate** table into the middle work pane to create a new dimension table called **dim_currency_rate** and click **ADD**.

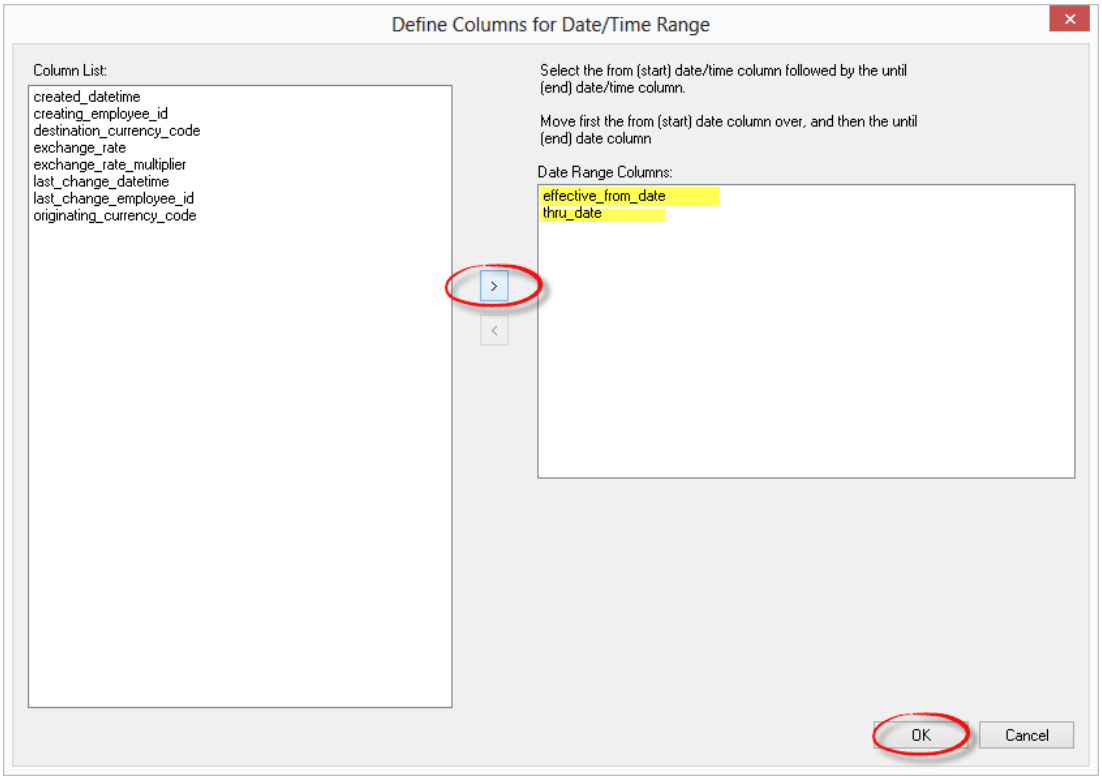


Select **Date Ranged** for the **Dimension Type**.

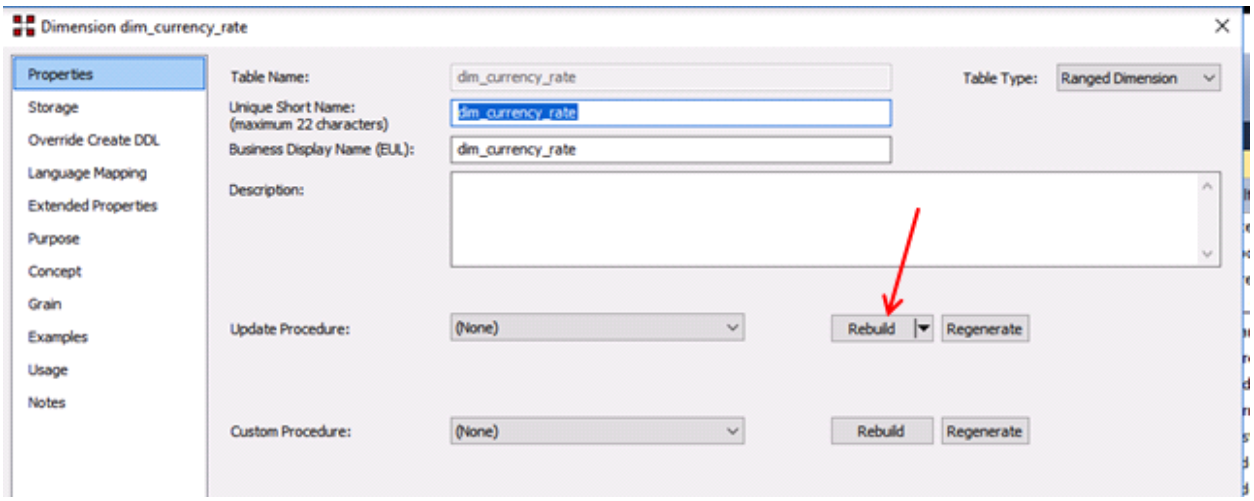


As you chose a date ranged dimension, RED prompts for you to define the start and end dates given in the source data.

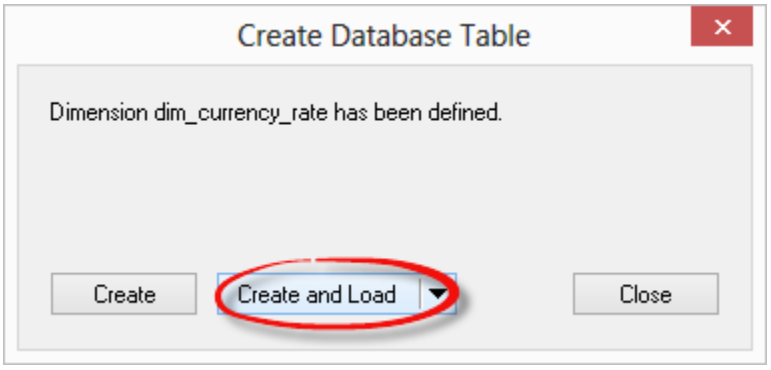
- Select the **effective_from_date** and **thru_date** in that order to add them to the **Date Range Columns** section.
- Click **OK**.



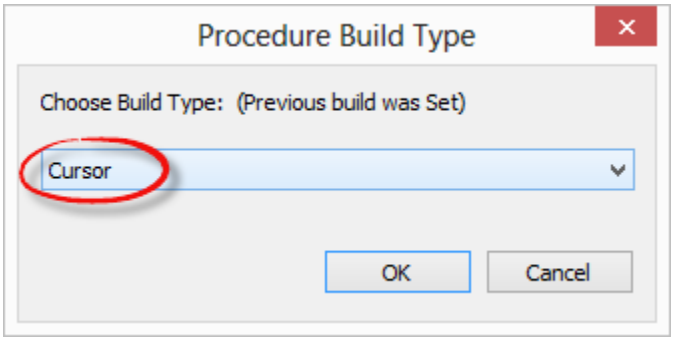
Build the dimension update procedure by clicking on the **Rebuild** button next to the **Update Procedure** field in the Properties dialog.



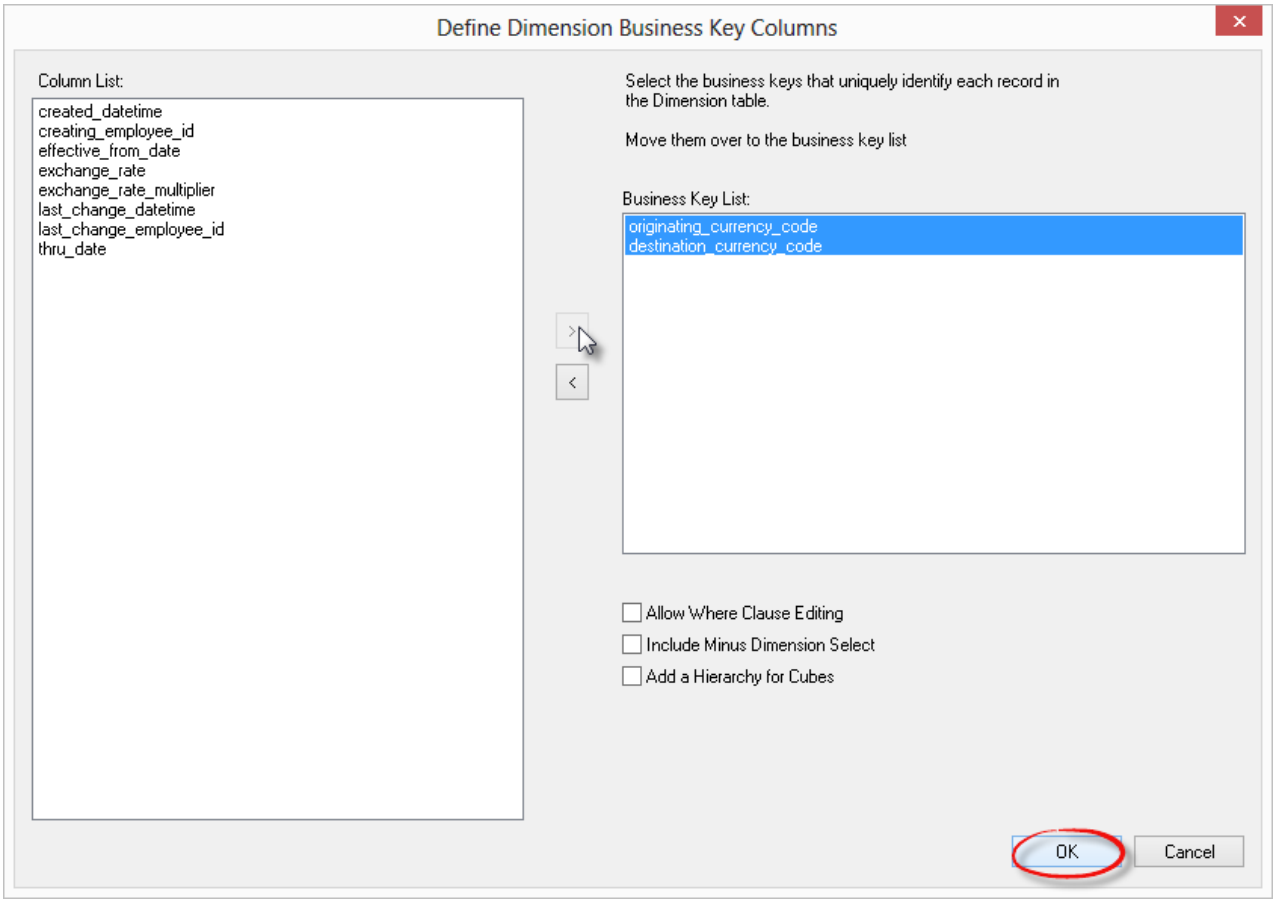
Click **Create and Load** to finish the dimension build to have RED build the table in the database and prompt for how to update the dimension.



Choose **Cursor** as the update type and click **OK**.



In the **Business Key List** add the **originating_currency_code** and **destination_currency_code** as being the non-date fields that uniquely tell you the currency variant you are interested in. Click **OK**.



Confirm the start date and end date for the **Date Range Columns** by clicking on **OK** again on the next dialog.
In the **Results** pane along the bottom, notice that the table has been created and the data loaded as per the update procedure.

Create a Dimension View

Often in a business environment, there will be occasions where you need to create subsets of a dimension or have clones of a dimension but with subtle name changes. These different perspectives or Views can result in massive admin overheads and potentially erroneous information. Given the criticality of the 'single source of truth' in a data warehouse being driven primarily through the dimensions, it is essential to deny the need for any such variations.
In the first scenario, you will look at the default Date Dimension, dim_date. Developed from years of data warehouse experience intertwined with best practice, the date dimension pre-delivers standard date categorization as well as powerful relative time analysis (month on month, same month last year, year to date etc.).

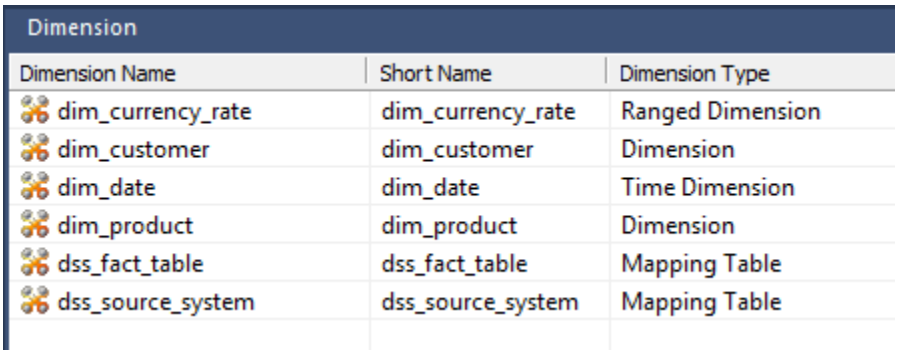
NOTE: dim_date is pre-delivered but in a customizable way. You can modify the update procedure (update_dim_date) to change how many days before and after today's date to populate by changing the line
v_no_of_days := 5000; -- number of days into future to generate records
as well as inserting your own code to deal with specific dates for your business.

TIP: It is essential that you update the date dimension with your requirements. The table dim_date is rarely refreshed but typically updated on a daily basis with the custom procedure daily_date_roll to ensure that the relative time periods are in line with today's date.

- The 'problem' most businesses experience with dates is the need to have many different dates within an organization. This results in many different date dimensions that are all intrinsically the same. Examples of this are order date, sale date, pay date, ship date, invoice date, dispatch date to name but a few.
  With RED, you re-use the definition of date and simply create a View over the top of the standard, single version of the truth, dim_date. *This is known as "**role playing**".*

NOTE: You could easily use dim_date repeatedly in your data warehouse but this would result in ambiguity for the end user plus a great admin overhead on the reporting layer to interpret what is required. With RED, the reporting later and business community will see multiple date dimensions cloned from the original dim_date.

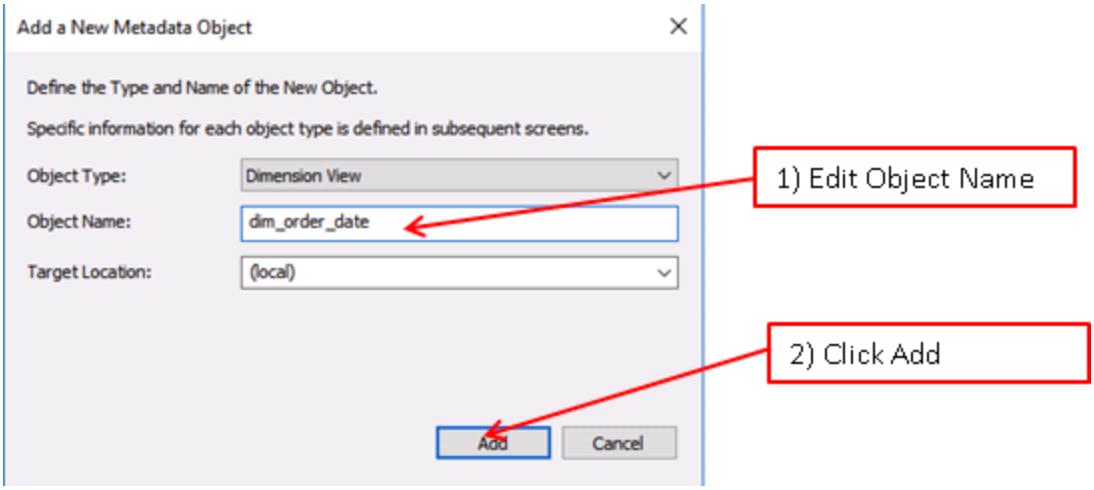
- From the object explorer, double-click on the **Dimension** group to set the context of the work pane to Dimension and list all the dimensions you have so far.


Dimension

Dimension Name	Short Name	Dimension Type
dim_currency_rate	dim_currency_rate	Ranged Dimension
dim_customer	dim_customer	Dimension
dim_date	dim_date	Time Dimension
dim_product	dim_product	Dimension
dss_fact_table	dss_fact_table	Mapping Table
dss_source_system	dss_source_system	Mapping Table

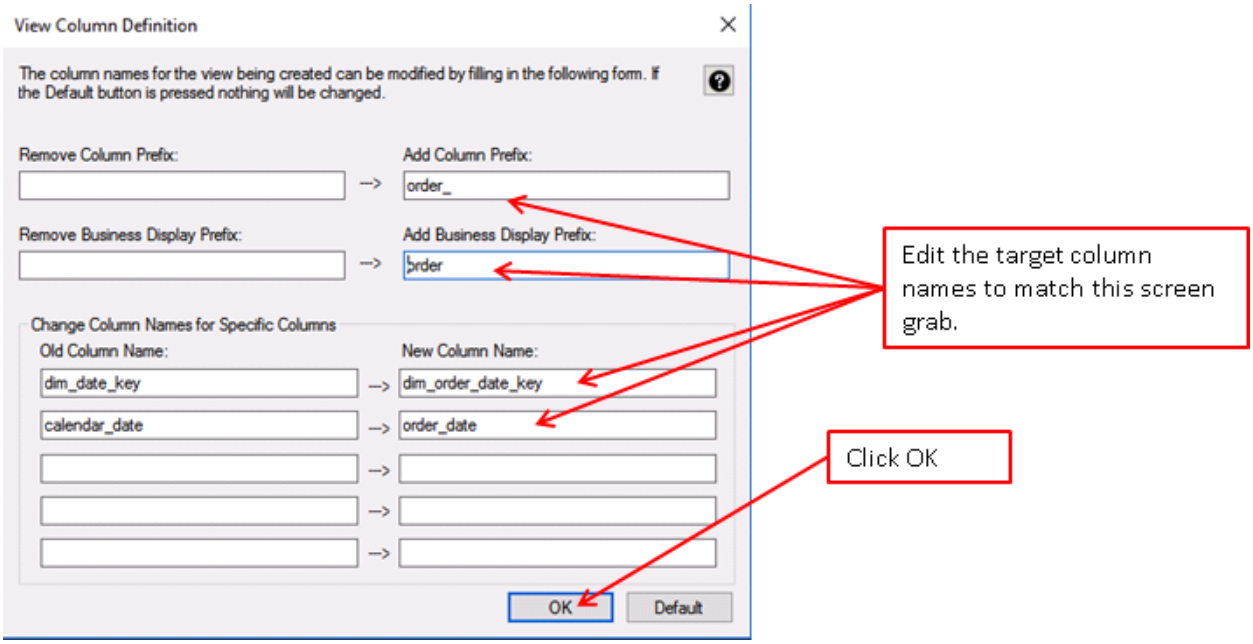
From the left pane, now double click Dimension, to set context to Dimension.

From the data warehouse connection browser, click on the red glasses to refresh the connection and drag the **dim_date** table to the middle work pane. Then edit the object name.

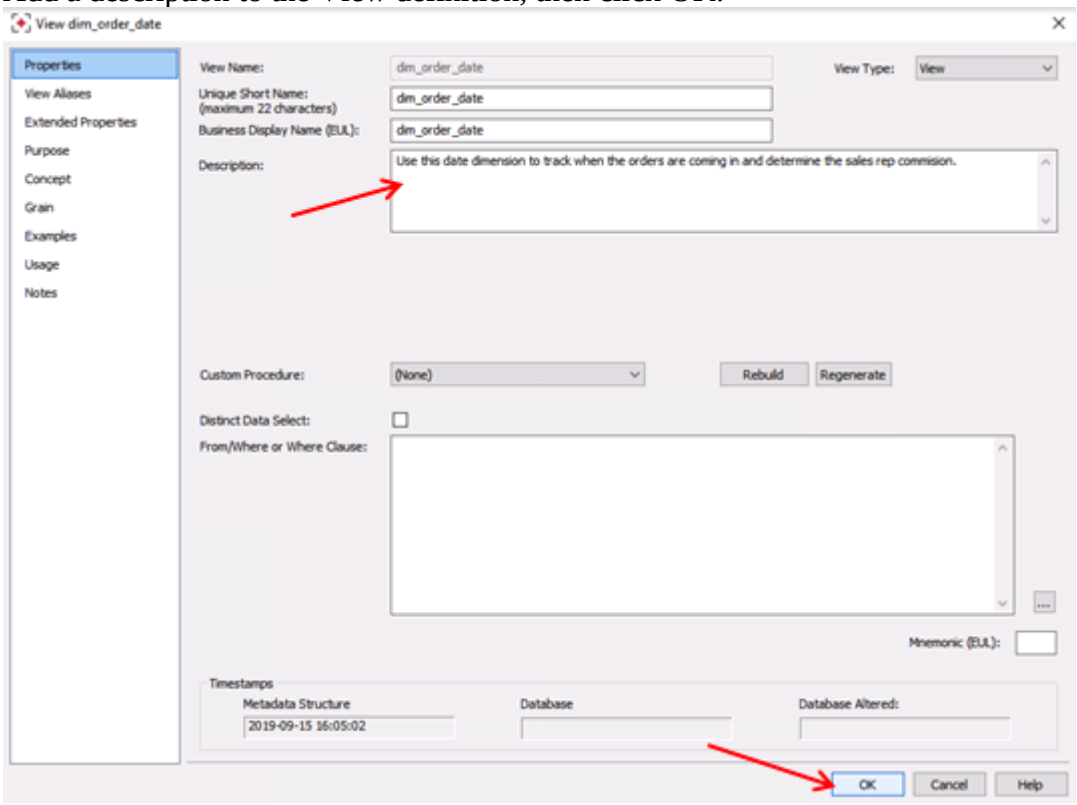


The **View Column Definition** dialog allows you to map how the fields in dim_date should be aliased to reflect their new purpose for order dates.
Similarly, if the column's Business Display Names had previously been prefixed by some text, you can remove it in the Remove **Business Display Prefix** field e.g. 'My Company' and add some new text to help set the business perspective of this date in the **Add Business Display Prefix.**
RED defaults your prefix to **order** as this was the text you added to the dimension view name on the previous screen.

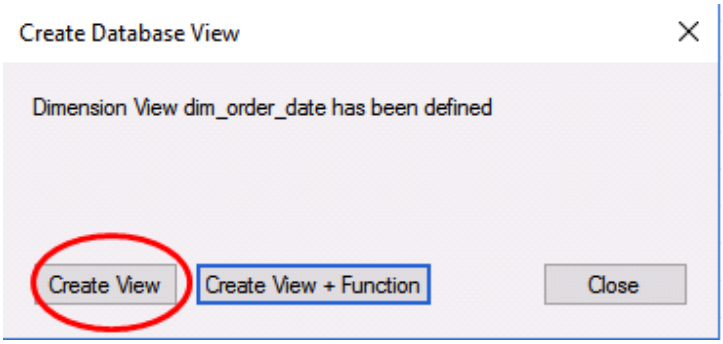
- Verify the Add Business Display Prefix is Order (note the space on the end which is included) to ensure that the business display names are updated more appropriately.
- The last section allows for fields / columns that can't be updated by the rule above and are specific cases. There is only one column with a specific name calendar_date.
- Change the New Columns Name for calendar_date to be order_date so your new view is completely clear about the purpose of the dates.
- Click OK to complete the definition of your new Order Date variant on dim_date.



Add a description to the View definition, then click OK.

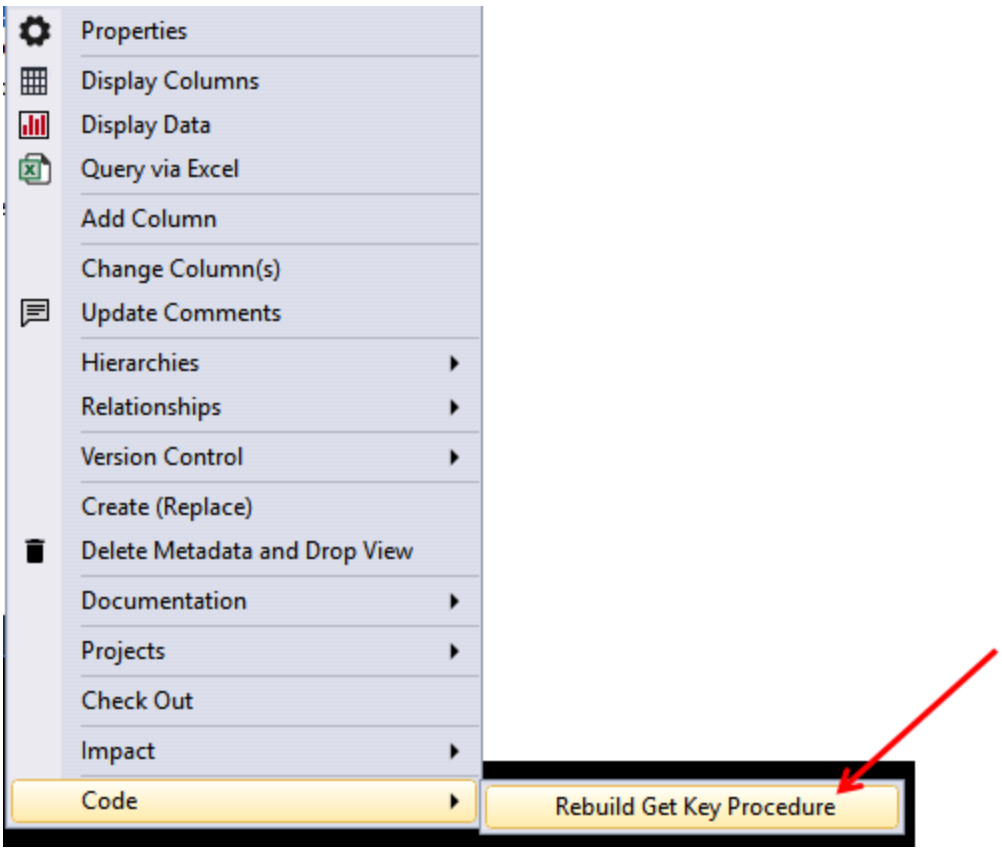


Click **Create** to complete the definition and synchronize the data warehouse.



In the left hand "Builder" Pane, open "Dimension View". You will see your new dim_order_date.

Right Click on the dim_order_date, select "code", then "Rebuild Get Key Procedure".



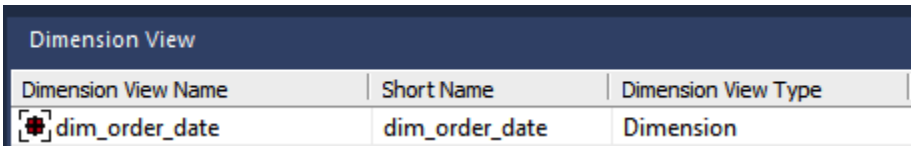
Click OK on the "Build Options" screen.

To see the results, right-click on the **dim_order_date** and select "display data".

Notice the name of the field is specific to order dates even though this is actually querying the dim_date table.
In this way you can create all the relevant date dimensions you require across the organization and are safe in the knowledge that each reflects the correct date structure and is completely clear as to the purpose for which it is designed.

Next create a further variant of dim_date but this time for the ship date of the product in the order:

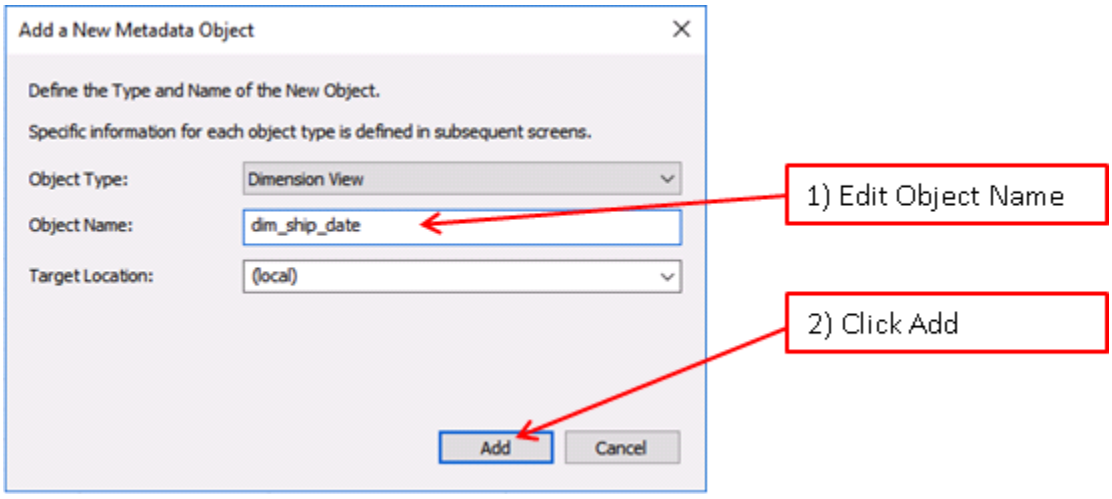
From the object explorer, double-click on the **Dimension View** group to set the context of the work pane to Dimension View and list all the Views you have so far, which is only dim_order_date



From the data warehouse connection browser, click on the red glasses to refresh the connection.

Drag the **dim_date** table to the middle work pane.

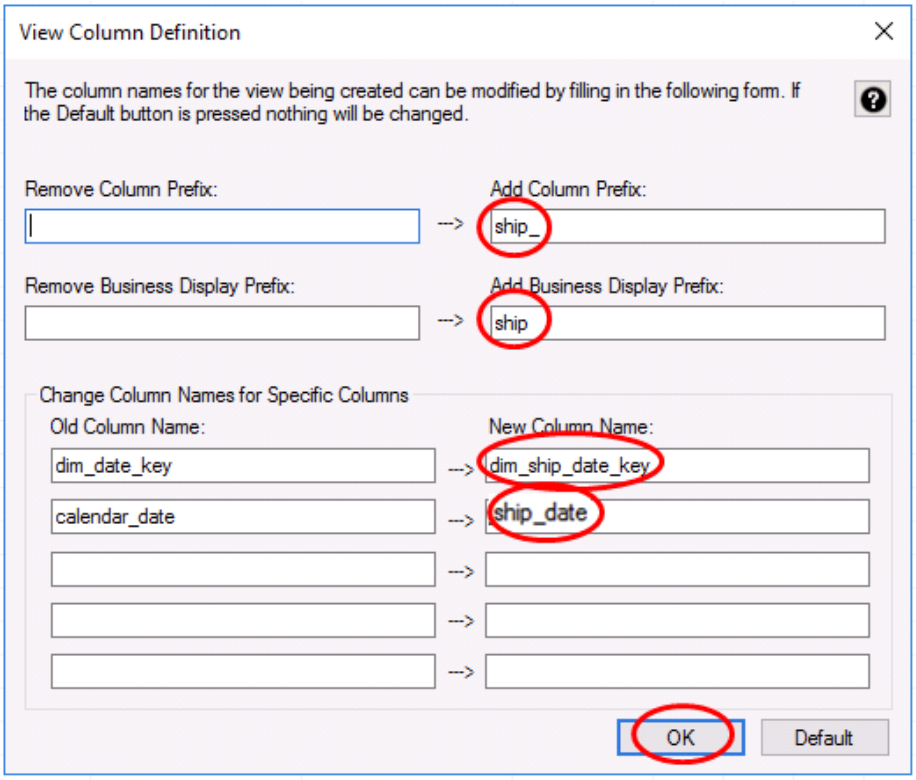
- Once again, RED understands that you must be looking to create a dimension view across this table and so sets the **Object Type** to **Dimension View**.
- The date variant you are looking to create is the product shipping date to ensure you can track when your products shipped to the customer and track turnaround times, so name this dimension view **dim_ship_date** and click **ADD**.



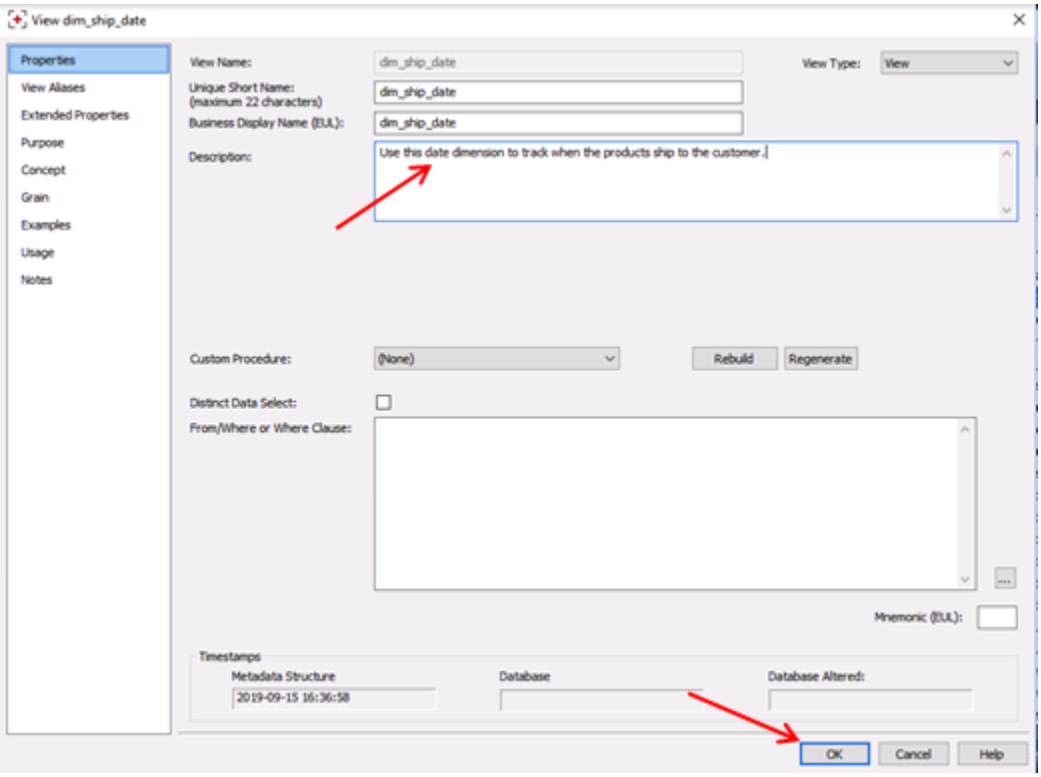
The **View Column Definition** form allows you to map how the fields in dim_date should be aliased to reflect their new purpose for the shipped dates.

RED defaults your prefix to **ship_** as this was the text you added to the dimension view name on the previous screen.

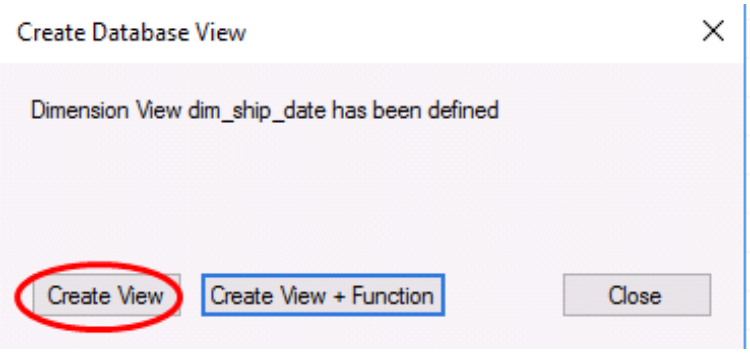
- Validate the **Add Business Display Prefix** is **Ship** (note the space on the end which is included) to ensure that the business display names are updated more appropriately.
- The last section allows for fields / columns that can't be updated by the rule above and are specific cases. There is only one column with a specific name **calendar_date**.
- Change the **New Columns Name** from **calendar_date** to **ship_date** so your new view will be completely clear about the purpose of the dates.
- Click **OK** to complete the definition of your new Ship Date variant on dim_date.



Add in a suitable description such as '**Use this date dimension to track when the products ship to the customer**.' and click **OK**.



Click **Create** to complete the definition and synchronize the data warehouse.



1. Right click on dim_ship_date, select code, then “Rebuild Get Key Procedure”.
2. Click OK on the “Build Options” screen.

Finally have a look at the reporting layer and business community will see:

- From the object explorer’s View group, right-click on dim_ship_date and display data.
- Notice the name of the field is specific to ship dates even though this is actually querying the dim_date table.

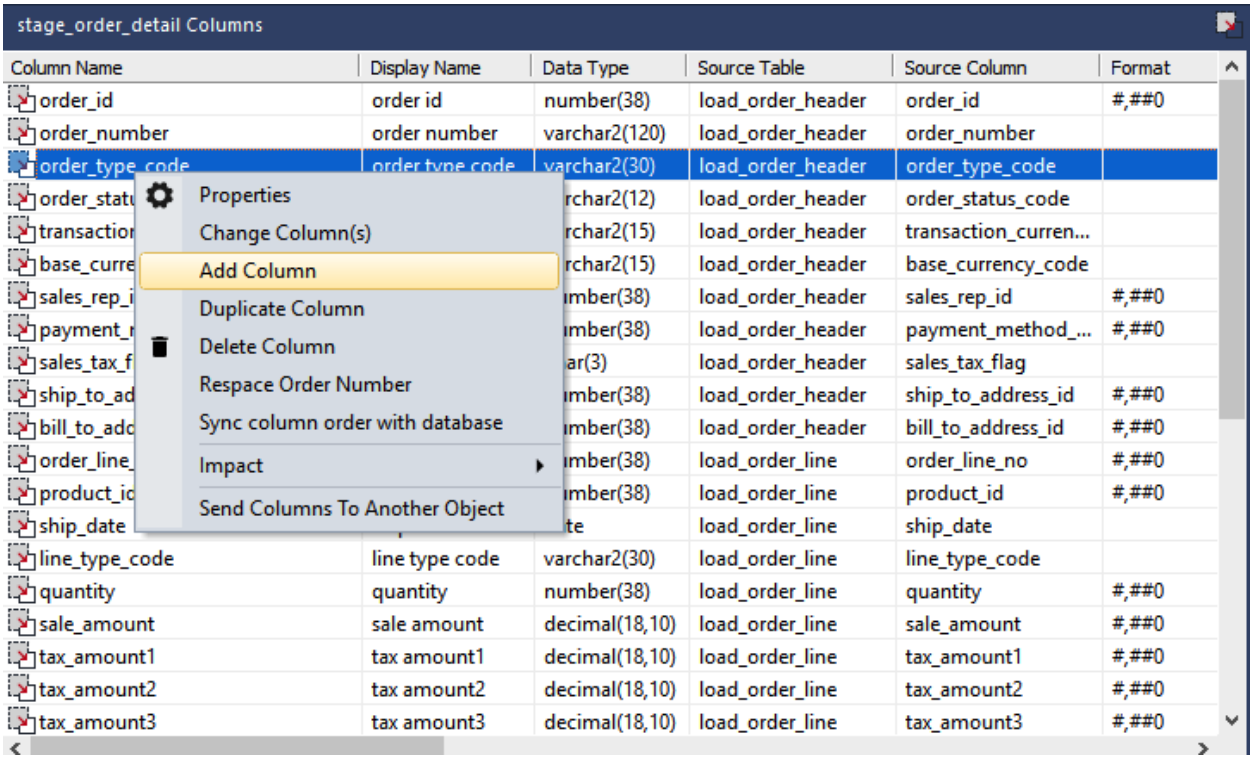
The report writer will see dim_date, dim_order_date, and dim_ship_date and have no need for complicated logic to re-use the same table in multiple joins to get the information required in their reports.

Create a Fact Table for Sales

In previous sections you created and manipulated the source data to get meaningful information to present to the business. Now all this information needs to be brought together into a single source of valid, approved, structured business information.
The first step when doing this is to bring all the data into a final stage table. Here all the required data is referenced, cleaned up, labeled in a business context, derived using business rules, categorized using the business defined structures / dimensions and presented in a simple, clear manner.
Remember you have already brought in the order header and order line tables and combined them in a new stage table in the data warehouse to alleviate the resource impact of a more complex extract query on the source.
This stage_order_detail table already has the transactional detail that you need but if you were to use this as the source for a Fact table, you should further enhance the business context by referencing your dimensions, relabeling them in a business context and adding value by incorporating business rules to derive more business information.

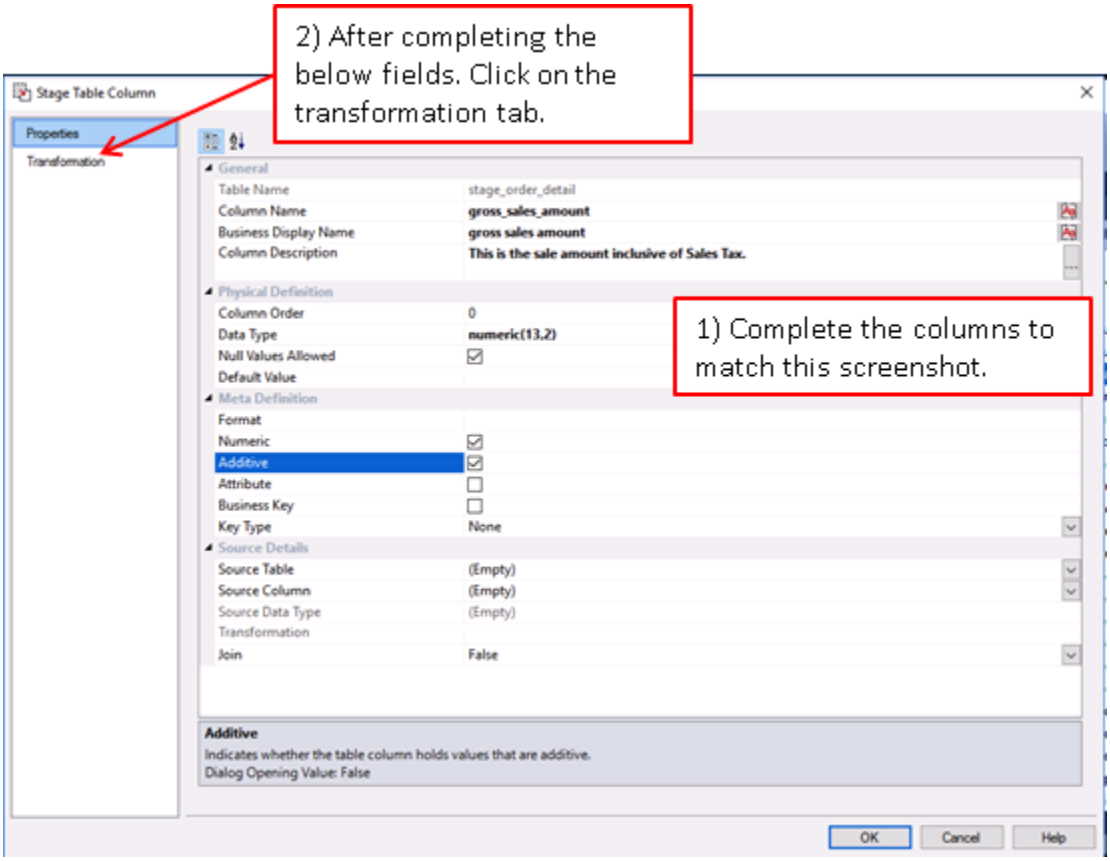
In the object explorer, click on the Stage Table **stage_order_detail** to set the work pane context to be at column level on the table.

- The columns are already fairly well defined and their business definitions were carried through all the way from the source, however the Sales Team are regularly having to calculate the Sale Amount inclusive of Sales Tax, so you should add this in. Right-click anywhere in the work pane and choose **Add Column**.



Notice your right-click options are all based around Columns given that your work pane context is currently set to columns.

- The column Properties dialog holds all the options that you need for a column whether sourced directly from a table, or transformed via some sort of calculation. You saw each of these Properties described in the Categorize section, so create a column **gross_sales_amount** as follows:



NOTE: As soon as you set a transformation, the **Source Table** and **Source Column** for each column are no longer used to generate code. They are still used for impact analysis and should be set to be indicative, usually to the most important source table/column being used in the transformation. If a column doesn't have a most important source column, then it is best practice to leave the **Source Table** blank and to set the **Source Column** to the column's name.

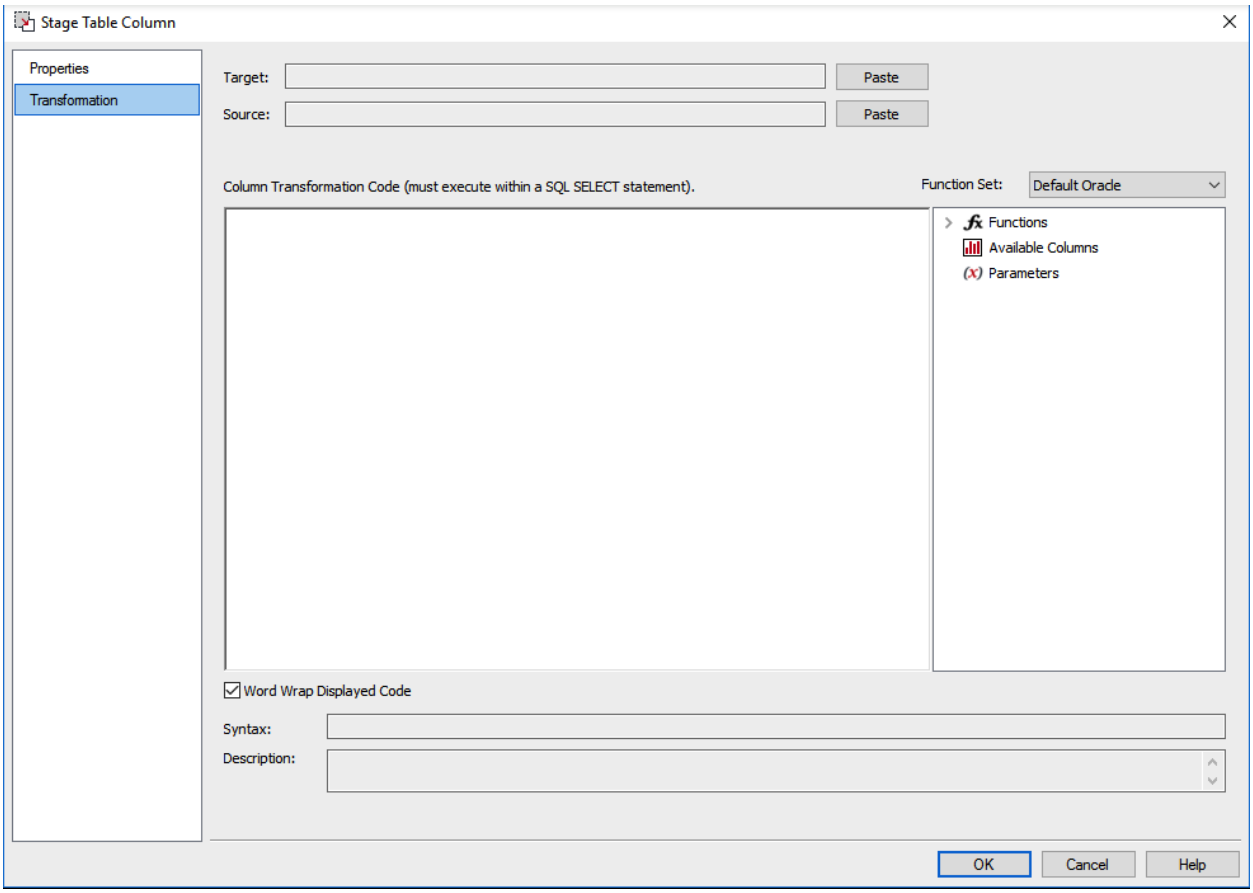
TIP: A quicker way to create this column would have been to right-click on the sale_amount column and choose 'Duplicate Column' to clone the column.

Click the **Transformation tab** (found in the left hand side of the screen).

The **Target** and **Source** fields are not populated because you have not chosen columns. These are critical in load tables as they will affect how the transformation is specified – **During Load** transformations would require the appropriate syntax from the Source column whilst **After Load**

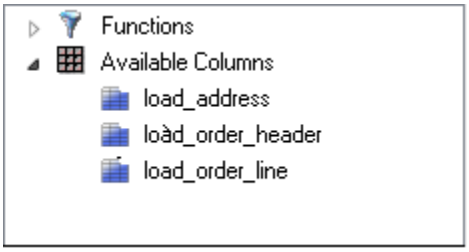
transformations would require the Target columns definition to be used. As soon as you start to define the transformation, **Target** will show **gross_sale_amount**, but as you will have multiple columns in the calculation, Source will remain blank.
The main pane is for defining the transformation itself. At this stage of your development, you are completely on the data warehouse (and not touching the source systems), the required syntax is that for your data warehouse.

NOTE: The statement you create must be capable of running in a SQL Select statement.

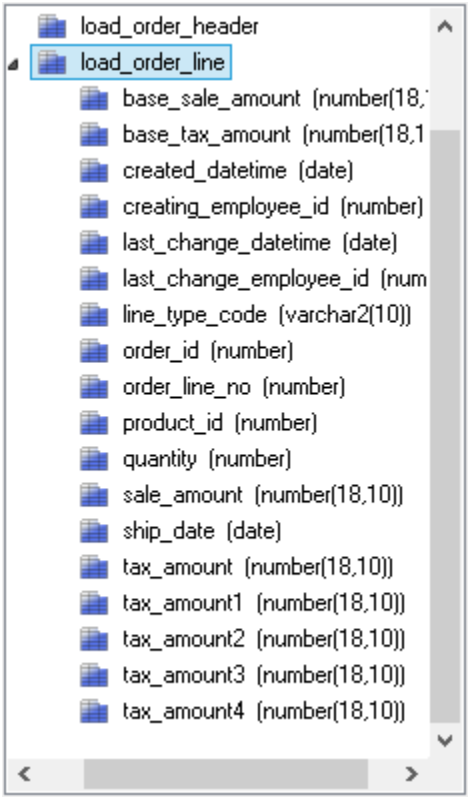


The right-hand pane helps ensure that you can create an appropriate statement with Functions mapped to the data warehouse equivalents and Columns listed from every table specified as the source tables for this stage table.

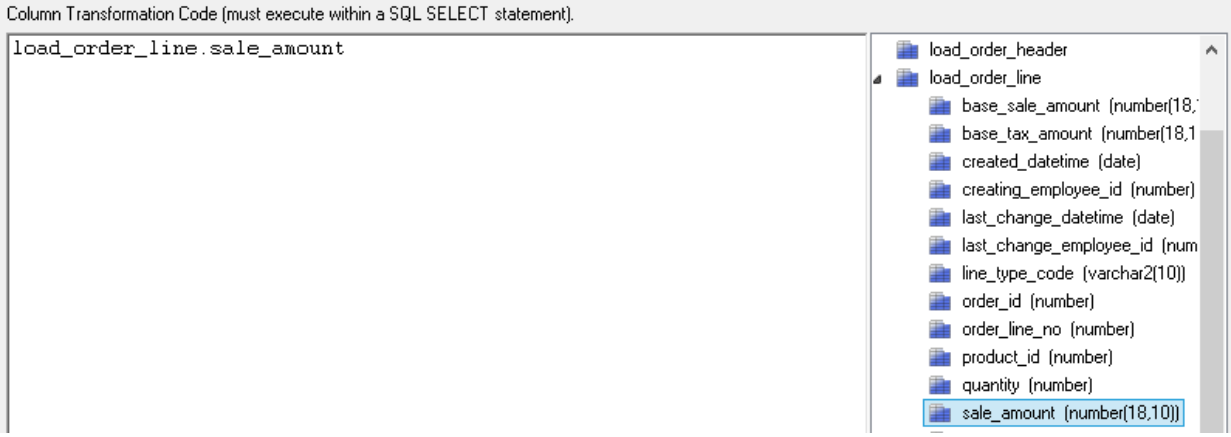
- Your calculation is really simple, so double-click on **Available Columns** to see the list of tables that this stage table uses as the source(s).



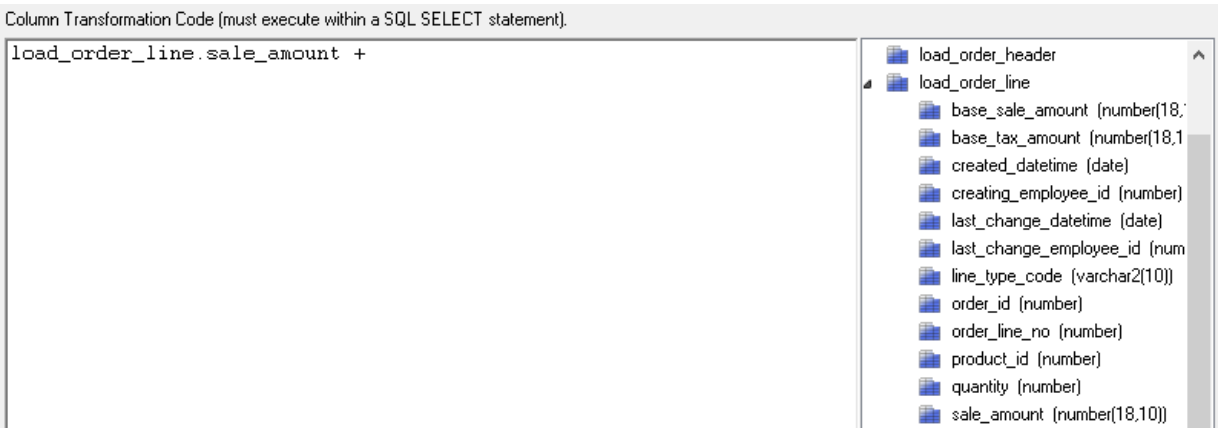
Double-click on **load_order_line** to see all available columns from that source table.



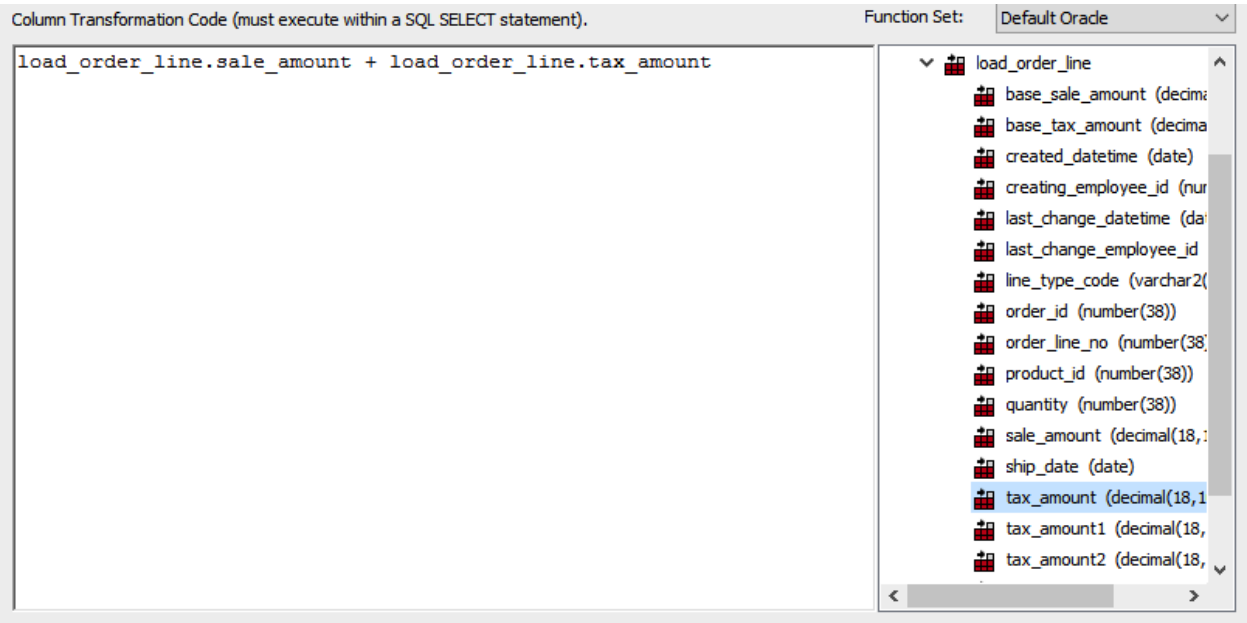
Double-click **sale_amount** from the list to add it to the left-hand transformation pane where the cursor is (currently at the start).



Click at the end of **load_order_line.sale_amount** then add a space, a plus '+', then another space to build up the statement.



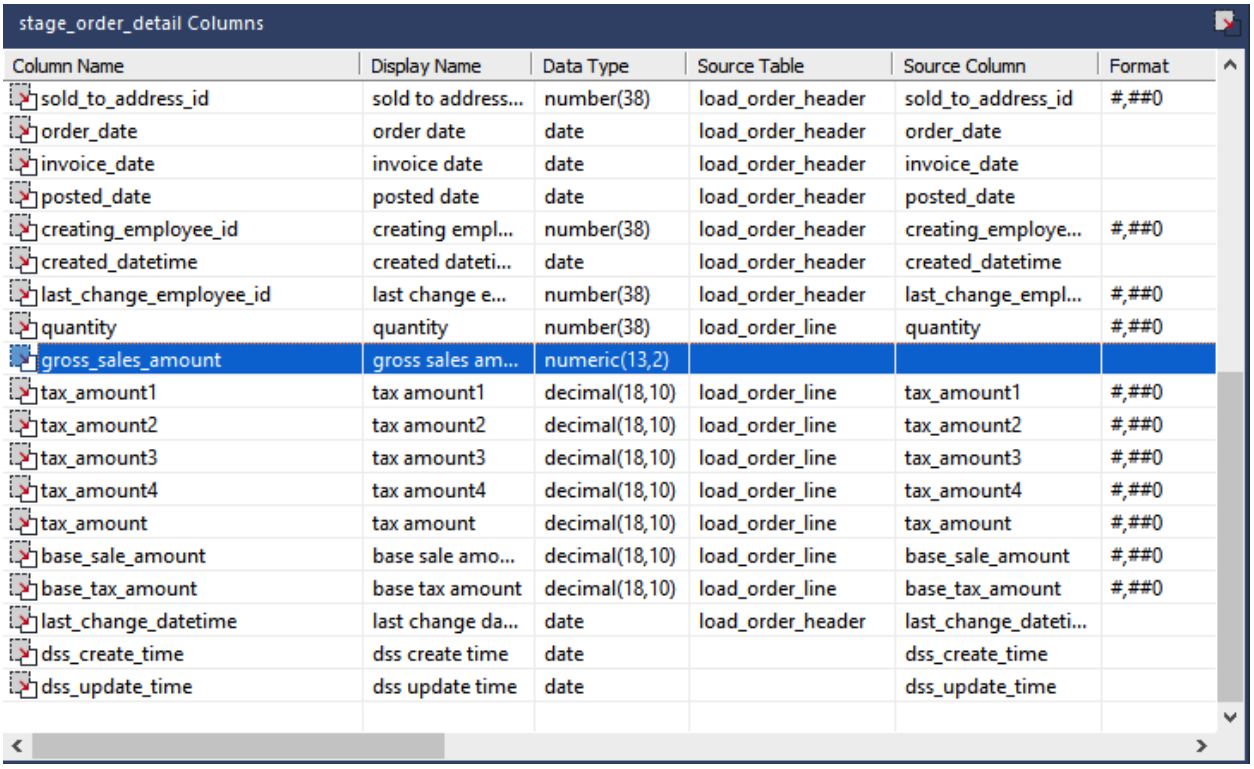
Double-click **tax_amount** from the list to add it to the left-hand transformation pane where the cursor is (currently at the end).



Click **OK** to save your definition.

In the work pane, notice your new column is tagged on to the end of the table. It would make more sense displayed after the **quantity** column.

- Click on your new **gross_sale_amount** column and drag it to above the **sale_amount** column.

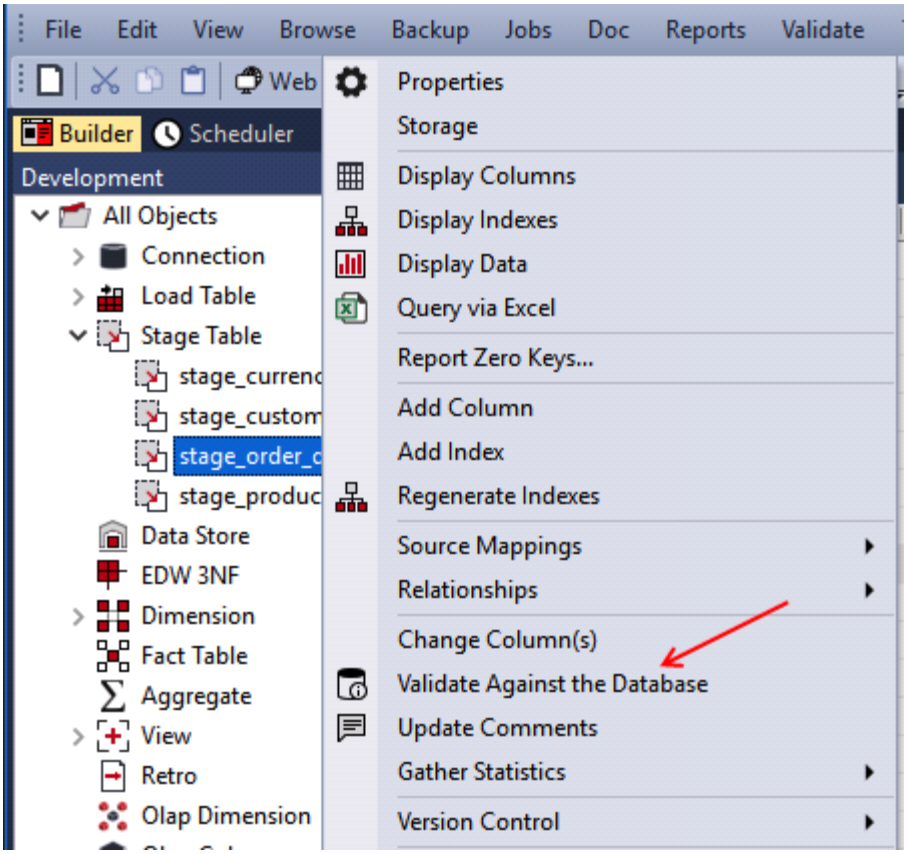


stage_order_detail Columns

Column Name	Display Name	Data Type	Source Table	Source Column	Format
sold_to_address_id	sold to address...	number(38)	load_order_header	sold_to_address_id	#,##0
order_date	order date	date	load_order_header	order_date	
invoice_date	invoice date	date	load_order_header	invoice_date	
posted_date	posted date	date	load_order_header	posted_date	
creating_employee_id	creating empl...	number(38)	load_order_header	creating_employe...	#,##0
created_datetime	created dateti...	date	load_order_header	created_datetime	
last_change_employee_id	last change e...	number(38)	load_order_header	last_change_empl...	#,##0
quantity	quantity	number(38)	load_order_line	quantity	#,##0
gross_sales_amount	gross sales am...	numeric(13,2)			
tax_amount1	tax amount1	decimal(18,10)	load_order_line	tax_amount1	#,##0
tax_amount2	tax amount2	decimal(18,10)	load_order_line	tax_amount2	#,##0
tax_amount3	tax amount3	decimal(18,10)	load_order_line	tax_amount3	#,##0
tax_amount4	tax amount4	decimal(18,10)	load_order_line	tax_amount4	#,##0
tax_amount	tax amount	decimal(18,10)	load_order_line	tax_amount	#,##0
base_sale_amount	base sale amo...	decimal(18,10)	load_order_line	base_sale_amount	#,##0
base_tax_amount	base tax amount	decimal(18,10)	load_order_line	base_tax_amount	#,##0
last_change_datetime	last change da...	date	load_order_header	last_change_dateti...	
dss_create_time	dss create time	date		dss_create_time	
dss_update_time	dss update time	date		dss_update_time	

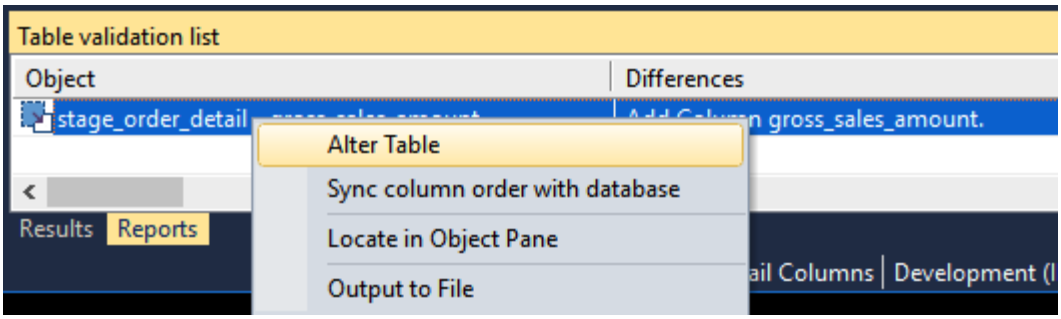
NOTE1: You are still in the metadata at the moment. Your table physically exists in the data warehouse without this field.
NOTE2: The order is purely visual at the moment, but the **Column Order** value in the column Properties has been updated to reflect this new position such that if the table was recreated in the data warehouse, the table would be physically created with the same column order.

Right-click on the **stage_order_detail** stage table and choose **Validate Against the Database**.



In the **Results** pane, notice the **Add Column gross_sales_amount** indicating that the metadata has a field that does not exist in the data warehouse.

- Right-click on this notice and choose **Alter Table**.



As you saw previously, the Alter statement gives you a suitable starting point but is completely customizable.

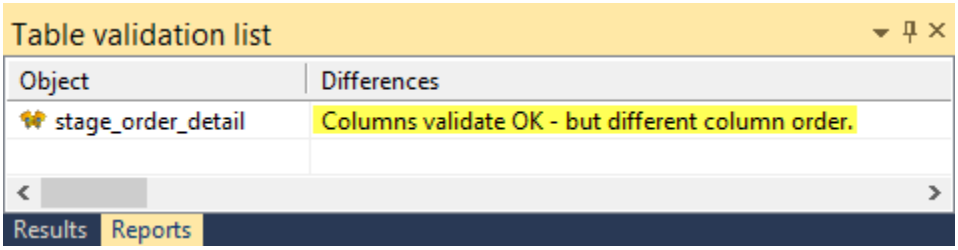
- This statement is fine for your requirements right now, so click on **Alter Table** to commit this SQL statement and adjust your physical table. Then **OK** on the next dialog.

NOTE: You specified the order of columns in your table, but an Alter statement will always add new columns to the end of an existing table. To force the same order, you need to recreate the table.

TIP: To get the column order you wish without losing the data (because of the recreate), copy the data into another (temporary) table, recreate the original (losing all the data), load the temporary table back into the newly recreated 'original' table and drop the temporary table.

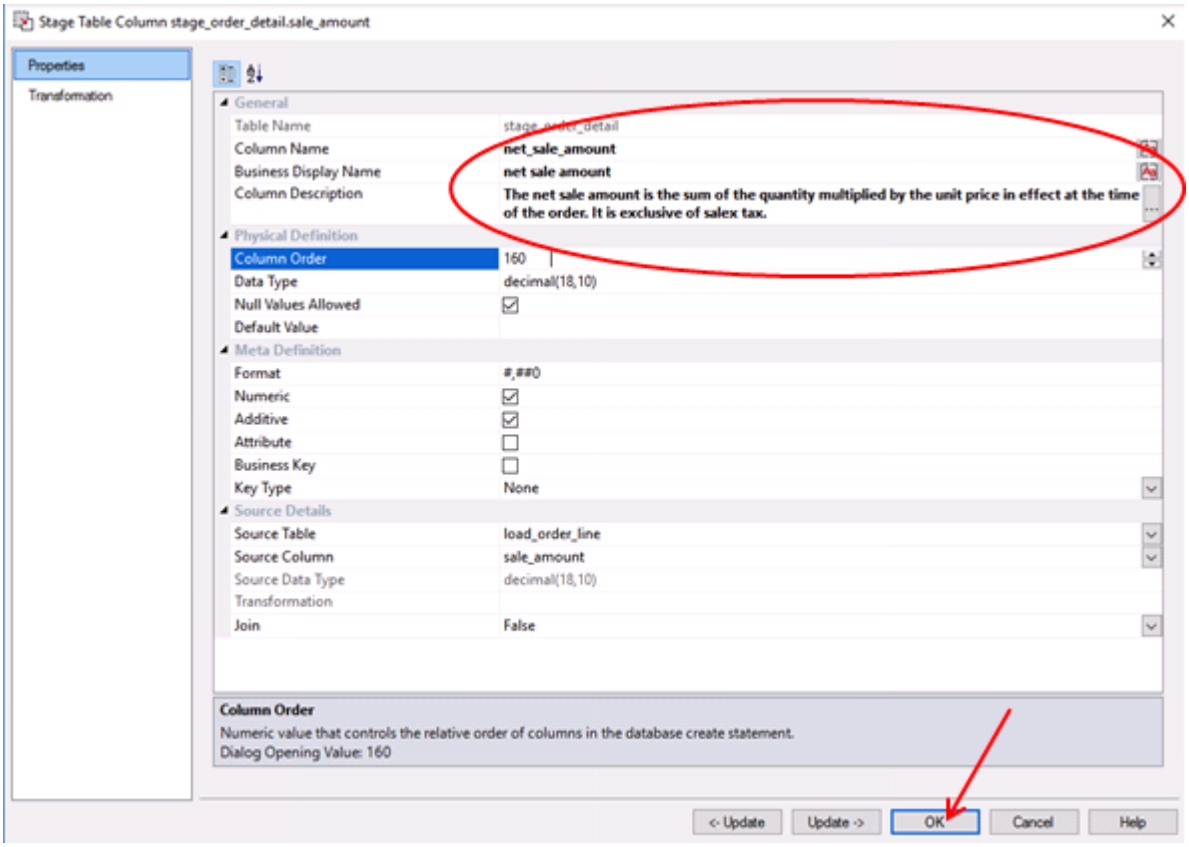
Right-click on the **stage_order_detail** stage table and again choose **Validate Against the Database**.

- Notice that this time the table **Validates OK** but with a caveat of ' – but with different column order'. This is perfectly okay and will not affect the data warehouse in any way.



Now that you have specified gross_sale_amount, you should be clearer about the use of sale amount.

- Double-click on the **sale_amount** column and rename it to **net_sale_amount** by changing the Column Name.
- Update the **Business Display Name** to **net sale amount.**
- Update the Business Definition to clarify that this is the value exclusive of tax with: "**The net sale amount is the sum of the quantity multiplied by the unit price in effect at the time of the order. It is exclusive of sales tax**."
- Click **OK**.



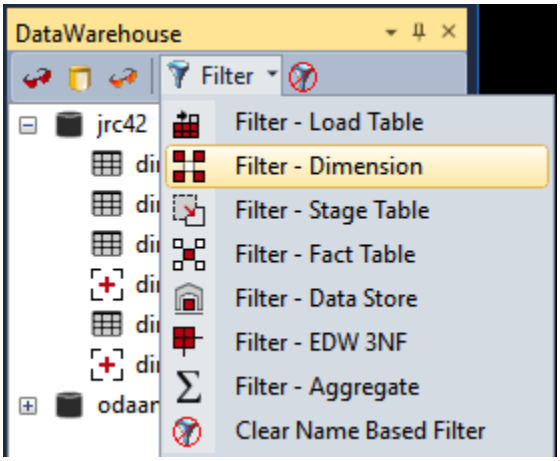
Again, validating would reflect that this table definition is different to the physical table but you can leave this for the moment as you are about to make some bigger changes.
Earlier in the tutorial, you created dimensions for:

- Customer (dim_customer)
- Product (dim_product)
- Order Date (dim_order_date)
- Ship Date (dim_ship_date)

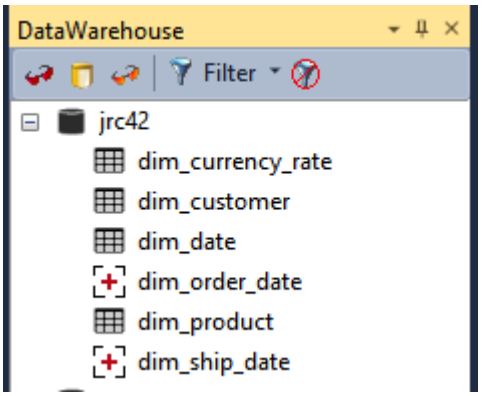
You should really use this to help categorize the information held within this new sales table. Stage tables, being the workhorse of your environment, are geared towards creating your business context for the reporting layer and making this the best place to link in these dimensions.
Once again, RED understands what you are looking to do and makes it easy for you to do it.

Add in the Order Date dimension to categorize the date you have in your transactions:

- In the data warehouse browser on the right, click on the **Filter – Dimension** button.



This will focus your attention on the dimensions in the data warehouse.



With your work pane still showing the columns of the stage table **stage_order_detail**, drag the **dim_order_date** view from the data warehouse browser onto the column **order_date** in the work pane.
WhereScape understands that you probably don't want to merge these two tables together, and since you dragged a dimension table, simply drops the unique key to the dimension table (dim_order_date_key) into the stage table beside the Business Key (order_date).

- Drag the **dim_customer** table over onto the **customer_code** column to link the tables and add in the **dim_customer_key** as your unique join to the dimension.
- Drag the **dim_ship_date** table over onto the **ship_date** column to link the two tables and add in the **dim_ ship_date _key** as your unique join to the dimension.
- Drag the **dim_product** table over onto the **product_id** column to link the two tables and add in the **dim_product_key** as your unique join to the dimension.

stage_order_detail Columns

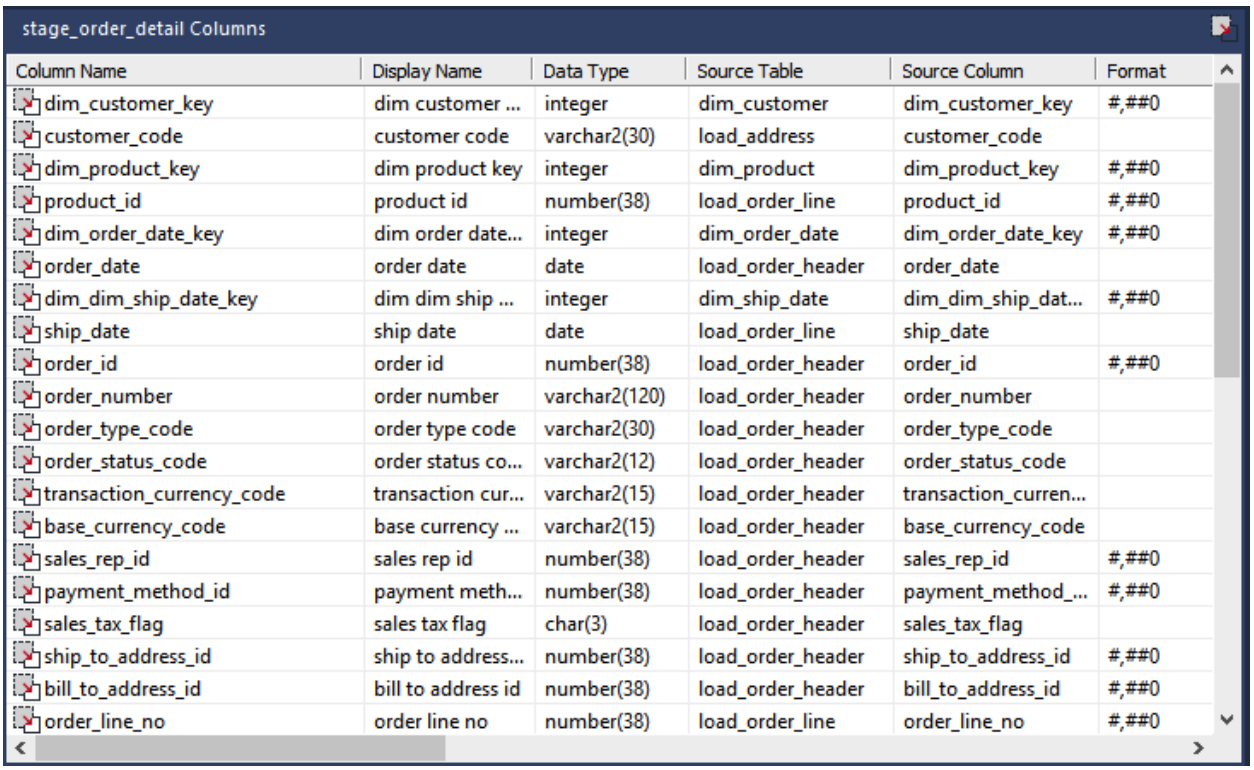
Column Name	Display Name	Data Type	Source Table	Source Column	Format
order_id	order id	number	load_order_header	order_id	#,##0
order_number	order number	varchar2(50)	load_order_header	order_number	
order_type_code	order type code	varchar2(10)	load_order_header	order_type_code	
order_status_code	order status co...	varchar2(4)	load_order_header	order_status_code	
transaction_currency_c...	transaction cur...	varchar2(5)	load_order_header	transaction_curren...	
base_currency_code	base currency ...	varchar2(5)	load_order_header	base_currency_code	
sales_rep_id	sales rep id	number	load_order_header	sales_rep_id	#,##0
payment_method_id	payment meth...	number	load_order_header	payment_method_...	#,##0
sales_tax_flag	sales tax flag	char(1)	load_order_header	sales_tax_flag	
customer_code	customer code	varchar2(10)	load_address	customer_code	
ship_to_address_id	ship to address...	number	load_order_header	ship_to_address_id	#,##0
bill_to_address_id	bill to address id	number	load_order_header	bill_to_address_id	#,##0
sold_to_address_id	sold to address...	number	load_order_header	sold_to_address_id	#,##0
dim_order_date_key	dim order date...	integer	dim_order_date	dim_order_date_key	#,##0
order_date	order date	date	load_order_header	order_date	

Now you have all the fields you need in your stage table to feed into your fact table. You have relabeled and added in calculations to fit with the business requirement as well as ensuring a single version of the truth by validating with your dimensions.

NOTE: Best practice is mixed as to whether business keys should remain in Fact tables or not. There is great benefit to the reporting products if the business keys are left in as this will eliminate the necessity to always join to the dimension tables to just get the business key in a report.

Finally, give this table a much more sensible order with the dimensional items first (followed by their business keys), the dates, and finally the granular detail:

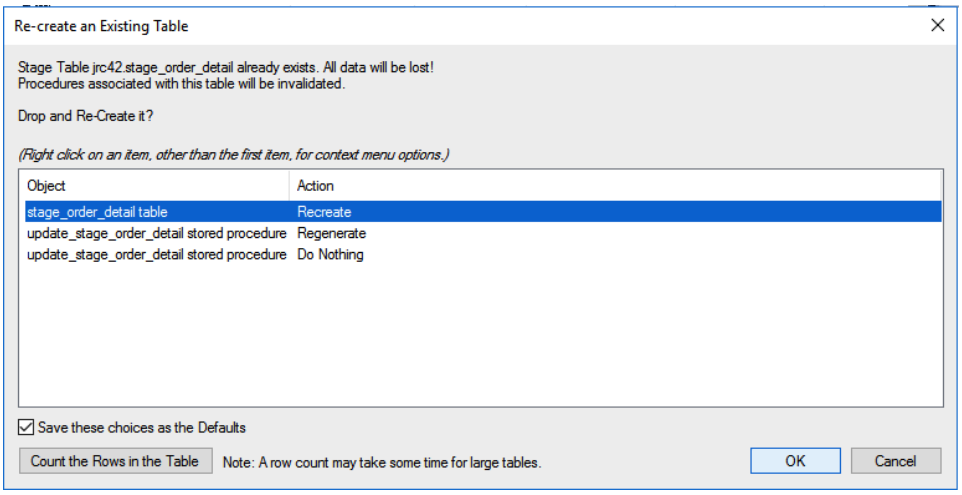
- Drag and drop your columns to re-arrange the table into the following order.



Column Name	Display Name	Data Type	Source Table	Source Column	Format
dim_customer_key	dim customer ...	integer	dim_customer	dim_customer_key	#,##0
customer_code	customer code	varchar2(30)	load_address	customer_code	
dim_product_key	dim product key	integer	dim_product	dim_product_key	#,##0
product_id	product id	number(38)	load_order_line	product_id	#,##0
dim_order_date_key	dim order date...	integer	dim_order_date	dim_order_date_key	#,##0
order_date	order date	date	load_order_header	order_date	
dim_dim_ship_date_key	dim dim ship ...	integer	dim_ship_date	dim_dim_ship_dat...	#,##0
ship_date	ship date	date	load_order_line	ship_date	
order_id	order id	number(38)	load_order_header	order_id	#,##0
order_number	order number	varchar2(120)	load_order_header	order_number	
order_type_code	order type code	varchar2(30)	load_order_header	order_type_code	
order_status_code	order status co...	varchar2(12)	load_order_header	order_status_code	
transaction_currency_code	transaction cur...	varchar2(15)	load_order_header	transaction_curren...	
base_currency_code	base currency ...	varchar2(15)	load_order_header	base_currency_code	
sales_rep_id	sales rep id	number(38)	load_order_header	sales_rep_id	#,##0
payment_method_id	payment meth...	number(38)	load_order_header	payment_method_...	#,##0
sales_tax_flag	sales tax flag	char(3)	load_order_header	sales_tax_flag	
ship_to_address_id	ship to address...	number(38)	load_order_header	ship_to_address_id	#,##0
bill_to_address_id	bill to address id	number(38)	load_order_header	bill_to_address_id	#,##0
order_line_no	order line no	number(38)	load_order_line	order_line_no	#,##0

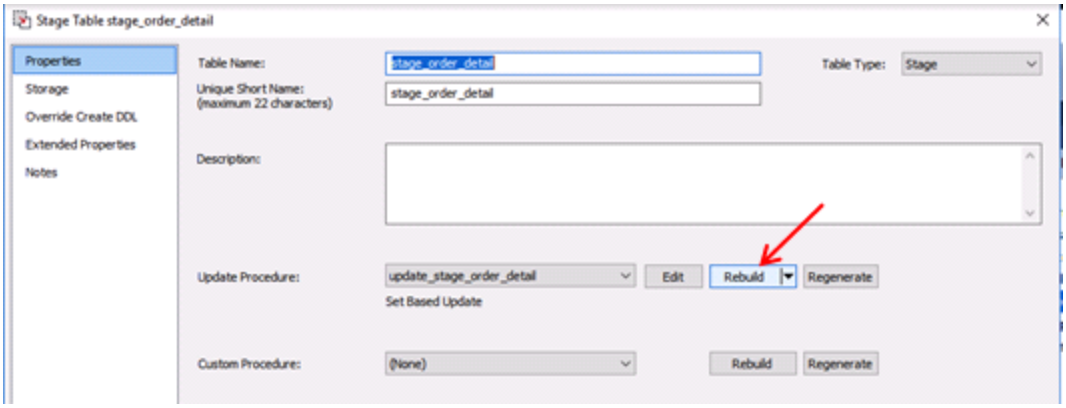
At this point, **Validate** will throw up many issues but you are still developing your prototype and so this time you will just rebuild the table, losing the data that is in it:

- Right-click on **stage_order_detail** and choose **Create (ReCreate)**.
- Choose simply **Recreate** to get your table right in the data warehouse.



Now you need to re-define how this table will be populated when you run an update:

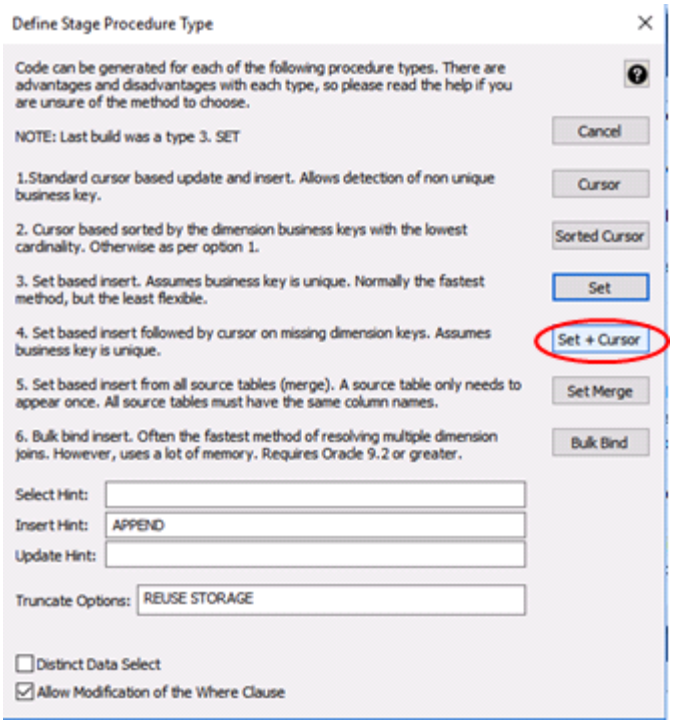
- Double-click on the **stage_order_detail** table to open up the Properties sheet and click **Rebuild** next to the **Update Procedure** field.



- RED will take you back through the process of updating this table, remembering your previous choice, but also adding in further prompts to understand how to update the new information in the table.
  It is IMPORTANT to note at this point that NOT all of the customer codes in the stage table exist in the customer dimension **dim_customer.** Therefore you will need to define your stage procedure type as **Set + Cursor**. By using this stage procedure type and setting the appropriate flag when building the stage procedure, you can request that any missing dimension records be automatically added.

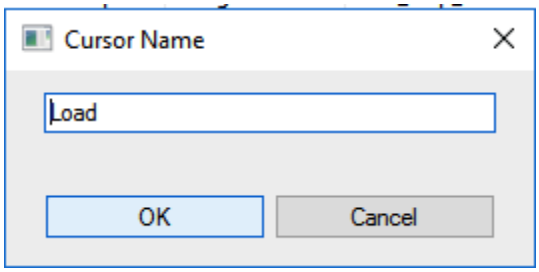
NOTE: At a later stage you will load in the customer details from the Sales Source System; and do a set merge on dim_customer to fill in the details for the added records.

- Click **Set + Cursor** on the **Define Stage Procedure Type**.

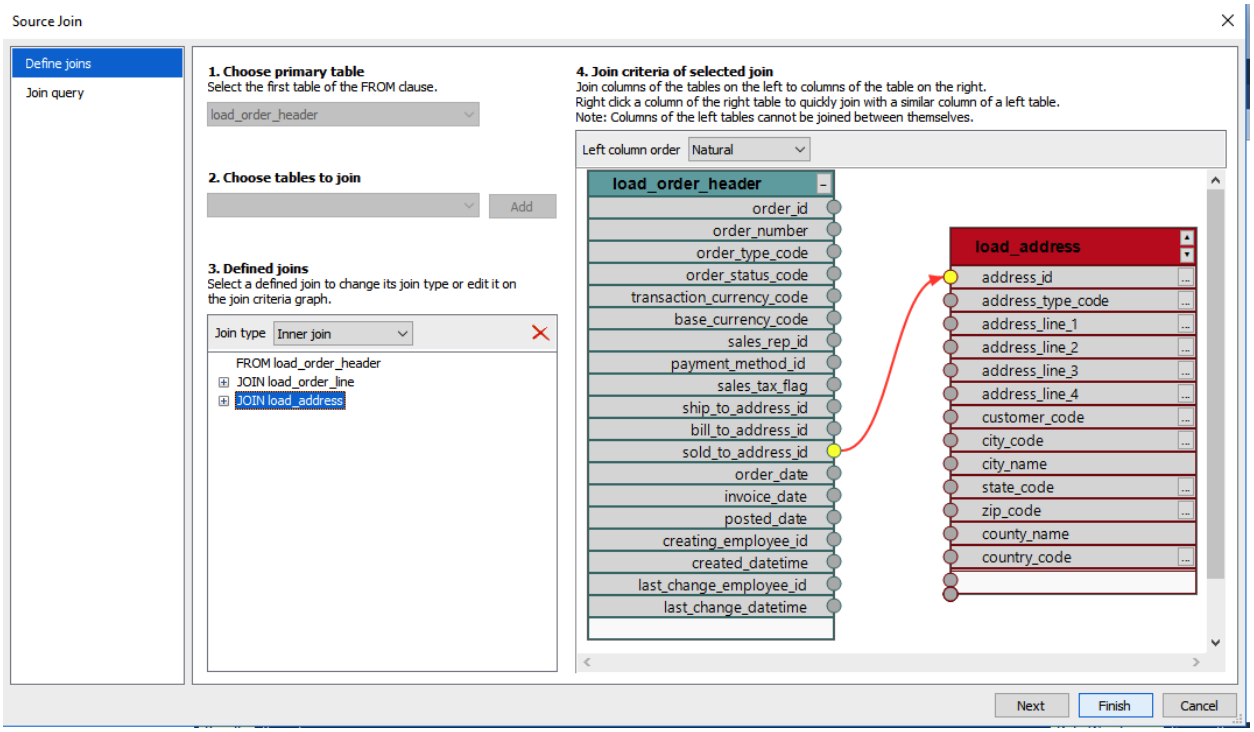


You still do not need any **Parameters** so click **OK** without selecting any.
On the **Provide Cursor Mapping dialog,** the original source tables and their joins are still set, so simply click **OK** without changing this relationship.

For the cursor name. The name “load” will be fine. Click OK.

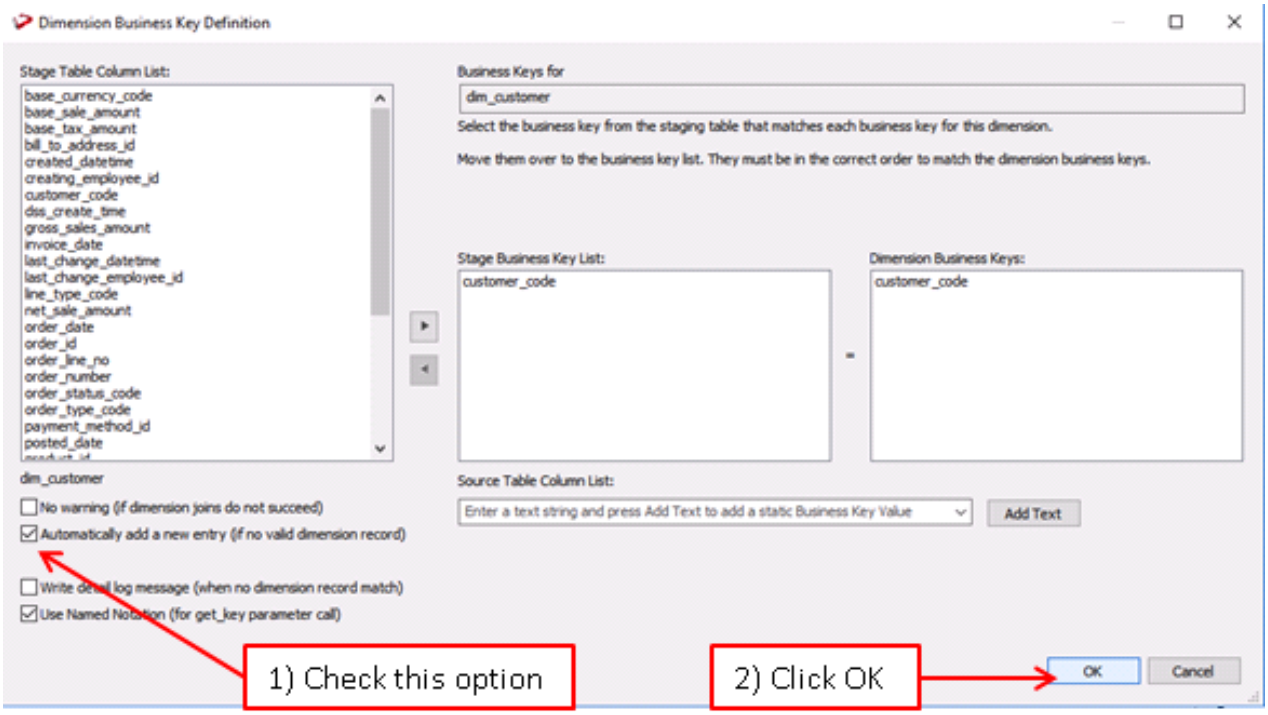


The "Source Join" screen will appear, allowing you to make any needed updates to the previously defined joins. You should be good, so click "Finish".



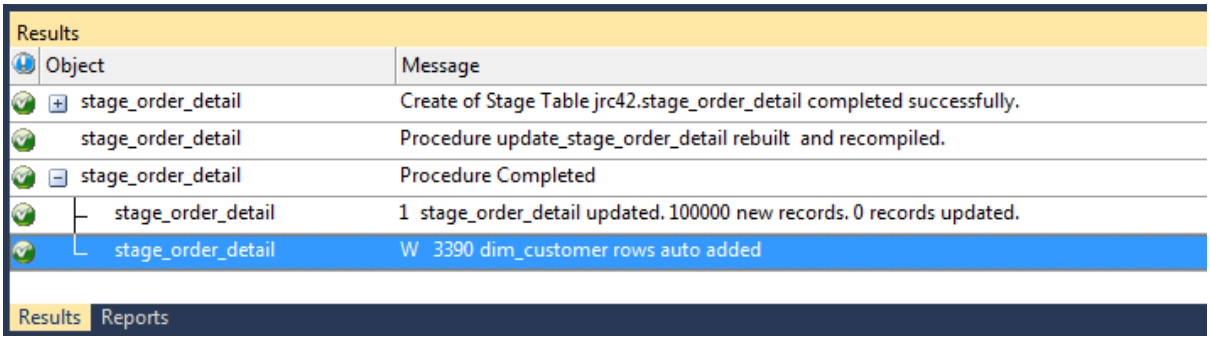
Now RED will prompt you for the information relating to the dimensions. You need to help RED understand what field(s) in the transactional data maps to each of the dimensions. This is the screen for the customer dimension but the rest are in the same style:
On the left, you have all of the transactional columns coming into the stage table.
On the right, you have the business key specified in the **Customer** Dimension.
Usually you would need to select **customer_code** from the left to add it to the **Stage Business Key List** indicating that **customer_code** is the business key for each customer in the transactional data and should map to the **customer_code** business key of the dimension. Since the names are the same, RED has auto-matched them for you.

- To request that any missing dimension records be automatically added, select the check box to **Automatically Add a New Entry (if no valid dimension record).**
- Note that it is only necessary to select this check box for the customer dimension.
- Click **OK.**



- The **product** map is already set because the column names are the same - click **OK** to accept this mapping for dim_product.
- The **order date** map is already set because the column names are the same – click **OK** to accept this mapping for dim_order_date.
- Finally, the **ship date** map is also already set because the column names are the same – click **OK** to accept this mapping for dim_ship_date.
- In the **Results** pane, notice that the update procedure has been rebuilt and recompiled successfully.

To see what happens when you update this table, right-click on **stage_order_detail** and select **Execute Update Procedure.**
In the **Results** pane, notice that the stage table has been updated and that records have been added to the customer dimension **dim_customer.**

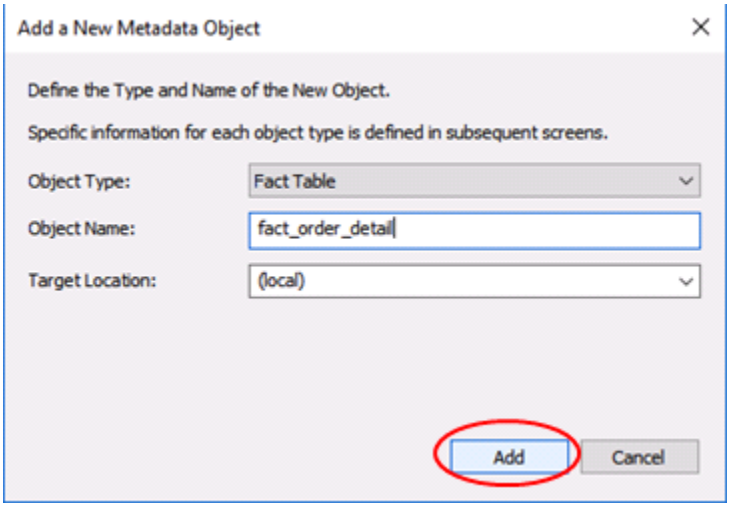


Right-click on **stage_order_detail** and select **View Data** to explore the information you have created.
Now you have your stage table ready to populate your fact table. Stage tables are the workhorses of the environment and are geared towards data preparation. They typically handle the changed data as opposed to all data but this is dependent on how you design your build.
Fact tables are the granular detail of the subject in your data warehouse. They have lookups to the structure within the organization and are usually incrementally updated as opposed to full refreshes.
Finally, to feed your stage table into your fact table, you need to set the context of your work pane this time to work with Fact Tables:

1. In the object explorer, double-click on the **Fact Table** group on the left pane – the work pane should list all fact tables currently defined in the metadata but it will display an empty screen because there are still no Fact tables at the moment.
2. Click the stage table filter to display only the stage tables in your warehouse.

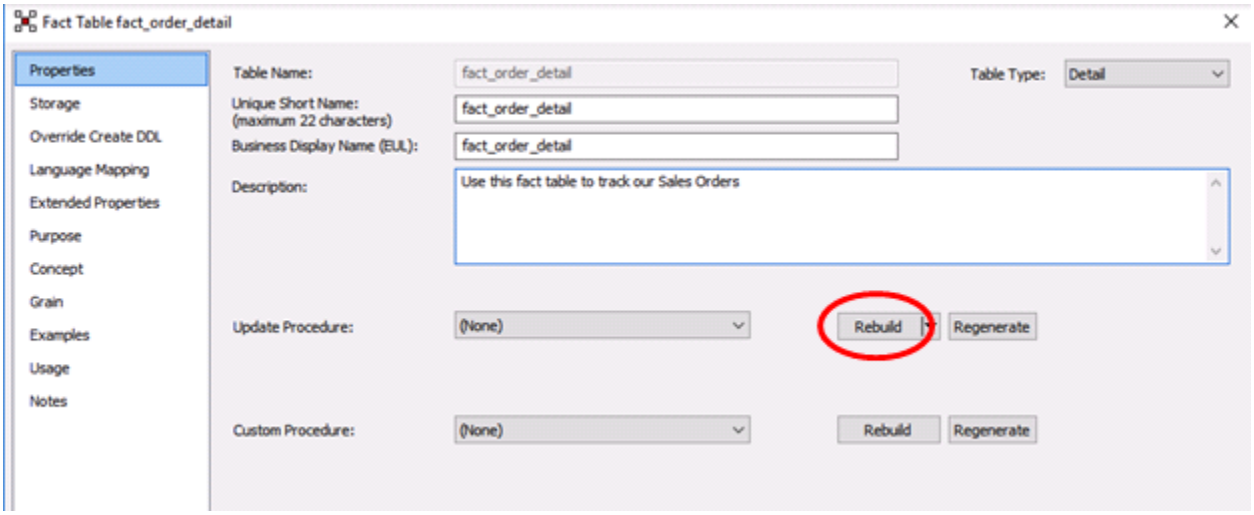
Drag the stage table **stage_order_detail** from the data warehouse browser onto the work pane to start the Fact Table wizard.

- Click **ADD** to accept the default generated name **fact_order_detail**.

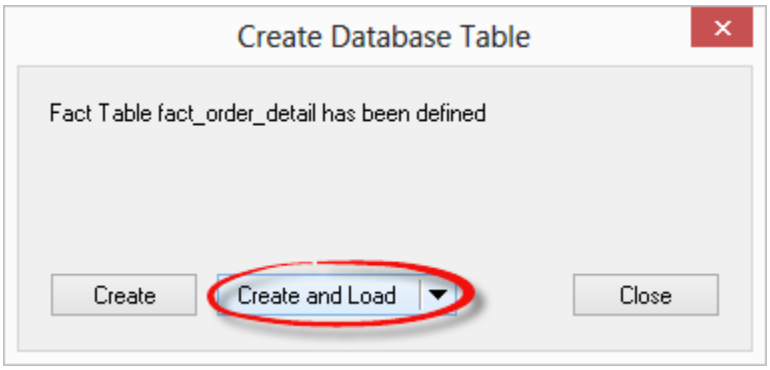


In the **Properties** dialog, notice again the large number of sections, which are once again, for documentation purposes.

- Set the **Description** as '**Use this fact table to track your Sales Orders**'.
- In order to refresh the table you need to build the Update Procedure, so click on **Rebuild** next to the **Update Procedure** field.

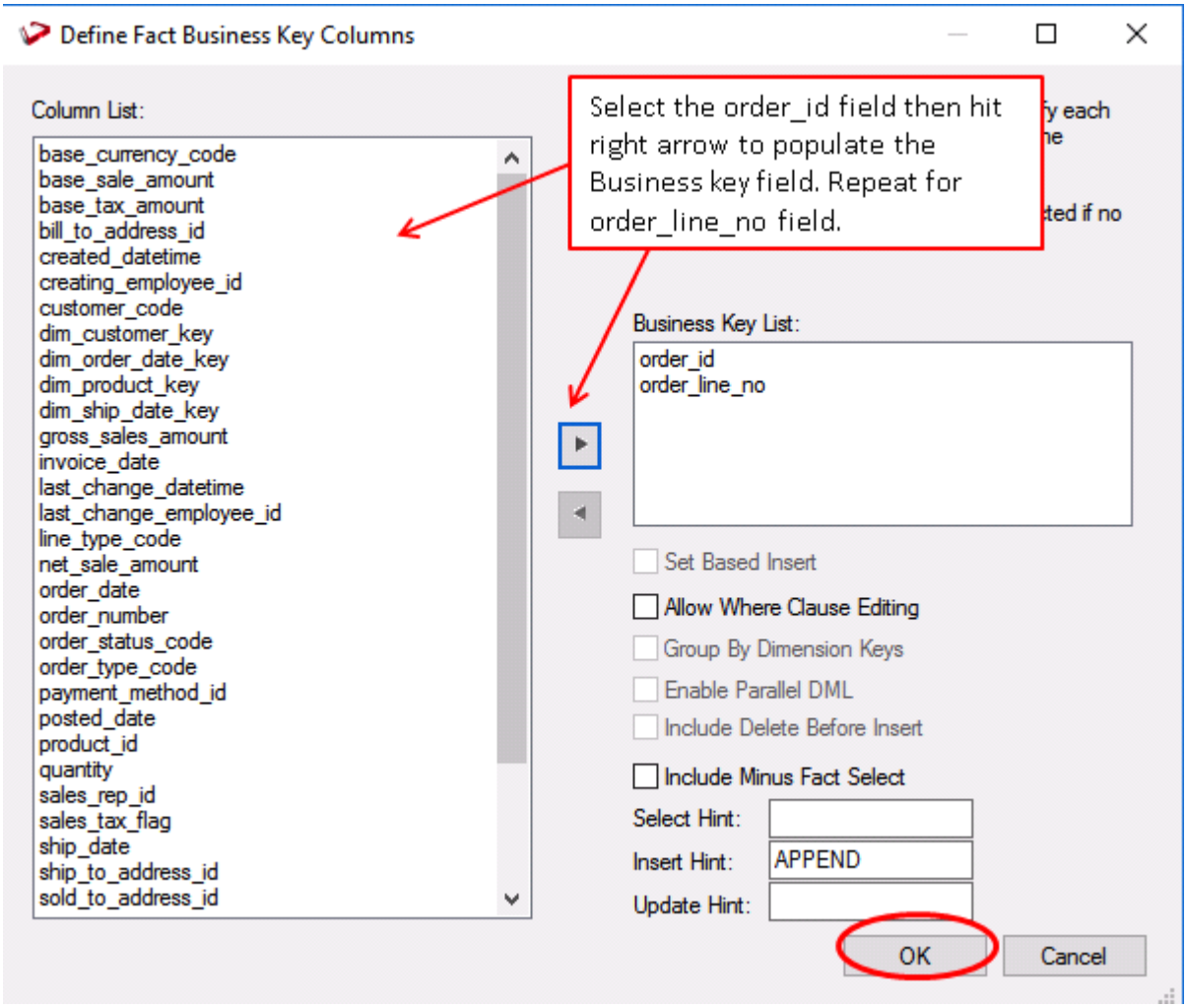


Click **Create and Load** to create the table in the database.



In the screen **Define Fact Business Key Columns** RED needs to understand which columns in the transactional data coming through uniquely identify each record i.e. the unique business key.

- Double-click on **order_id** and **order_line_no** to add them to the **Business Key List** and click **OK**.
- This is all that it needs as most of the real work has already been done in the stage table.

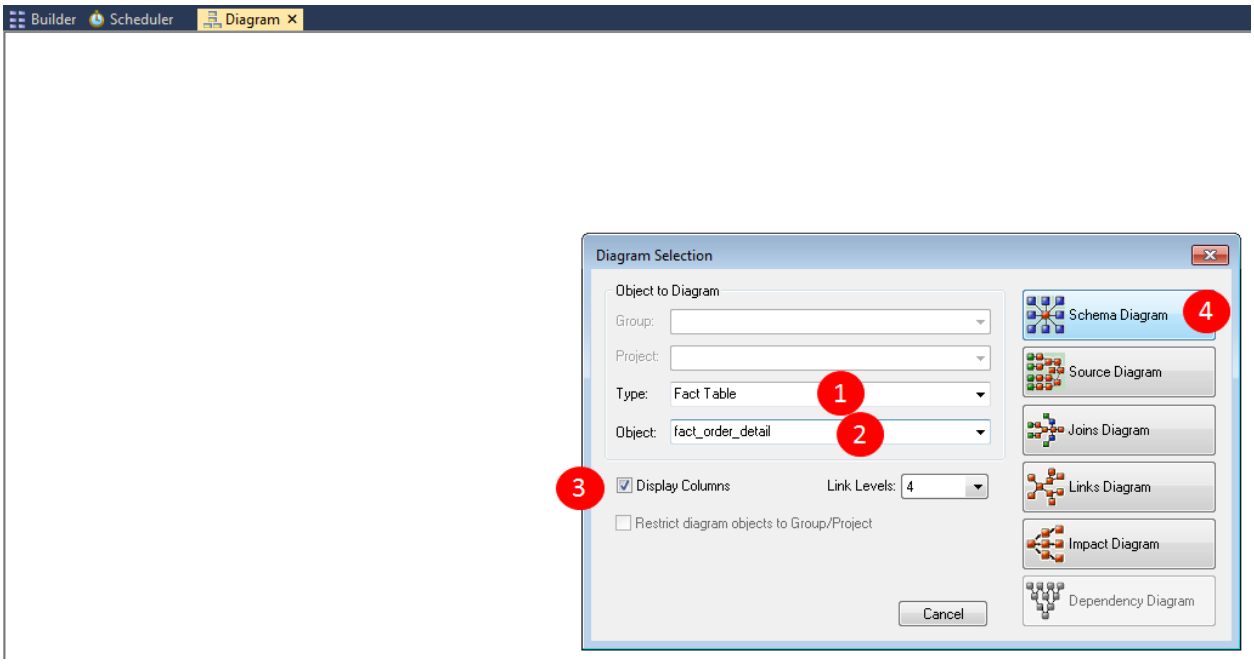


Notice in the **Results** pane, all records were new. (You should see 100,000 new records)
Right-click on **fact_order_detail** and choose **Execute Update Procedure** to re-run the update. This time there are no new records but all records had their timestamp updated to reflect their new update dates.
The fact table will default to checking all records first and adding in any new records it finds – all based on the business key you specified.
You now have a series of loads and stages feeding into your new fact table to meet the business requirement of analyzing their sales orders.

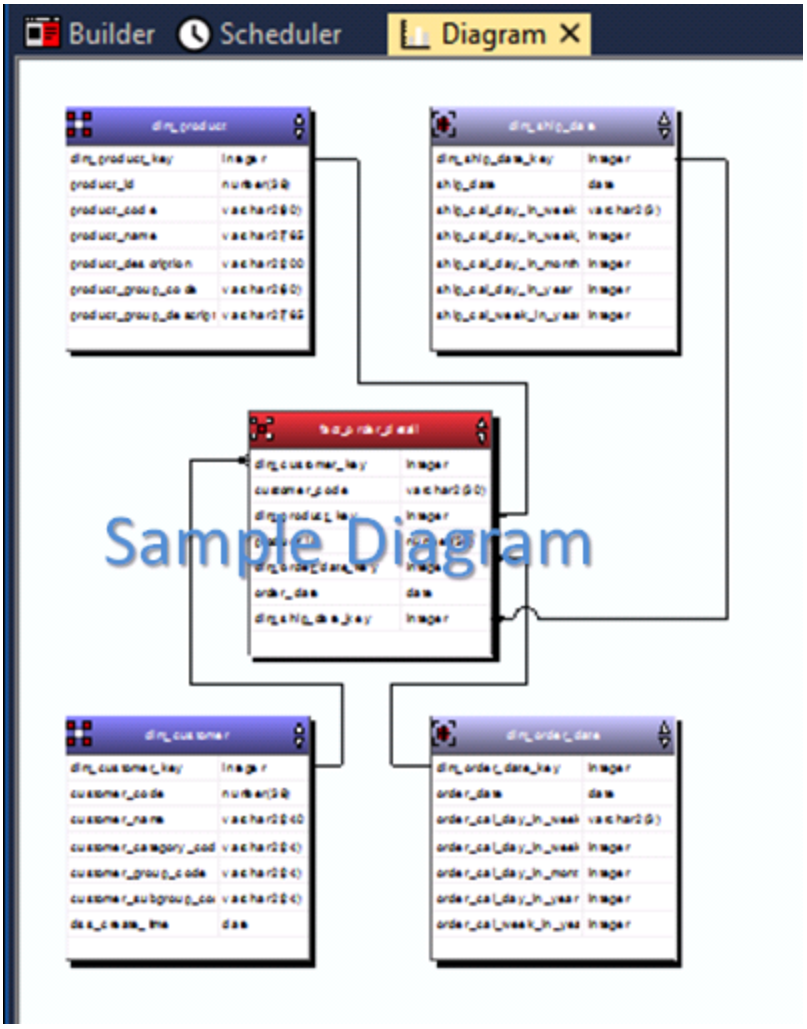
Congratulations! You have Instantiated a dimensional model in an Oracle Database and populated the data model using WhereScape RED.

View the star schema you created via a diagram, click on the **Diagram tab** and in the **Diagram Selection** dialog, choose **Fact Table** as the Type and **fact_order_detail** as the Object.

Click “Display columns” checkbox.
Click the **Schema Diagram** button to display.



The diagram should look something like:



Take a screenshot of your diagram and save it as a pdf file. You will have one more screenshot at the end of the assignment. You may combine both screenshots into a single pdf for uploading to Canvas.

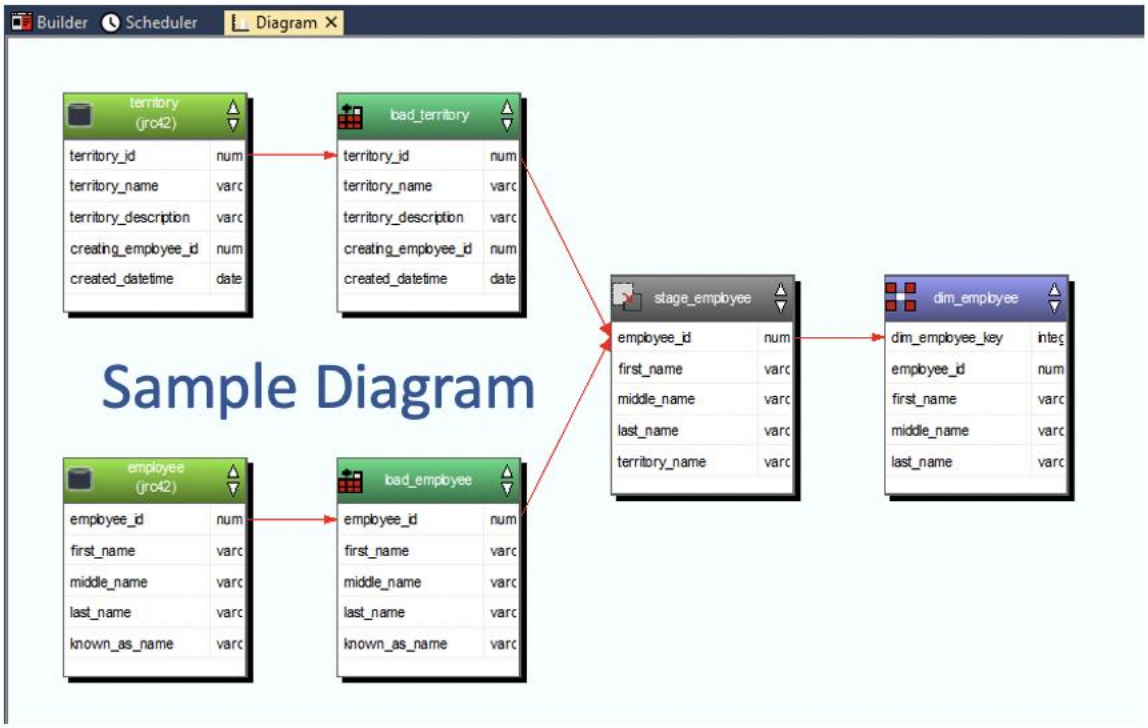
Stretch Portion of Tutorial (15 of the total 75 points)

Now let's apply the basics from what we have learned going through the tutorial. We will be building one more simple dimension table, but without screen shots and step-by-step instructions.

You will be creating a dimension table named DIM_EMPLOYEE, which will be built from Two tables found in the SALES schema: EMPLOYEE and TERRITORY. Remember the process for building a dimension table in RED. Start by clicking on the "Builder" tab to get out of the Diagram tab.

1) If the needed tables have not been pulled in as LOAD tables, do so.
2) Use a STAGE table to join the two tables (use a simple join on territory_id) and only pull in territory_name from the territory table. The update procedure type will be a SET. Don't forget to execute your update procedure.

3) Build your dimension table. Should be type “Normal”, which is a SCD type 1. The update procedure will also be a type “SET”. Remember the “business key” is the same as a natural key. In this case it is employee_id.
4) Don’t forget to verify you were successful in loading your DIM_EMPLOYEE table.
5) Finally, similar to the schema diagram you created for the fact table, create a “source diagram” for your new DIM_EMPLOYEE table. Take a screen shot of this diagram and upload it to Canvas, along with your fact table schema diagram. The source diagram should look similar to this:



Congratulations! You have completed the Assignment 4 WhereScape RED tutorial. Be sure to upload your pdf containing the screen shots of the two diagrams to Canvas before the assignment deadline.

You will be graded on the two final diagrams and the presence of the related tutorial tables (with data) in your netid schema.